

Вичерпний довідник із JSBSim

Архітектура, аеродинаміка та моделювання літальних апаратів —
зсередини

Практичний інженерний посібник, що охоплює мову конфігурування XML у JSBSim, рівняння руху, методи інтегрування, функційно-табличну аеродинамічну систему, підсистему силової установки, компоненти системи керування польотом і робочий процес створення аеродинамічних таблиць даних, отриманих з CFD, для власної моделі літального апарата.



Розділ 1: Вступ до JSBSim	13
1.1 Що охоплює цей документ	13
1.2 Чим JSBSim відрізняється від типового рушія симуляції	13
1.3 Домовленості, прийняті в цьому документі	13
1.4 Українсько-англійський словник ключових термінів	14
Розділ 2: Архітектура та хід виконання	16
2.1 Порядок виконання моделей	16
2.2 Цикл Run у псевдокоді	16
2.3 Модулі побіжно	17
Розділ 3: Системи координат, осі та одиниці	18
3.1 Конструктивна система (геометрія в XML)	18
3.2 Зв'язана із тілом (зв'язана) система	18
3.3 Швидкісна (аеродинамічна) та зв'язана зі стійкістю системи	18
3.4 Локальна NED та ECI системи	19
3.5 Домовленості про знаки	19
3.6 Одиниці	19
Розділ 4: XML літального апарата, розділ за розділом	20
4.1 Корінь: fdm_config	20
4.2 fileheader: авторство та походження	20
4.3 metrics: геометрія літального апарата	20
4.4 mass_balance: вага, CG, інерція	21
4.5 ground_reactions: шасі, лижі та контакти	22
4.6 external_reactions: додаткові сили	22
4.7 propulsion: двигуни, баки, рушії	23
4.8 flight_control: формування команд	24
4.9 autopilot та system: той самий набір, інша область	24
4.10 aerodynamics: ядро моделі	25
4.11 input та output	25
Розділ 5: Аеродинаміка — поглиблений розгляд	26
5.1 Шість осей та функційно-таблична модель	26
5.2 Як обчислюється окрема функція	26
5.3 Мова функцій	26
5.4 Таблиці: 1-вимірні, 2-вимірні, 3-вимірні та вищих розмірностей	27
5.5 Властивості, що часто використовуються в аеродинаміці	28
5.6 Знерозмірнення вручну	28
5.7 Розв'язаний приклад: момент крену від кутової швидкості крену	29

5.8 Статичні похідні — отримання їх із CFD	29
5.9 Динамічні похідні (демпфування) — CFD з рухом	30
5.10 Похідні керованості	30
5.11 Вплив екрана та число Рейнольдса	31
5.12 Звалювання та гістерезис звалювання	31

Розділ 6: Рівняння руху зсередини **32**

6.1 Вектор стану	32
6.2 Кінематика орієнтації через кватерніон	32
6.3 Поступальна динаміка в інерціальній системі	32
6.4 Обертальна динаміка	33
6.5 Збирання повної сили та моменту	33
6.6 Наземне тертя як задача з обмеженнями	33
6.7 Балансування	33
6.8 Модель атмосфери	34
6.9 Вітри та турбулентність	34
6.10 Дерево властивостей	34

Розділ 7: Побудова власного літального апарата, крок за кроком **35**

7.1 Крок 1: Структура каталогу	35
7.2 Крок 2: Каркас XML	35
7.3 Крок 3: Геометрія, маса та шасі	35
7.4 Крок 4: Силова установка	36
7.5 Крок 5: Система керування польотом	36
7.6 Крок 6: Аеродинаміка з CFD	36
7.7 Крок 7: Початкові умови та скрипт димового тесту	37
7.8 Крок 8: Цикл ітерацій	37

Розділ 8: Робочий процес CFD-to-JSBSim **39**

8.1 Вибір розв'язувача CFD	39
8.2 Рекомендована матриця випадків	39
8.3 Метод вимушених коливань для похідних демпфування	40
8.4 Квазістатичний метод (швидший)	40
8.5 Виокремлення та табулювання	40
8.6 Допоміжний скрипт Python: CSV із CFD → XML JSBSim	40
8.7 Валідація: від чисел CFD до відчуття польоту	41

Розділ 9: Довідкові додатки **42**

9.1 Шпаргалка з дерева властивостей	42
9.2 Поширені компоненти FCS	42
9.3 Перехресний довідник типів двигунів	43
9.4 Карта вихідного коду	43

9.5 Бібліографія та джерела	43
9.6 Підсумкові міркування	44
Розділ 10: Візуальні діаграми циклу FDM	45
10.1 Цикл симуляції верхнього рівня	45
10.2 Потік даних між моделями за один такт	46
10.3 Схема зв'язаної, вітрової та конструктивної систем координат	47
10.4 Потік від аеродинамічних осей до сил	47
Розділ 11: Силова установка — довідник за типами двигунів	48
11.1 Поршневі двигуни	48
11.2 Турбіна (турбореактивний/турбовентиляторний двигун)	48
11.3 Турбогвинтовий двигун	49
11.4 Ракетні двигуни	49
11.5 Електричні та безколекторні двигуни постійного струму	50
11.6 Несучий гвинт вертольота	50
11.7 Повітряні гвинти	50
11.8 Паливні баки	51
Розділ 12: Компоненти системи керування польотом — довідник	52
12.1 Суматор / Чистий коефіцієнт підсилення / Масштабування аероповерхні	52
12.2 Фільтри	52
12.3 Інтегратор	53
12.4 ПІД-регулятор	53
12.5 Кінематичний компонент, зона нечутливості, перемикач	53
12.6 Огортання датчика та привода	53
Розділ 13: Атмосфера, вітри та Земля	55
13.1 Стандартна атмосфера ISA 1976	55
13.2 Вітри та зсув вітру	55
13.3 Пориви (дискретні)	55
13.4 Мікропорив	55
13.5 Інерціальна модель: гравітація та обертання Землі	56
Розділ 14: Орієнтація через кватерніони — вступ до коду	57
14.1 Означення та домовленості	57
14.2 Кінематичне рівняння	57
14.3 Перенормування	57
14.4 Видобування кутів Ейлера	58
Розділ 15: Повний розібраний приклад: простий літак	59
15.1 Заголовок та довідкові дані	59
15.2 Маса, центрування та інерція	59
15.3 Шасі	60

15.4 Силова установка	61
15.5 Система керування польотом	62
15.6 Аеродинаміка (репрезентативні значення з CFD)	63
Розділ 16: Початкові умови, сценарії та скидання	67
16.1 Початкові умови (файли скидання)	67
16.2 Сценарії (подієво-орієнтовані випробування)	67
Розділ 17: Процес валідації	69
17.1 Крок 1: діагностика балансування	69
17.2 Крок 2: розімкнені перехідні характеристики	69
17.3 Крок 3: вилучення лінійної моделі	69
17.4 Крок 4: перевірка експлуатаційного діапазону	70
17.5 Крок 5: перевірка пілотажних властивостей	70
17.6 Крок 6: порівняння з льотними випробуваннями	70
Розділ 18: Поширені помилки та способи їх діагностики	71
Розділ 19: API мовою Python	72
19.1 Швидкий старт	72
19.2 Читання та запис властивостей	72
19.3 Балансування та лінеаризація	72
19.4 Пакетні параметричні дослідження	72
Розділ 20: Розширений довідник властивостей	73
20.1 Час	73
20.2 Атмосфера та вітри	73
20.3 Положення	73
20.4 Орієнтація	73
20.5 Швидкості	73
20.6 Аеродинамічний стан	74
20.7 Сили та моменти (зв'язана система)	74
20.8 Прискорення	74
20.9 Команди та положення FCS	74
20.10 Силова установка	75
20.11 Керування симуляцією	75
Розділ 21: Математика для динаміки польоту	77
21.1 Вектори у тривимірному просторі	77
21.2 Матриці обертання в $SO(3)$	77
21.3 Аерокосмічна послідовність Ейлера 3-2-1	77
21.4 Тензор інерції	78
21.5 Перетворення тензора при обертанні	78
Розділ 22: Кватерніони — теорія і практика	79

22.1 Означення: H , алгебра кватерніонів	79
22.2 Добуток Гамільтона	79
22.3 Одиничні кватерніони та обертання	79
22.4 Кінематичне рівняння: $\dot{q} = \frac{1}{2} q \otimes \omega$	79
22.5 Кватерніони проти кутів Ейлера проти DCM	80
Розділ 23: Чисельне інтегрування — теорія	81
23.1 Однокрокові методи	81
23.2 Багатокрокові методи: Adams-Bashforth	81
23.3 Області стійкості та вибір кроку	82
Розділ 24: Динаміка твердого тіла Ньютона-Ейлера	83
24.1 Інерціальна форма (закони Ньютона)	83
24.2 Форма у зв'язаній системі через теорему про перенесення	83
24.3 Урахування обертання планети	83
24.4 Спрощення, специфічні для повітряних суден	83
Розділ 25: Основи механіки рідин і газів	84
25.1 Закони збереження	84
25.2 Безрозмірні числа	84
25.3 Пограничні шари	84
25.4 Відрив, зрив і закритичні режими	85
Розділ 26: Теорія виникнення піднімальної сили	86
26.1 Бернуллі — не відповідь (а рівний час проходження хибний)	86
26.2 Кутта-Жуковський: правильна відповідь	86
26.3 Теорія тонкого профілю	86
26.4 Крила скінченного розмаху: несна лінія Прандтля	86
26.5 Поправка на стисливість	87
Розділ 27: Сила опору — повний розклад	88
27.1 Поверхневе тертя (шкідливий опір)	88
27.2 Опір форми (тиску)	88
27.3 Індуктивний опір (зумовлений піднімальною силою)	88
27.4 Опір інтерференції	88
27.5 Хвильовий опір	88
27.6 Параболічна полярна опору та максимальна аеродинамічна якість L/D	88
Розділ 28: Стійкість і керованість — повна таблиця похідних	90
28.1 Угоди про знаки та означення	90
28.2 Повна таблиця похідних	90
28.3 Нейтральна точка та запас статичної стійкості	90
28.4 Маневрена точка	90
Розділ 29: Власні рухи повітряного судна та лінеаризація	91

29.1 Лінеаризація навколо балансування	91
29.2 Форма у просторі станів	91
29.3 Поздовжні рухи	91
29.4 Бічно-курсіві рухи	91
Розділ 30: Аеродинаміка профілю	93
30.1 Сімейство профілів NACA	93
30.2 Типові характеристики	93
30.3 Вплив числа Рейнольдса на БПЛА	93
30.4 Критичне число Маха	93
Розділ 31: Методи CFD — теорія і практика	94
31.1 Панельні методи	94
31.2 Вихрова решітка	94
31.3 Моделі турбулентності RANS	94
31.4 LES, DES, гібридні методи	94
31.5 Валідаційні задачі	94
Розділ 32: Вимушені коливання та похідні демпфування	95
32.1 Постановка задачі	95
32.2 Зведена частота	95
32.3 Виділення похідних	95
Розділ 33: Геодезія та еліпсоїд WGS-84	96
33.1 Визначальні сталі WGS-84	96
33.2 Геодезична та геоцентрична широта	96
33.3 Означення висоти	96
33.4 Радіуси кривини	97
Розділ 34: Перетворення координат докладно	98
34.1 Геодезичні → ECEF (замкнена форма)	98
34.2 ECEF → геодезичні (ітерація Боврінга)	98
34.3 Поворот ECEF → ECI	98
34.4 ECEF → NED у точці (ϕ_0, λ_0)	98
34.5 NED → зв'язана (3-2-1 Тейта-Брайана)	98
Розділ 35: Обертання Землі та сила тяжіння	99
35.1 Кутова швидкість обертання Землі та фіктивні сили	99
35.2 Сферична сила тяжіння (проста)	99
35.3 Нормальна сила тяжіння за Сомільяною (поверхня WGS-84)	99
35.4 Збурення сили тяжіння J2	99
Розділ 36: Магнітне поле та навігація	100
36.1 Світова магнітна модель (WMM 2025)	100
36.2 GPS у WGS-84	100

36.3 Відстань по великому колу за гаверсинусом	100
Розділ 37: Системи відліку часу в аерокосмічній галузі	101
Розділ 38: Стандартна атмосфера докладно	102
38.1 Опорні значення на рівні моря	102
38.2 Структура шарів (від 0 до 86 км)	102
38.3 Похідні величини	102
Розділ 39: Вітер, турбулентність і пориви	104
39.1 Профіль вітру поблизу поверхні	104
39.2 Дискретні пориви (MIL-F-8785C, 1-косинус)	104
39.3 Неперервна турбулентність — СЩП за Драйденом	104
39.4 Мікропорив (Вікрой, Vicroy)	104
Розділ 40: Теорія силової установки	105
40.1 Поршневі двигуни — цикл Отто	105
40.2 Турбіна — цикл Брайтона	105
40.3 Ракета — $F = \dot{m} V_e + (p_e - p_a)A_e$	105
40.4 Електродвигуни — BLDC	105
Розділ 41: Теорія повітряного та несного гвинта	106
41.1 Повітряний гвинт — імпульсна теорія + елемент лопаті	106
41.2 Р-фактор та ефекти, спричинені гвинтом	106
41.3 Гвинти зі сталою частотою обертання	106
41.4 Несний гвинт вертольота	106
Розділ 42: Обробка сигналів для САК	108
42.1 Фільтри в неперервному часі	108
42.2 Дискретизація: білінійне (Тастіна) перетворення	108
42.3 Дискретизація ПІД-регулятора	108
42.4 Правила налаштування ПІД-регулятора	108
Розділ 43: Лінійна задача доповнюваності та контактне тертя	109
43.1 Формулювання LCP	109
43.2 LCP шасі в JSBSim	109
Розділ 44: Алгоритм балансування зсередини	110
44.1 FGTrimAxis: елементарна комірка	110
44.2 Режими балансування	110
44.3 Зовнішній цикл та збіжність	110
Розділ 45: Контрольні приклади верифікації NASA NESC 2015	111
45.1 Приклади атмосферного польоту	111
Розділ 46: Повне дерево властивостей	112
46.1 simulation/	112

46.2 ic/	112
46.3 position/, attitude/	113
46.4 velocities/	113
46.5 aero/, atmosphere/	113
46.6 forces/, moments/, accelerations/	114
46.7 fcs/, gear/	114
46.8 propulsion/, inertia/, metrics/	114
Розділ 47: Мова функцій — повний довідник операторів	115
Розділ 48: Процедури верифікації та валідації	116
48.1 Ієрархія V&V	116
48.2 Контрольний список верифікації (на кожен реліз)	116
48.3 Валідація за льотними даними	116
48.4 Пілотажні характеристики (MIL-F-8785C / MIL-HDBK-1797)	116
Розділ 49: Конвеєр від CFD до FDM: від OpenFOAM до моделі JSBSim	118
49.1 Навіщо будувати аеродинамічну модель з CFD	118
49.2 Що потрібно JSBSim: перелік необхідних коефіцієнтів	118
49.3 Складова (покомпонентна) модель і квазістаціонарне припущення	119
49.4 Матриця плану експерименту	119
49.5 Знерозмірювання: міст між CFD і JSBSim	119
49.6 Дорожня карта Частини III	120
Розділ 50: Геометрія та побудова сітки для літаків в OpenFOAM	121
50.1 Герметична геометрія та виділення особливостей	121
50.2 Розрахункова область	121
50.3 Фонова сітка: blockMesh	121
50.4 Тілоконформна сітка: snappyHexMesh	122
50.5 Пристінкове розрізнення та y^+	122
50.6 Незалежність від сітки	122
50.7 Кермові поверхні та рухома геометрія	123
Розділ 51: Статичні аеродинамічні коефіцієнти з OpenFOAM	124
51.1 Вибір розв'язувача	124
51.2 Граничні умови	124
51.3 Задання кута атаки та кута ковзання	124
51.4 Функційний об'єкт forceCoeffs	125
51.5 Збіжність та осереднення	125
51.6 Виконання розгортки	126
Розділ 52: Динамічні похідні стійкості з OpenFOAM	127
52.1 Похідні та їхній зміст	127
52.2 Безрозмірна кутова швидкість та зведена частота	127

52.3 Метод 1: стаціонарне обертання в неінерціальній системі	127
52.4 Метод 2: вимушені коливання	128
52.5 Метод 3: вертикальне переміщення та індикальний відгук	128
52.6 Бічно-коліїні похідні	129
52.7 Порівняння методів	129
Розділ 53: Зведення даних CFD у XML літака JSBSim	130
53.1 Вибір системи осей та точки зведення	130
53.2 Головна таблиця відображення	130
53.3 Ключове розуміння: значення I Є похідною	130
53.4 Статичні криві як таблиці	131
53.5 Прирости від керм	131
53.6 Умови знаків та осей: підводні камені	131
53.7 Розв'язаний приклад: повний набір осей, отриманий з CFD	132
Розділ 54: Автоматизація конвеєра від OpenFOAM до JSBSim	134
54.1 Шаблонування випадків за матрицею умов	134
54.2 Розбір коефіцієнтів	134
54.3 Видача таблиць JSBSim	135
54.4 Інструментарій та відтворюваність	135
Розділ 55: Верифікація: замикання циклу CFD-JSBSim-політ	136
55.1 Статичні перевірки: чи балансується там, де каже CFD	136
55.2 Динамічні перевірки: лінеаризуйте та порівняйте моди	136
55.3 Порівняння з незалежними даними	136
55.4 Застереження щодо екстраполяції	137
55.5 Цикл ітерацій	137
Розділ 56: Джерела аеродинамічних даних і сходинки достовірності	139
56.1 Сходинки достовірності	139
56.2 Зчитування даних із POH або сертифіката типу	139
56.3 Digital DATCOM	139
56.4 Методи вихрової решітки та панельні методи	140
56.5 Аеродинамічна труба та льотні випробування	140
56.6 Поєднання джерел і подібні літаки	140
Розділ 57: Aeromatic: первинне створення літаючої моделі	141
57.1 Що ви йому подаєте	141
57.2 Що він створює	141
57.3 Припущення та обмеження	141
57.4 Робочий процес уточнення та поширені пастки	141
Розділ 58: Достовірність маси, центрування, інерції та палива	142
58.1 Блок mass_balance	142

58.2 Оцінювання тензора інерції	142
58.3 Точкові маси та компонування завантаження	142
58.4 Паливні баки та зміщення CG у польоті	142
Розділ 59: Достовірність силової установки на практиці	144
59.1 Поршневі двигуни	144
59.2 Турбіни: турбореактивні та турбовентиляторні двигуни	145
59.3 Турбогвинтові, електричні та ракетні двигуни	145
59.4 Паливна система та узгодження характеристик	145
Розділ 60: Повітряні гвинти, рушії та регулятор постійних обертів	146
60.1 Файл повітряного гвинта	146
60.2 Коефіцієнти, відносна хода та число Маха	146
60.3 Регулятор постійних обертів	146
60.4 Р-фактор, напрям, авторотація гвинта та реверс	147
Розділ 61: Шасі та керування рухом по землі	148
61.1 Закон сили стійки	148
61.2 Тертя, керування поворотом і гальма	148
61.3 Поверхня землі	149
61.4 Налаштування стабільної, реалістичної поведінки на землі	149
Розділ 62: Системи керування польотом на практиці	150
62.1 Каталог компонентів	150
62.2 Шлях керування, від початку до кінця	150
62.3 Механічне, бустерне керування та fly-by-wire	151
62.4 Достовірність приводів	151
Розділ 63: Покращення стійкості, автопілоти та наведення	152
63.1 Покращення стійкості	152
63.2 Канали автопілота з ПІД-регуляторами	152
63.3 Налаштування контурів	152
63.4 Наведення та навігація	153
Розділ 64: Датчики, системи та моделювання відмов	154
64.1 Недосконалі датчики	154
64.2 Довільні підсистеми	154
64.3 Зовнішні реакції та плавучість	154
64.4 Введення відмов	155
Розділ 65: Моделювання великих кутів атаки, звалювання та штопора	156
65.1 Моделювання звалювання	156
65.2 Гістерезис звалювання	156
65.3 Штопор та зрив у некерований режим	156
65.4 Інші крайові ефекти	157

Розділ 66: Сценарії для автоматизованих випробувань, налаштування та system ID	158
66.1 Анатомія сценарію запуску	158
66.2 Дії set: крок, лінійна зміна, експонента	158
66.3 Вивід, notify та пакетні робочі процеси	159
66.4 Випробування збуреннями та турбулентністю	159
Розділ 67: Узгодження льотних і пілотажних характеристик	160
67.1 Балансування та точки характеристик	160
67.2 Цільові пілотажні характеристики	160
67.3 Енергетичні методи та цикл ітерацій	160
Розділ 68: Інтеграція з FlightGear та зовнішніми симуляторами	162
68.1 FlightGear	162
68.2 Сокети, рідні протоколи та інші рушії	162
68.3 Поширені пастки інтеграції	162
Розділ 69: Зведений контрольний список достовірності та поширені пастки	163
69.1 Геометрія, маса та центрування	163
69.2 Аеродинаміка	163
69.3 Силова установка, шасі та керування	163
69.4 Перевірка та інтеграція	163
Розділ 70: Додаткова література та авторитетні джерела	165
70.1 Аеродинаміка та теорія профілю	165
70.2 Динаміка польоту та керування	165
70.3 CFD та моделювання турбулентності	165
70.4 Атмосфера та геодезія	165
70.5 Силові установки	166
70.6 Чисельні методи	166
70.7 Ресурси щодо JSBSim	166
70.8 OpenFOAM та робочий процес CFD→FDM	166
70.9 Побудова FDM, джерела даних і валідація	167
Розділ 71: Глосарій та покажчик позначень	168

Розділ 1: Вступ до JSBSim

JSBSim — це багатоплатформна об'єктно-орієнтована модель динаміки польоту (FDM) з відкритим кодом, написана мовою C++, що слугує фізичним ядром для широкого спектра аерокосмічних застосунків: FlightGear, проєкту Antoinette на Unreal Engine, симуляцій Software-in-the-Loop для ArduPilot та PX4, середовищ навчання з підкріпленням на кшталт gym-jsbsim, а також академічних досліджень, на які посилаються тисячі разів. Вона реалізує нелінійну симуляцію твердого тіла з шістьма ступенями свободи з точною моделлю Землі (еліпсоїд WGS-84, обертання, ефект Коріоліса), стандартною атмосферою ISA 1976, налаштовуваним вітром і турбулентністю, силовою установкою (поршневою, турбінною, турбогвинтовою, ракетною, електричною, гвинтокрилою), реакціями опори та повністю скриптованою системою керування польотом на основі XML.

1.1 Що охоплює цей документ

Цей довідник побудований як занурення в JSBSim згори донизу. Ми починаємо з архітектури та циклу виконання, потім проходимо кожен розділ XML літального апарата, далі поглиблено розглядаємо аеродинаміку (математичний апарат, таблиці та домовленості) і завершуємо практичним робочим процесом CFD-to-JSBSim для побудови власної моделі літального апарата. Кожне твердження підкріплене посиланнями на вихідний код у форматі `file:line`.

1.2 Чим JSBSim відрізняється від типового рушія симуляції

- **Моделювання лише через XML:** літальні апарати описуються декларативним XML — для додавання нового планера перекомпіляція не потрібна.
- **Функційно-таблична аеродинаміка:** аеродинамічні коефіцієнти виражаються як функції, що поєднують таблиці, властивості та арифметичні оператори. Завдяки цьому модель повністю керується даними і її легко наповнювати з CFD чи даних аеродинамічної труби.
- **Дерево властивостей:** кожна змінна стану, керувальний вхід, вихід FCS і аеродинамічний коефіцієнт доступні за ієрархічним рядковим шляхом (`aero/qbar-psf`, `velocities/p-aero-rad_sec` тощо). Усе піддається спостереженню, а більшість значень — запису.
- **Фізика круглої Землі:** рівняння руху інтегруються у системі відліку ECI з урахуванням коріолісового та відцентрового прискорень від обертання Землі. Опціональне моделювання гравітаційного моменту підтримує космічні апарати.
- **Алгоритм балансування:** надійний розв'язувач кореня методом обмеження інтервалу балансує апарат у режим усталеного польоту (поздовжній, повний, у віражі, з виходом із пікірування, наземний, користувацький).

1.3 Домовленості, прийняті в цьому документі

Відстані та інерція за замовчуванням подаються в американських звичаєвих одиницях (фути, дюйми, $\text{slug}\cdot\text{ft}^2$) — це типове налаштування JSBSim — однак усі елементи допускають явний атрибут `unit`. Математичні позначення відповідають домовленості Etkin/Stevens-Lewis: α — кут атаки (angle of attack), β — кут ковзання (sideslip), p, q, r — кутові швидкості у зв'язаній системі, $\bar{q} = \frac{1}{2}\rho V^2$ — швидкісний напір (dynamic pressure), S , b , $\bar{c}$ — площа крила, розмах, середня аеродинамічна хорда.

1.4 Українсько-англійський словник ключових термінів

Це переклад зберігає всі назви, ідентифікатори, шляхи властивостей, елементи XML, цитати джерел (■■■■:■■■■) та одиниці виміру в оригіналі. Щоб полегшити звірку з англomовною документацією, вихідним кодом і самим деревом властивостей JSBSim, важливі терміни в тексті при першій згадці супроводжуються оригіналом у дужках, напр. «кут атаки (angle of attack)». Повний перелік ключових відповідників зібрано в таблиці нижче.

Українською	English	Українською	English
модель динаміки польоту	flight dynamics model (FDM)	рівняння руху	equations of motion
дерево властивостей	property tree	система керування польотом	flight control system (FCS)
планер	airframe	кут атаки	angle of attack
кут ковзання	sideslip	піднімальна сила	lift
сила опору (лобовий опір)	drag	бічна сила	side force
тангаж	pitch	крен	roll
рискання	yaw	момент тангажа	pitching moment
момент крену	rolling moment	момент рискання	yawing moment
коефіцієнт	coefficient	похідна стійкості	stability derivative
похідна демпфування	damping derivative	похідна керованості	control derivative
аеродинаміка	aerodynamics	аеродинамічний профіль	airfoil
крило	wing	видовження	aspect ratio
середня аеродинамічна хорда	mean aerodynamic chord (MAC)	швидкісний напір	dynamic pressure
число Маха	Mach number	число Рейнольдса	Reynolds number
пограничний шар	boundary layer	звалювання	stall
штопор	spin	гістерезис	hysteresis
центр мас	centre of gravity (CG)	тензор інерції	inertia tensor
запас поздовжньої стійкості	static margin	нейтральна точка	neutral point
балансування	trim	силова установка	propulsion
тяга	thrust	поршневий двигун	piston engine
турбіна	turbine	турбореактивний двигун	turbojet
турбовентиляторний двигун	turbofan	турбогвинтовий двигун	turboprop
тиск у впускному колекторі	manifold pressure	повітряний гвинт	propeller
регулятор постійних обертів	constant-speed governor	відносна хода	advance ratio
паливо	fuel	паливний бак	fuel tank
шасі	landing gear	реакції опори	ground reactions
амортизаційна стійка	strut	тертя	friction
гальмо	brake	коефіцієнт підсилення	gain
фільтр	filter	привод (виконавчий механізм)	actuator

датчик	sensor	автопілот	autopilot
система покращення стійкості	stability augmentation	демпфер рискання	yaw damper
кермова поверхня	control surface	відхилення	deflection
атмосфера	atmosphere	вітер	wind
турбулентність	turbulence	порив вітру	gust
розрахункова сітка	mesh	розв'язувач	solver
обчислювальна гідрогазодинаміка	CFD	вимушені коливання	forced oscillation
зведена частота	reduced frequency	аеродинамічна труба	wind tunnel
льотні випробування	flight test	пілотажні характеристики	handling qualities
короткоперіодичний рух	short period	фугоїд	phugoid
голландський крок	Dutch roll	спіральний рух	spiral mode
крен-мода	roll mode	власний рух (форма)	eigenmode
кватерніон	quaternion	верифікація	verification
валідація	validation		

Розділ 2: Архітектура та хід виконання

JSBSim побудований навколо центрального керувального класу (FGFDMExec), який володіє списком моделей із фіксованим порядком і керує ними через цикл із дискретним часом. Кожна модель зчитує свої вхідні дані з дерева властивостей (заповненого попередніми моделями на тому самому такті) і записує туди свої вихідні дані. Інтегрування рівнянь руху саме є однією з таких моделей.

2.1 Порядок виконання моделей

Моделі перелічено в FGFDMExec.h:225-241 і виконуються в такому порядку на кожному такті:

```
enum eModels {
    ePropagate=0,      // Integrate state from previous tick's accels
    eInput,            // Telnet/socket inputs
    eInertial,          // Gravity and Earth rotation vector
    eAtmosphere,        // T, p, rho, speed of sound at h
    eWinds,            // Steady wind + gusts + turbulence
    eSystems,          // FCS, autopilot, custom systems
    eMassBalance,       // CG, inertia tensor, total mass
    eAuxiliary,         // alpha, beta, qbar, Vt, Mach, ...
    ePropulsion,        // Engine thrust and torque
    eAerodynamics,     // Aero forces and moments
    eGroundReactions,   // Gear forces and friction
    eExternalReactions, // User-defined external forces
    eBuoyantForces,     // Lighter-than-air models
    eAircraft,          // Sum forces and moments at CG
    eAccelerations,     // Solve  $F = m a$ ,  $\tau = I \omega\text{-dot}$ 
    eOutput             // CSV, sockets, FlightGear, ...
};
```

Чому цей порядок важливий. Propagate виконується першим, бо просуває стан, використовуючи прискорення, обчислені наприкінці попереднього такту. Далі атмосфера й вітри дають тиск, густину та швидкість відносно повітряної маси, що потрібні Auxiliary для обчислення α , β , $\bar{q}$, M — які споживають аеродинамічні функції. Баланс мас виконується після FCS, щоб закон керування міг запросити зсув точкової маси (приклад: перекачування палива, скидання вантажу). Accelerations виконується останньою; її вихідні дані стають вхідними для Propagate на наступному такті.

2.2 Цикл Run у псевдокоді

```
FGFDMExec::Run() {                                     // FGFDMExec.cpp:404
    for child in childFDMs: child->Run()
    sim_time += dt; Frame++
    if (script) script->RunScript()
    for i in eModels:
        LoadInputs(i)      // copy upstream outputs to this model's inputs
        Models[i]->Run(holding)
}
```

Типовий крок інтегрування — $1.0 / 120.0$ с (див. FGFDMExec.cpp:99). На частоті 120 Гц JSBSim працює швидше за реальний час на одному ядрі CPU навіть для складних літальних апаратів. Крок задається через SetDeltaT() або через скрипт симуляції.

2.3 Модулі побіжно

Модуль	Вихідний код	Відповідальність
FGPropagate	models/FGPropagate.*	Інтегрує положення, швидкість, кватерніон
FGAccelerations	models/FGAccelerations.*	Рівняння Ньютона-Ейлера; наземне тертя через LCP
FGAerodynamics	models/FGAerodynamics.*	Підсумовує аеродинамічні сили/моменти на основі функцій
FGPropulsion	models/FGPropulsion.*	Двигуни (поршневий/турбінний/ракетний/електричний)
FGGroundReactions	models/FGGroundReactions.*	Шини, амортизаційні стійки, гальма, керування поворотом
FGAtmosphere	models/FGAtmosphere.*	ISA 1976 (T, p, ρ, a)
FGWinds	models/atmosphere/FGWinds.*	Усталений вітер, зсув, турбулентність Драйдена/Кармана
FGInertial	models/FGInertial.*	Гравітація (сферична або WGS-84), обертання планети
FGMassBalance	models/FGMassBalance.*	Маса, CG, тензор інерції, точкові маси
FGAuxiliary	models/FGAuxiliary.*	α , β , $\bar{q}$, V_t , число Маха, γ , перевантаження
FGFCS	models/FGFCS.*	Система керування польотом; граф каналів/компонентів
FGOutput	models/FGOutput.*	CSV, двійковий формат FlightGear, TCP, telnet
FGTrim	initialization/FGTrim.*	Розв'язувач кореня з обмеженням інтервалу для балансування
FGPropertyManager	input_output/FGPropertyManager.*	Ієрархічне дерево властивостей, увесь ввід/вивід через шляхи

Розділ 3: Системи координат, осі та одиниці

Правильне поводження із системами відліку — найпоширеніше джерело помилок під час побудови нового літального апарата. JSBSim використовує одразу кілька систем, і кожен розділ XML неявно обирає одну з них.

3.1 Конструктивна система (геометрія в XML)

Усі елементи `<location>` — шасі, CG, AERORP, точкові маси, розташування рушія двигуна — задаються у конструктивній (structural) системі відліку. Це зв'язана з літаком система, у якій вісь X зазвичай напрямлена назад уздовж фюзеляжу, Y — праворуч, Z — угору. Початок координат (0,0,0) за домовленістю розміщують поблизу протипожежної перегородки або носового базового перерізу. Типовою одиницею є дюйми.

Усередині FDM ці положення перетворюються у зв'язану із тілом (body-fixed) систему, яку використовують рівняння руху (X — уперед, Y — праворуч, Z — донизу), але самого перетворення в XML ви **не** бачите.

3.2 Зв'язана із тілом (зв'язана) система

Усі кутові швидкості, прискорення, швидкості у зв'язаній системі та компоненти інерції подаються у зв'язаній системі:

- **X** — уперед, крізь ніс.
- **Y** — назовні крізь праве крило.
- **Z** — донизу, доповнюючи правобічну трійку.

Поступальна швидкість у зв'язаній системі — це трійка властивостей `velocities/u-fps`, `v-fps`, `w-fps`. Кутові швидкості — `velocities/p-rad_sec`, `q-rad_sec`, `r-rad_sec`. Тензор інерції в XML задається у конструктивній системі, але під час завантаження повертається у зв'язану систему.

3.3 Швидкісна (аеродинамічна) та зв'язана зі стійкістю системи

Швидкісна (wind) система повернута відносно зв'язаної на кут ковзання β та кут атаки α : її вісь X напрямлена уздовж відносного вітру, сила опору діє вздовж $-X$, піднімальна сила діє вздовж $-Z$ (угору відносно потоку). Коли ви пишете `<axis name="LIFT">` у розділі аеродинаміки, JSBSim вважає, що ви працюєте у швидкісних осях. Зв'язана зі стійкістю (stability) система — це проміжна система, повернута відносно зв'язаної лише на α (без β); деякі класичні похідні стійкості табулюють саме в ній, і ви можете увімкнути її через `frame="STABILITY"`.

JSBSim виконує перетворення автоматично: сили, обчислені у LIFT/DRAG/SIDE, повертаються матрицею T_{w2b} у зв'язану систему, моменти додаються в точці AERORP, а потім переносяться в CG за формулою $r \times F$.

3.4 Локальна NED та ECI системи

Локальна NED (North-East-Down, північ-схід-вниз) — це локально-дотична система з центром у літаку. JSBSim використовує її для вітрів, наземної траєкторії (γ , ψ_{gt}) та локальних кутів Ейлера ϕ , θ , ψ . **ECI** (Earth-Centred Inertial, геоцентрична інерціальна) — це необертова система, у якій рівняння руху фактично інтегруються; FGPropagate підтримує $v_{inertial}$ та $r_{inertial}$ внутрішньо і перетворює їх у зв'язану/локальну для виведення.

3.5 Домовленості про знаки

- **Відхилення руля висоти:** додатне — задньою кромкою донизу (створює від'ємний момент тангажа), виводиться до fcs/elevator-pos-rad.
- **Відхилення елеронів:** диференціальне — C-172 використовує left-aileron-pos-rad, де додатне = донизу (кренить праворуч).
- **Відхилення руля напрямку:** додатне — задньою кромкою ліворуч (відхиляє ніс ліворуч).
- **Відцентрові добутки інерції:** класична домовленість трактує I_{xz} як інтеграл $+xz \, dm$. Деякі джерела використовують від'ємну домовленість; щоб перемкнути, встановіть negated_crossproduct_inertia="true" на елементі <mass_balance>.
- **Угору чи вниз:** Z_{body} додатне донизу. Конструктивне Z, задане в XML, зазвичай додатне вгору — JSBSim виконує обернення внутрішньо.

3.6 Одиниці

Length	FT (default), IN (default for <location>), M, KM, CM
Area	FT2 (default), M2, IN2, CM2
Mass	LBS (default), KG, SLUG (1 slug = 14.594 kg)
Inertia	SLUG*FT2 (default), KG*M2
Force	LBS, N
Pressure	PSF (default), PSI, INHG, ATM, PA, N/M2
Velocity (output)	FPS, KTS, M/S (per-property)
Angle	RAD (aero default), DEG (FCS default)
Spring	LBS/FT, N/M
Damping	LBS/FT/SEC, N/M/SEC

Завжди оголошуйте одиниці явно на кожному елементі, що приймає атрибут unit — приховані розбіжності типових значень спричиняють більшість звітів про помилки на кшталт «мій літак сам злітає боком».

Розділ 4: XML літального апарата, розділ за розділом

Літальний апарат — це єдиний XML-файл із кореневим елементом `<fdm_config>`. Порядок розділів примусово задається XSD; канонічну послідовність показано нижче. За вступним матеріалом ідуть приблизно 10 підрозділів, по одному на кожен головний розділ.

4.1 Корінь: `fdm_config`

```
<?xml version="1.0"?>
<fdm_config name="MyAircraft" version="2.0" release="PRODUCTION"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:noNamespaceSchemaLocation="http://jsbsim.sourceforge.net/JSBSim.xsd">

  <fileheader>          ... </fileheader>
  <planet>              ... </planet>      <!-- optional -->
  <metrics>             ... </metrics>
  <mass_balance>        ... </mass_balance>
  <ground_reactions>    ... </ground_reactions>
  <external_reactions>  ... </external_reactions>
  <buoyant_forces>      ... </buoyant_forces>
  <propulsion>          ... </propulsion>
  <system .../>         ... </system>      <!-- 0..N -->
  <autopilot ...>       ... </autopilot>    <!-- 0..1 -->
  <flight_control ...>  ... </flight_control>
  <aerodynamics>       ... </aerodynamics>
  <input ...>          ... </input>        <!-- 0..N -->
  <output ...>         ... </output>       <!-- 0..N -->
</fdm_config>
```

Атрибути: `name` (обов'язковий) — відображувана назва; `version` (обов'язковий, наразі 2.0); `release` = PRODUCTION | ALPHA | BETA (ALPHA/BETA виводять попередження під час завантаження).

4.2 `fileheader`: авторство та походження

Суто метадані. JSBSim ігнорує весь вміст усередині, проте його зчитують інструменти, що сканують парк моделей. Використовуйте його.

```
<fileheader>
  <author>Ada Lovelace</author>
  <email>ada@example.com</email>
  <organization>Acme Aircraft</organization>
  <license licenseName="GPL"
    licenseURL="http://www.gnu.org/licenses/gpl.html"/>
  <filecreationdate>2026-05-24</filecreationdate>
  <version>$Revision: 1.0 $</version>
  <description>Acme A-1 light sport aircraft.</description>
  <note>Stall hysteresis modelled per NACA TN-3909.</note>
  <limitation>Gear aero drag not modelled.</limitation>
  <reference title="AGARD-AR-265"
    author="ESDU"
    date="1988"
    URL="https://example.org/AR265.pdf"/>
</fileheader>
```

4.3 `metrics`: геометрія літального апарата

`<metrics>` задає величини для знерозмірнення: `S` (площа крила), `b` (розмах крила), `Ĉ` (середня аеродинамічна хорда), площі/плечі оперення та три базові точки AERORP, EYEPOINT і VRP.

```

<metrics>
  <wingarea unit="FT2"> 174.0 </wingarea>
  <wingspan unit="FT" > 35.8 </wingspan>
  <chord unit="FT" > 4.9 </chord>
  <htailarea unit="FT2"> 21.9 </htailarea>
  <htailarm unit="FT" > 15.7 </htailarm>
  <vtailarea unit="FT2"> 16.5 </vtailarea>
  <vtailarm unit="FT" > 0 </vtailarm>
  <wing_incidence unit="DEG"> 1.5 </wing_incidence> <!-- optional -->

  <location name="AERORP" unit="IN"> <!-- 25% MAC -->
    <x>43.2</x> <y>0</y> <z>59.4</z>
  </location>
  <location name="EYEPOINT" unit="IN">
    <x>37</x> <y>0</y> <z>48</z>
  </location>
  <location name="VRP" unit="IN">
    <x>42.6</x> <y>0</y> <z>38.5</z>
  </location>
</metrics>

```

Три базові точки мають конкретне значення:

- **AERORP** — аеродинамічна базова точка (aerodynamic reference point). Усі аеродинамічні моменти спочатку підсумовуються відносно цієї точки, а потім переносяться в CG за формулою $M_{cg} = M_{arp} + r_{arp \rightarrow cg} \times F$. За домовленістю розміщується на 25% середньої аеродинамічної хорди крила.
- **EYEPOINT** — положення очей пілота. Використовується FlightGear та візуальними системами; впливає на перевантаження на рівні голови пілота (accelerations/n-pilot-{x,y,z}-norm).
- **VRP** — візуальна базова точка (visual reference point), початок координат для зовнішніх засобів перегляду.

4.4 mass_balance: вага, CG, інерція

Споряджена вага плюс список точкових мас — це те, на основі чого JSBSim обчислює повну масу, положення CG і тензор інерції в CG. Точкові маси охоплюють екіпаж, пасажирів, вантаж та користувацькі елементи, які можуть переміщуватися під час виконання (підвісні вантажі, перекачуване паливо). Маса баків додається автоматично підсистемою силової установки і витрачається впродовж польоту.

```

<mass_balance negated_crossproduct_inertia="false">
  <ixx unit="SLUG*FT2"> 948 </ixx>
  <iyy unit="SLUG*FT2"> 1346 </iyy>
  <izz unit="SLUG*FT2"> 1967 </izz>
  <ixy unit="SLUG*FT2"> 0 </ixy>
  <ixz unit="SLUG*FT2"> 0 </ixz>
  <iyz unit="SLUG*FT2"> 0 </iyz>
  <emptywt unit="LBS"> 1500 </emptywt>
  <location name="CG" unit="IN">
    <x>41</x> <y>0</y> <z>36.5</z>
  </location>
  <pointmass name="Pilot">
    <weight unit="LBS"> 180 </weight>
    <location unit="IN"> <x>36</x><y>-14</y><z>24</z> </location>
  </pointmass>
  <pointmass name="Baggage">
    <weight unit="LBS"> 0 </weight>
    <location unit="IN"> <x>95</x><y>0</y><z>24</z> </location>
    <form shape="cylinder"> <!-- optional inertia shape -->
      <radius unit="IN"> 6 </radius>
      <length unit="IN"> 24 </length>
    </form>
  </pointmass>
</mass_balance>

```

Інерція відносно CG: значення, які ви задаєте в ixx , iyu , izz , ixy , ixz , iyz , — це інерція спорядженої конструкції відносно CG у конструктивній системі. JSBSim повертає їх у зв'язану систему, а потім додає внески кожної точкової маси за теоремою про паралельні осі. Атрибут `negated_crossproduct_inertia` існує тому, що половина підручників використовує протилежну домовленість про знак для добутоків інерції.

4.5 ground_reactions: шасі, лижі та контакти

Елементи `<contact>` описують усе, що може торкатися землі. Є два типи:

- **BOGEY** — колесо. Має амортизаційну стійку (пружність + демпфування), тертя шини (статичне, динамічне, кочення), опціональне керування поворотом, опціональні гальма та опціональне прибирання.
- **STRUCTURE** — контакт без кочення, як-от законцівка крила, лижа, фюзеляж чи хвостова опора. Зазвичай дуже жорсткий з низьким тертям.

```
<ground_reactions>
  <contact type="BOGEY" name="NOSE">
    <location unit="IN">
      <x>-6.8</x> <y>0</y> <z>-19.5</z>
    </location>
    <static_friction> 0.80 </static_friction>
    <dynamic_friction> 0.50 </dynamic_friction>
    <rolling_friction> 0.02 </rolling_friction>
    <spring_coeff unit="LBS/FT"> 1800 </spring_coeff>
    <damping_coeff unit="LBS/FT/SEC"> 600 </damping_coeff>
    <damping_coeff_rebound unit="LBS/FT/SEC"> 1200 </damping_coeff_rebound>
    <max_steer unit="DEG"> 10 </max_steer>
    <brake_group> NONE </brake_group>
    <retractable>0</retractable>
  </contact>
  <contact type="BOGEY" name="LEFT_MAIN">
    <location unit="IN"> <x>58.2</x><y>-43</y><z>-15.5</z> </location>
    <static_friction>0.8</static_friction>
    <dynamic_friction>0.5</dynamic_friction>
    <rolling_friction>0.02</rolling_friction>
    <spring_coeff unit="LBS/FT"> 5400 </spring_coeff>
    <damping_coeff unit="LBS/FT/SEC"> 1600 </damping_coeff>
    <brake_group> LEFT </brake_group>
  </contact>
  <contact type="STRUCTURE" name="LEFT_WINGTIP">
    <location unit="IN"> <x>43.2</x><y>-214.8</y><z>59.4</z> </location>
    <static_friction>0.2</static_friction>
    <dynamic_friction>0.2</dynamic_friction>
    <spring_coeff unit="LBS/FT"> 20000 </spring_coeff>
    <damping_coeff unit="LBS/FT/SEC"> 2000 </damping_coeff>
  </contact>
</ground_reactions>
```

Сили тертя обчислюються ітеративним розв'язувачем множників Лагранжа методом проєктованого Гаусса-Зейделя (Catto 2005) усередині `FGAccelerations::CalculateFrictionForces()`, тож багатоколісні контакти одночасно задовольняють обмеження невладопроникнення та кулонівського тертя.

4.6 external_reactions: додаткові сили

Сюди потрапляє все, що прикладає силу чи момент, які не є ні аеродинамічними, ні від силової установки, ні наземним контактом: парашути, буксирні троси, прискорювачі JATO, магнітні катапульти.

```
<external_reactions>
  <property>fcs/parachute_reef_pos_norm</property> <!-- declare -->

  <force name="parachute" frame="WIND">
    <function>
```

```

    <product>
      <property>aero/qbar-psf</property>
      <property>fcs/parachute_reef_pos_norm</property>
      <value>1.0</value>
      <value>10000.0</value>      <!-- chute area in sq-ft -->
    </product>
  </function>
  <location unit="FT"> <x>1</x><y>0</y><z>0</z> </location>
  <direction> <x>-1</x><y>0</y><z>0</z> </direction>
</force>
</external_reactions>

```

4.7 propulsion: двигуни, баки, рушії

Силова установка дворівнева: XML-файл двигуна (завантажується з \$JSBSIM_ROOT/engine/) описує двигун; XML літального апарата посилається на нього й додає рушій (тіло, що прикладає власне саму силу) та один чи кілька баків. Типи двигунів такі:

- **piston_engine** — поршневий двигун внутрішнього згоряння з таблицями потужності залежно від обертів, дросельної заслінки, паливо-повітряної суміші, тиску у впускному колекторі (MAP) та висоти.
- **turbine_engine** — турбореактивний/турбовентиляторний двигун із таблицями режимів малого газу / максимального безфорсажного / форсажного (AB).
- **turboprop_engine** — турбогвинтовий двигун із таблицями потужності на валу.
- **rocket_engine** — твердопаливний або рідкопаливний ракетний двигун із характеристиками тяги/lsp.
- **electric_engine** — двигун постійного струму, потужність × дросель.
- **brushless_dc_motor** — крива моменту залежно від обертів.
- **rotor** — гвинт гелікоптера (індуктивний потік + махові рухи).

```

<propulsion>
  <engine file="eng_io320">
    <feed>0</feed>      <!-- consume from tank 0 -->
    <feed>1</feed>
    <thruster file="prop_75in2f">
      <location unit="IN"> <x>-37.7</x><y>0</y><z>26.6</z> </location>
      <orient unit="DEG">
        <pitch>0</pitch><roll>0</roll><yaw>0</yaw>
      </orient>
      <sense>1</sense>      <!-- 1 = right-hand prop -->
      <p_factor>5</p_factor> <!-- effective offset for P-factor -->
    </thruster>
  </engine>

  <tank type="FUEL" number="0">
    <location unit="IN"> <x>56</x><y>-112</y><z>59.4</z> </location>
    <type>AVGAS</type>
    <capacity unit="LBS"> 185 </capacity>
    <contents unit="LBS"> 100 </contents>
    <priority>1</priority>
  </tank>
  <tank type="FUEL" number="1">
    <location unit="IN"> <x>56</x><y> 112</y><z>59.4</z> </location>
    <capacity unit="LBS"> 185 </capacity>
    <contents unit="LBS"> 100 </contents>
  </tank>
</propulsion>

```

Приклад файлу двигуна (engine/eng_io320.xml):

```

<piston_engine name="I0320">
  <minmp unit="INHG"> 10.0 </minmp>
  <maxmp unit="INHG"> 28.5 </maxmp>
  <displacement unit="IN3"> 320.0 </displacement>
  <maxhp> 160.0 </maxhp>

```

```

<bsfc>                0.32 </bsfc>
<cycles>              4.0 </cycles>
<idlerpm>            550.0 </idlerpm>
<maxrpm>             2700.0 </maxrpm>
<maxthrottle>        1.0 </maxthrottle>
<minthrottle>        0.1 </minthrottle>
<sparkfaildrop>      0.1 </sparkfaildrop>
</piston_engine>

```

4.8 flight_control: формування команд

FCS — це орієнтований граф компонентів, упорядкованих у іменовані канали. Входи надходять із команд пілота (fcs/elevator-cmd-norm тощо, задаються пристроєм вводу чи скриптом), а виходи надходять у властивості аеродинамічного відхилення (fcs/elevator-pos-rad тощо), які зчитують аеродинамічні функції.

```

<flight_control name="FCS: c172">

  <channel name="Pitch">
    <summer name="Pitch Trim Sum">
      <input>fcs/elevator-cmd-norm</input>
      <input>fcs/pitch-trim-cmd-norm</input>
      <clipto> <min>-1</min><max>1</max> </clipto>
    </summer>

    <aerosurface_scale name="Elevator Control">
      <input>fcs/pitch-trim-sum</input>
      <gain>0.01745</gain>          <!-- deg -> rad -->
      <range> <min>-28</min><max>23</max> </range>
      <output>fcs/elevator-pos-rad</output>
    </aerosurface_scale>
  </channel>

  <channel name="Flaps">
    <kinematic name="Flaps Control">
      <input>fcs/flap-cmd-norm</input>
      <traverse>
        <setting><position>0</position> <time>0</time></setting>
        <setting><position>10</position><time>2</time></setting>
        <setting><position>20</position><time>2</time></setting>
        <setting><position>30</position><time>2</time></setting>
      </traverse>
      <output>fcs/flap-pos-deg</output>
    </kinematic>
  </channel>
</flight_control>

```

Доступні компоненти FCS включають summer, pure_gain, scheduled_gain, aerosurface_scale, kinematic, deadband, switch, fcs_function, pid, lag_filter, lead_lag_filter, washout_filter, second_order_filter, integrator, sensor, actuator, а також узагальнений fcs_function, який дозволяє вбудувати будь-який вираз мовою функцій як блок. PID-регулятори та фільтри дискретизують свою неперервно-часову форму за допомогою кроку симуляції dt.

4.9 autopilot та system: той самий набір, інша область

<autopilot> та <system> використовують ті самі компоненти FCS та структуру каналів. Їх виокремлено, бо вони виконуються як окремі підгрупи: типова практика — розміщувати базове формування команд ручки/педалей у flight_control, класичні контури автопілота (утримання висоти, утримання курсу, автомат тяги) — в autopilot, а логіку авіоніки чи електросистеми — в system (яка може завантажуватися із зовнішніх файлів у \$AC/Systems/).

4.10 aerodynamics: ядро моделі

Аеродинаміка достатньо обширна, щоб заслуговувати на власний розділ — див. розділ 5. Її каркас такий:

```
<aerodynamics>

  <alphalimits unit="RAD">          <!-- physical alpha clamp -->
    <min>-0.087</min>
    <max> 0.280</max>
  </alphalimits>

  <hysteresis_limits unit="RAD">    <!-- stall hysteresis -->
    <min>0.09</min>
    <max>0.36</max>
  </hysteresis_limits>

  <!-- Helper functions (often ground effect, induced velocity) -->
  <function name="aero/function/kCLge"> ... </function>

  <!-- Six axes: three forces and three moments -->
  <axis name="DRAG"> ... </axis>
  <axis name="SIDE"> ... </axis>
  <axis name="LIFT"> ... </axis>
  <axis name="ROLL"> ... </axis>
  <axis name="PITCH"> ... </axis>
  <axis name="YAW"> ... </axis>
</aerodynamics>
```

4.11 input та output

<input> вмикає зовнішній інтерфейс, зазвичай сокет TCP/UDP для взаємодії команда/запит у стилі telnet. <output> трапляється значно частіше: це конфігурація журналювання для CSV-файлів, двійкового формату FlightGear, сокетів тощо.

```
<output name="flight.csv" type="CSV" rate="60">
  <property> aero/qbar-psf </property>
  <property> velocities/vt-fps </property>
  <rates>      ON </rates>
  <velocities>ON </velocities>
  <forces>     ON </forces>
  <moments>    ON </moments>
  <position>   ON </position>
  <fcs>        OFF</fcs>
  <aerosurfaces>OFF</aerosurfaces>
</output>

<output type="FLIGHTGEAR" rate="60" protocol="UDP" port="5500"/>

<input port="1138"/>    <!-- telnet on TCP 1138 -->
```

Розділ 5: Аеродинаміка — поглиблений розгляд

5.1 Шість осей та функційно-таблична модель

Усередині <aerodynamics> серцем моделі є довільна кількість елементів <function>, згрупованих у блоки <axis>. JSBSim підсумовує кожну функцію всередині осі, щоб отримати повну силу чи момент цієї осі у фізичних одиницях (lbf, ft·lbf). Жодного додаткового знерозмірнення вона **не** виконує — множник $\bar{q} \cdot S$ ви за домовленістю вбудовуєте самі.

Назви осей відповідають індексам (FGAerodynamics.cpp:57-69):

Forces (axis index 0..2)	Moments (axis index 3..5)
0 DRAG	3 ROLL (l)
1 SIDE	4 PITCH (m)
2 LIFT	5 YAW (n)

Alternative force axes:	Alternative moment frames:
X, Y, Z (body)	frame="WIND"
AXIAL, NORMAL (axial/norm)	frame="STABILITY"

5.2 Як обчислюється окрема функція

Кожна функція щотакту повертає скаляр. Канонічний доданок сили опору має такий вигляд:

```
<function name="aero/coefficient/CD0">
  <description>Drag at zero lift</description>
  <product>
    <property>aero/qbar-psf</property>      <!-- q-bar -->
    <property>metrics/Sw-sqft</property>    <!-- S -->
    <value>0.027</value>                    <!-- CD0 -->
  </product>
</function>
```

$$F_{D,0} = C_{D0} \cdot \bar{q} \cdot S$$

Усередині FGAerodynamics::Run() значення, повернуті всіма нащадками <axis name="DRAG">, підсумовуються у комірку DRAG масиву vFnative; комірки LIFT та SIDE заповнюються так само; потім поворот матрицею T_{w2b} переводить сили у швидкісних осях у зв'язану систему, моменти, обчислені ROLL/PITCH/YAW, додаються в точці AERORP, і нарешті моменти переносяться в CG за формулою $M_{cg} = M_{arp} + r_{arp \rightarrow cg} \times F$.

5.3 Мова функцій

Мова функцій JSBSim — це невелике XML-дерево виразів у стилі Lisp. Листками є <value> (константа) та <property> (посилання на дерево властивостей); внутрішні вузли — це оператори.

Arithmetic	sum, difference, product, quotient, pow, sqrt, abs, min, max, avg
Trigonometric	sin, cos, tan, asin, acos, atan, atan2
Exponential	exp, ln, log2, log10
Logical	lt, le, gt, ge, eq, neq, and, or, not, ifthen, switch
Modular	mod, floor, ceil, fmod, roundmultiple
Unit	toradians, todegrees
Random	random (Gaussian), urandom (uniform)
Constants	pi, value/v, integer
Table	table (1-D, 2-D, 3-D, n-D)
Property	property/p

Цікавіший приклад — індуктивний опір, $C_{Di} = C_L^2 / (\pi \cdot AR \cdot e)$ — перетворений на функцію JSBSim:

```
<function name="aero/coefficient/CDi">
  <description>Induced drag</description>
  <product>
    <property>aero/qbar-psf</property>
    <property>metrics/Sw-sqft</property>
    <quotient>
      <pow>
        <property>aero/cl-squared</property>      <!-- CL^2 -->
        <value>0.5</value>                        <!-- *not* powered -->
      </pow>
    </product>
    <product>
      <pi/>
      <property>metrics/aspectratio</property>
      <value>0.85</value>                        <!-- e -->
    </product>
  </quotient>
</product>
</function>
```

5.4 Таблиці: 1-вимірні, 2-вимірні, 3-вимірні та вищих розмірностей

Таблиці — це робоча конячка будь-якої нетривіальної моделі: вони дають змогу закодувати отриманий з CFD пошук будь-якого коефіцієнта за будь-якою комбінацією змінних стану. JSBSim використовує **лінійну інтерполяцію** за кожним виміром; поза табульованим діапазоном граничне значення фіксується (без екстраполяції), тож покрити робочу область — ваше завдання.

1-вимірна таблиця (векторний пошук за однією змінною):

```
<table>
  <independentVar lookup="row">aero/alpha-rad</independentVar>
  <tableData>
    -0.0900  -0.2200
     0.0000   0.2500
     0.0900   0.7300
     0.1500   1.0000
     0.2800   1.1500
  </tableData>
</table>
```

2-вимірна таблиця (наприклад, сила опору залежно від alpha та кута відхилення закритків):

```
<table>
  <independentVar lookup="row">aero/alpha-rad</independentVar>
  <independentVar lookup="column">fcs/flap-pos-deg</independentVar>
  <tableData>
           0.0    10.0   20.0   30.0
    -0.0873  0.0041  0.0000  0.0005  0.0014
    -0.0349  0.0003  0.0057  0.0108  0.0141
     0.0000  0.0052  0.0168  0.0251  0.0299
     0.0524  0.0240  0.0452  0.0583  0.0655
     0.1571  0.0962  0.1353  0.1573  0.1690
  </tableData>
</table>
```

3-вимірна таблиця (наприклад, момент тангажа залежно від alpha, руля висоти, числа Маха). Ви укладаєте 2-вимірні таблиці стосом, кожна з атрибутом `breakPoint` для третього виміру:

```
<table>
  <independentVar lookup="row">aero/alpha-rad</independentVar>
  <independentVar lookup="column">fcs/elevator-pos-rad</independentVar>
  <independentVar lookup="table">velocities/mach</independentVar>
```

```

<tableData breakPoint="0.30">                                <!-- Mach = 0.30 -->
    -0.4  -0.2   0.0   0.2   0.4
    -0.1   0.20  0.11  0.04 -0.06 -0.16
    0.0    0.15  0.07  0.00 -0.07 -0.15
    0.1    0.11  0.04 -0.04 -0.12 -0.21
</tableData>

<tableData breakPoint="0.80">                                <!-- Mach = 0.80 -->
    -0.4  -0.2   0.0   0.2   0.4
    -0.1   0.18  0.10  0.03 -0.05 -0.14
    0.0    0.13  0.06  0.00 -0.06 -0.13
    0.1    0.10  0.03 -0.03 -0.10 -0.19
</tableData>
</table>

```

Вищі розмірності підтримуються рекурсивно. На практиці 3-вимірні таблиці — це межа того, що варто будувати вручну; для 4-вимірних і вище використовуйте скрипт для генерації XML із бази даних CFD.

5.5 Властивості, що часто використовуються в аеродинаміці

aero/qbar-psf	$q\text{-bar} = 0.5 \cdot \rho \cdot V_t^2$ (psf)
aero/alpha-rad	angle of attack (radians)
aero/beta-rad	sideslip angle (radians)
aero/alphadot-rad_sec	alpha-dot
aero/betadot-rad_sec	beta-dot
aero/mag-beta-rad	beta (useful in drag terms)
aero/bi2vel	$b / (2 V)$ --- roll-rate non-dim factor
aero/ci2vel	$cbar / (2 V)$ --- pitch-rate non-dim factor
aero/h_b-mac-ft	height above ground / chord, for ground effect
aero/cl-squared	CL^2 (computed from previous tick's lift)
velocities/p-aero-rad_sec	roll rate in aero frame (rad/s)
velocities/q-aero-rad_sec	pitch rate in aero frame
velocities/r-aero-rad_sec	yaw rate in aero frame
velocities/mach	Mach number
velocities/vt-fps	true airspeed (ft/s)
metrics/Sw-sqft	wing area
metrics/bw-ft	wing span
metrics/cbarw-ft	mean aerodynamic chord
fcs/elevator-pos-rad	elevator deflection from FCS output
fcs/aileron-pos-rad	aileron
fcs/rudder-pos-rad	rudder
fcs/flap-pos-deg	flap (commonly in degrees)
gear/gear-pos-norm	gear position (0..1)
fcs/speedbrake-pos-rad	speedbrake

Кутові швидкості в аеродинамічній системі p-aero, q-aero, r-aero — це швидкості у зв'язаній системі, повернуті у швидкісну систему (повернуту лише на α — зв'язану зі стійкістю систему). Для похідних демпфування завжди використовуйте саме їх, а не «сирі» p-rad_sec.

5.6 Знерозмірнення вручну

Оскільки JSBSim не знерозмірнює за вас, на вас лягає відповідальність правильно записати розмірну формулу. Це підручникові формули, які ви вбудовуєте в <product>:

$\text{Lift} = C_L \cdot \bar{q} \cdot S$
$\text{Drag} = C_D \cdot \bar{q} \cdot S$
$\text{Side} = C_Y \cdot \bar{q} \cdot S$
$\text{Roll moment} = C_l \cdot \bar{q} \cdot S \cdot b$
$\text{Pitch moment} = C_m \cdot \bar{q} \cdot S \cdot \bar{c}$

$$\text{Yaw moment} = C_n \cdot \bar{q} \cdot S \cdot b$$

Для похідних демпфування помножте на відповідний множник $b/(2V)$ або $\bar{c}/(2V)$:

$$\text{Roll-rate moment} = C_{lp} \cdot \bar{q} \cdot S \cdot b \cdot [b/(2V)] \cdot p$$

$$\text{Pitch-rate moment} = C_{mq} \cdot \bar{q} \cdot S \cdot \bar{c} \cdot [\bar{c}/(2V)] \cdot q$$

$$\text{Yaw-rate moment} = C_{nr} \cdot \bar{q} \cdot S \cdot b \cdot [b/(2V)] \cdot r$$

$$\dot{\alpha} \text{ moment} = C_{m_{\dot{\alpha}}} \cdot \bar{q} \cdot S \cdot \bar{c} \cdot [\bar{c}/(2V)] \cdot \dot{\alpha}$$

5.7 Розв'язаний приклад: момент крену від кутової швидкості крену

3 aircraft/c172p/c172p.xml:733:

```
<function name="aero/coefficient/Clp">
  <description>Roll moment due to roll rate (roll damping)</description>
  <product>
    <property>aero/qbar-psf</property>          <!-- q-bar -->
    <property>metrics/Sw-sqft</property>         <!-- S -->
    <property>metrics/bw-ft</property>            <!-- b -->
    <property>aero/bi2vel</property>              <!-- b/(2V) -->
    <property>velocities/p-aero-rad_sec</property> <!-- p -->
    <value>-0.4840</value>                       <!-- Clp -->
  </product>
</function>
```

C_{lp} для C-172 є сталим і дорівнює $-0,484$ — це демпфування крену крилом. Перемноження дає момент крену у зв'язаній системі в $\text{ft} \cdot \text{lbf}$, чого й очікує вісь ROLL.

5.8 Статичні похідні — отримання їх із CFD

Статичні похідні ($\partial C/\partial \alpha$, $\partial C/\partial \beta$, $\partial C/\partial \delta$) характеризують усталений аеродинамічний відгук літального апарата на зміни α , β та відхилень органів керування. Їх отримують із CFD, обраховуючи літак як тверде тіло в усталеному стані на сітці точок (α , β , δ , M) і зчитуючи отримані сили та моменти. Наведений нижче перелік — це типовий мінімальний набір розгортки для дозвукового класичного літального апарата:

Похідна	Незалежні змінні	Розгортка CFD
$C_L(\alpha, M, \text{flap})$	α, M, flap	усталений крейсер за $\alpha \in [-6^\circ, 18^\circ]$, звалювання = фіксація
$C_D(\alpha, M, \text{flap}, \text{gear})$	$\alpha, M, \text{flap}, \text{gear}$	Та сама матриця; C_D = сила опору/ $\bar{q}S$
$C_m(\alpha, M, \delta_e)$	α, M, δ_e	α -розгортка за кожного δ_e та числа Маха
$C_Y(\beta, \delta_r)$	β, δ_r	β -розгортка $\in [-15^\circ, 15^\circ]$
$C_l(\beta, \alpha, \delta_a)$	β, α, δ_a	β -розгортка + прирости δ_a
$C_n(\beta, \alpha, \delta_r)$	β, α, δ_r	β -розгортка + прирости δ_r
$C_{L\alpha}$	α	нахил кривої C_L - α
$C_{m\alpha}$	α	нахил кривої C_m - α (від'ємний $\rightarrow$ стійкий)
$C_{n\beta}$	β	нахил кривої C_n - β (додатний $\rightarrow$ стійкий)
C_{lp}	β	ефект поперечного V

5.9 Динамічні похідні (демпфування) — CFD з рухом

Динамічні похідні описують момент, створений кутовою швидкістю (p , q , r , $\dot{\alpha}$). Їх не можна отримати зі статичного знімка геометрії в усталеному стані; потрібен рухомий CFD або класичні інженерні аналоги.

- **Вимушені коливання:** в URANS чи LES змусьте літак коливатися синусоїдально навколо відповідної осі із заданими амплітудою та зведеною частотою. Виокремте синфазну та квадратурну складові моменту — вони дають відповідно статичні та похідні демпфування. Стандартні промислові інструменти: ANSYS Fluent, OpenFOAM з *overset*, STAR-CCM+, NASA OVERFLOW.
- **Квазістатичний метод:** обрахуйте кілька усталених випадків CFD зі сталою кутовою швидкістю p , накладеною на інерціальну систему; виокремте момент крену з кожного. Нахил $\Delta C_l / \Delta (pb/2V)$ і є C_{l_p} . Швидко, але обмежено малими кутами.
- **Метод смуг / несної лінії:** дешевий інженерний запасний варіант. Для початкового запуску моделі візьміть довідникові оцінки (Roskam, Etkin, USAF DATCOM) — для класичних літальних апаратів вони лежать у межах $\pm 30\%$ від високоточних даних.
- **Ідентифікація системи:** якщо у вас уже є дані льотних випробувань, підберіть похідні демпфування, мінімізуючи нев'язку симуляції на маневрі-дублеті. Саме це насправді й роблять статті AIAA SciTech про «модель, валідовану в польоті».

Похідні демпфування, які слід отримати щонайменше:

Символ	Значення	Типовий знак	Метод
C_{l_p}	Демпфування крену	від'ємний	вимушені коливання навколо X
C_{l_r}	Крен від швидкості рискання	додатний	вимушені коливання навколо Z
C_{m_q}	Демпфування тангажа	від'ємний	вимушені коливання навколо Y
$C_{m_{\dot{\alpha}}}$	Тангаж від $\dot{\alpha}$	від'ємний	вимушені коливання занурення
C_{n_r}	Демпфування рискання	від'ємний	вимушені коливання навколо Z
C_{n_p}	Рискання від крену	малий, зі знаком	вимушені коливання навколо X
C_{Y_p}	Бічна сила від крену	малий	з того самого прогону коливань навколо X
C_{Y_r}	Бічна сила від рискання	малий	з того самого прогону коливань навколо Z

5.10 Похідні керованості

Похідні керованості ($C_{m_{\delta_e}}$, $C_{l_{\delta_a}}$, $C_{n_{\delta_r}}$, ...) — це нахили моменту залежно від відхилення органа керування. Вони статичні, тож ви отримуєте їх у тому самому пакеті усталеного CFD, що й статичні похідні, просто включивши відхилення органа керування як вісь розгортки. Для нелінійної ефективності керування (дуже поширеної за великих α чи великих відхилень) віддавайте перевагу 2-вимірній таблиці (α , δ), а не одному коефіцієнту.

```
<function name="aero/coefficient/Cmde">
  <description>Pitch moment due to elevator</description>
  <product>
    <property>aero/qbar-psf</property>
    <property>metrics/Sw-sqft</property>
    <property>metrics/cbarw-ft</property>
    <table>
      <independentVar lookup="row">aero/alpha-rad</independentVar>
      <independentVar lookup="column">fcs/elevator-pos-rad</independentVar>
      <tableData>
```

```

        -0.40  -0.20   0.00   0.20   0.40
    -0.10   0.22   0.11   0.00  -0.11  -0.22
     0.00   0.21   0.10   0.00  -0.10  -0.21
     0.10   0.18   0.09   0.00  -0.09  -0.18
     0.20   0.12   0.06   0.00  -0.06  -0.12
    </tableData>
  </table>
</product>
</function>

```

5.11 Вплив екрана та число Рейнольдса

Вплив екрана (екранний ефект) за домовленістю реалізують як множник до піднімальної сили та сили опору, обчислюваний як 1-вимірна таблиця залежно від висоти над землею, нормованої на середню хорду (aero/h_b-mac-ft):

```

<function name="aero/function/kCLge">
  <description>Lift multiplier due to ground effect</description>
  <table>
    <independentVar>aero/h_b-mac-ft</independentVar>
    <tableData>
      0.00  1.203
      0.10  1.127
      0.20  1.073
      0.50  1.019
      1.00  1.002
      1.10  1.000
    </tableData>
  </table>
</function>

```

Використовуйте aero/function/kCLge як множник у функції піднімальної сили. Той самий підхід опрацьовує вплив числа Маха (таблиця залежно від velocities/mach) та число Рейнольдса (таблиця залежно від velocities/reynolds), якщо у вас є дані CFD для їхнього наповнення.

5.12 Звалювання та гістерезис звалювання

<alphalimits> обмежує фізичний α , що використовується в аеродинамічних обчисленнях. <hysteresis_limits> задає смугу перемикання — JSBSim надає aero/stall-hyst-norm, що дорівнює 0 нижче нижньої межі та 1 вище верхньої межі. Використовуйте її як другу вісь таблиці піднімальної сили, щоб закодувати різницю між кривими піднімальної сили до та після звалювання й отримати правильну поведінку під час виходу зі звалювання.

Розділ 6: Рівняння руху зсередини

Увесь XML і всі таблиці зрештою живлять невеликий набір звичайних диференціальних рівнянь. У цьому розділі розглянуто математику так, як її реалізує JSBSim.

6.1 Вектор стану

```
VehicleState {                                     // FGPropagate.h:100
  FGLocation      vLocation;                       // ECEF position (ft)
  FGColumnVector3 vUVW;                            // body-frame velocity rel. ECEF
  FGColumnVector3 vPQR;                            // body-frame angular rate rel. ECEF
  FGColumnVector3 vPQRi;                          // body-frame angular rate rel. ECI
  FGQuaternion    qAttitudeLocal;                 // body w.r.t. local NED
  FGQuaternion    qAttitudeECI;                   // body w.r.t. ECI <-- primary
  FGColumnVector3 vInertialVelocity;              // ECI frame velocity
  FGColumnVector3 vInertialPosition;              // ECI frame position
  // ... history dequeues for multi-step integrators
};
```

6.2 Кінематика орієнтації через кватерніон

Орієнтація просувається як кватерніон $q_{i \rightarrow b}$ від інерціальної до зв'язаної системи. Похідна за часом — це кінематичне рівняння

$$\dot{q} = \frac{1}{2} \cdot q \otimes \omega_{b/i}$$

де $\omega_{b/i}$ — кутова швидкість тіла відносно інерціальної системи, виражена у зв'язаній системі (vPQRi). `FGQuaternion::GetQDot()` у JSBSim обчислює саме це; `FGPropagate::CalculateQuatdot()` є обгорткою над ним. Доступно кілька інтеграторів:

```
0 eNone           freeze
1 eRectEuler      q(n+1) = q(n) + dt * q-dot(n)
2 eTrapezoidal    q(n+1) = q(n) + 0.5*dt*(q-dot(n)+q-dot(n-1))
3 eAdamsBashforth2 q(n+1) = q(n) + dt*(1.5 q-dot(n) - 0.5 q-dot(n-1))
4 eAdamsBashforth3 23/12, -16/12, +5/12 weights
5 eAdamsBashforth4 55/24, -59/24, 37/24, -9/24
6 eAdamsBashforth5 classical 5-step coefficients
7 eBuss1          q(n+1) = q(n) * exp(0.5*dt*omega)
8 eBuss2          augmented with omega-dot terms
9 eLocalLinearization Barker et al. (NASA TN D-7347)
```

Типові інтегратори — прямокутний метод Ейлера для орієнтації та кутової швидкості, Adams-Bashforth-2 для поступальної швидкості, Adams-Bashforth-3 для положення. Інтегратори Buss цікаві тим, що точно зберігають норму кватерніона (без потреби в перенормуванні, якого прямокутний метод Ейлера потребує щотакту).

6.3 Поступальна динаміка в інерціальній системі

JSBSim розв'язує другий закон Ньютона в інерціальній системі та перетворює назад у зв'язані координати для виведення:

$$\dot{v}_{body} = F/m - (p_{body} + 2 \cdot T_{i2b} \cdot \Omega_{planet}) \times v_{body} - T_{i2b} \cdot \Omega_{planet} \times (\Omega_{planet} \times r_i)$$

Перший доданок — це звичайне $F = ma$. Другий — коріолісове прискорення, включно з внеском від обертання Землі ($\Omega_{planet} \approx 7.292 \cdot 10^{-5}$ rad/s навколо осі Z системи ECI). Третій — відцентрове прискорення від обертання навколо центра Землі. Для польотів у межах атмосфери на масштабах транспортних літаків ці доданки малі, але помітні — саме завдяки їм симулятор узгоджується з контрольними прикладами NASA-2015.

6.4 Обертальна динаміка

$$\mathbf{J} \cdot \dot{\boldsymbol{\omega}}_i = \mathbf{M} - \boldsymbol{\omega}_i \times (\mathbf{J} \cdot \boldsymbol{\omega}_i)$$

Доданок $\boldsymbol{\omega} \times (\mathbf{J}\boldsymbol{\omega})$ — це ейлерів гіроскопічний момент (ненульовий завжди, коли тензор інерції не ізотропний). `FGAccelerations::CalculatePQRdot()` обчислює його безпосередньо. Тензор інерції $\mathbf{J}$ у зв'язаній системі підтримується `FGMassBalance` і оновлюється щотакту, щоб враховувати вигоряння палива та зсуви точкових мас. Опціональний момент від градієнта гравітації вмикається через властивість `simulation/gravitational-torque`; корисно для орбітальної механіки.

6.5 Збирання повної сили та моменту

`FGAircraft` підсумовує сили та моменти з моделей аеродинаміки, силової установки, опори, зовнішніх реакцій, плавучості і подає їх як єдині $\mathbf{F}$ та $\mathbf{M}$ у зв'язаній системі моделі `Accelerations`. Гравітацію окремо опрацьовує `FGInertial` (сферична або WGS-84 з доданком $\mathbf{J}_2$) і додає її безпосередньо до вектора прискорення — виключення її з $\mathbf{M}$ означає, що момент гравітації відносно CG автоматично дорівнює нулю (гравітація за визначенням діє в CG).

6.6 Наземне тертя як задача з обмеженнями

Звичайне явне інтегрування жорстких пружинно-демпферних моделей шасі є нестійким. Натомість JSBSim формулює задачу тертя як задачу лінійної доповняльності (LCP): кожна точка контакту накладає обмеження невзаємопроникнення за нормаллю до ЗПС і обмеження кулонівського тертя по дотичній до неї. Невідомими є множники Лагранжа λ . Система $\mathbf{A} \cdot \lambda = \mathbf{b}$ з $\mathbf{A} = \mathbf{J} \cdot \mathbf{M}^{-1} \cdot \mathbf{J}^T$ розв'язується ітеративно (проектований метод Гаусса-Зейделя, Catto 2005) до 50 ітерацій на такт — див. `FGAccelerations::CalculateFrictionForces()`. Контактні сили тертя та момент $\mathbf{r} \times \mathbf{F}$ потім додаються до векторів $\mathbf{F}$ та $\mathbf{M}$.

6.7 Балансування

Балансування реалізовано в `FGTrim` розв'язувачем кореня з обмеженням інтервалу, що застосовується незалежно до кожної пари вісь-керування й ітерується у зовнішньому циклі, доки всі осі не збіжаться. Це **не** метод Ньютона-Рафсона — JSBSim використовує варіант методу січних із лінійною інтерполяцією та коефіцієнтом релаксації 0,9 для придушення коливань. Режими такі:

Режим	Обмеження	Органи керування
tLongitudinal	$\dot{w}=0, \dot{u}=0, \dot{q}=0$	α , дросель, руль висоти
tFull	поздовжній + $\dot{v}=0, \dot{p}=0, \dot{r}=0$ + утримання ψ	+ ϕ , елерони, руль напрямку, β
tGround	$\dot{w}=0, \dot{q}=0, \dot{p}=0$	висота, θ , ϕ
tPullup	поздовжній за цільового перевантаження	α , дросель, руль висоти
tTurn	координований віраж за цільового кута крену	дросель, руль висоти, руль напрямку
tCustom	користувачькі пари (стан, керування)	будь-які

6.8 Модель атмосфери

FGStandardAtmosphere реалізує Стандартну атмосферу США 1976 року через кусково-лінійні вертикальні градієнти температури від 0 до 86 км. Властивості надаються в розділі atmosphere/: T-R, P-psf, rho-slugs_ft3, a-fps. Можна створити власний підклас атмосфери; модель C-172 зсуває температуру, щоб змодельовати ефекти висоти за щільністю.

6.9 Вітри та турбулентність

FGWinds підтримує:

- Усталений вітер у NED (задається через atmosphere/wind-north-fps, wind-east-fps, wind-down-fps).
- Лінійний зсув вітру з висотою.
- Пориви вітру (модель «дискретного пориву» 1-cosine, що використовується в MIL-F-8785C).
- Неперервну турбулентність за Драйденом або фон Карманом із налаштовуваними інтенсивністю та масштабами.
- Модель мікропориву (microburst) з трьома складовими та вихровим кільцем.

FGAuxiliary споживає вектор вітру для обчислення швидкості відносно повітряної маси, яку використовує аеродинаміка.

6.10 Дерево властивостей

FGPropertyManager є обгорткою над деревом властивостей SimGear. Кожна змінна стану, керувальний вхід, сигнал FCS, аеродинамічний коефіцієнт та атмосферна величина доступні за рядковим шляхом; значення курсують між моделями цілком через нього. Елементи <property> в XML літального апарата можуть оголошувати нову властивість (створюючи її за потреби); компоненти FCS, аеродинамічні функції та зовнішні інтерфейси всі прив'язуються до цього дерева.

```
# Inspect a property in the JSBSim Python module
import jsbsim
fdm = jsbsim.FGFDMEExec(None)
fdm.load_model("c172p")
fdm.run_ic()
print(fdm["velocities/vt-fps"], fdm["aero/alpha-deg"])
```

Розділ 7: Побудова власного літального апарата, крок за кроком

Це практичний посібник до дії. Ми припускаємо, що у вас є доступ до **геометрії** (CAD або принаймні три проекції), **масових характеристик** (вага, CG, в ідеалі оцінка інерції), **даних двигуна** (криві потужності чи тяги) та **аеродинамічних даних** (CFD або аеродинамічна труба). Наведений нижче робочий процес поєднує їх у робочу модель JSBSim за пару тижнів неповної зайнятості.

7.1 Крок 1: Структура каталогу

```
$JSBSIM_ROOT/aircraft/MyAcft/
  MyAcft.xml           # the main XML (this is what JSBSim loads)
  reset00.xml          # initial conditions for default launch
  Systems/             # (optional) external <system> files
  Engines/             # (optional) aircraft-specific engine files
$JSBSIM_ROOT/scripts/
  MyAcft_takeoff.xml    # a script that loads MyAcft + reset00
```

Двигуни та повітряні гвинти можуть розміщуватися у спільному для проєкту каталозі `$JSBSIM_ROOT/engine/`, якщо ви хочете повторно використовувати їх для кількох планерів.

7.2 Крок 2: Каркас XML

```
<?xml version="1.0"?>
<fdm_config name="MyAcft" version="2.0" release="ALPHA"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:noNamespaceSchemaLocation="http://jsbsim.sourceforge.net/JSBSim.xsd">

  <fileheader>
    <author>You</author>
    <filecreationdate>2026-05-24</filecreationdate>
    <description>My new aircraft.</description>
  </fileheader>

  <metrics>           ... </metrics>
  <mass_balance>      ... </mass_balance>
  <ground_reactions>  ... </ground_reactions>
  <propulsion>        ... </propulsion>
  <flight_control name="FCS: MyAcft">
    <channel name="Pitch"> ... </channel>
    <channel name="Roll">  ... </channel>
    <channel name="Yaw">   ... </channel>
  </flight_control>
  <aerodynamics>     ... </aerodynamics>
  <output type="CSV" rate="60" name="MyAcft.csv">
    <rates>ON</rates>
    <velocities>ON</velocities>
    <position>ON</position>
  </output>
</fdm_config>
```

7.3 Крок 3: Геометрія, маса та шасі

Геометрія: з вашого CAD витягніть площу крила S , розмах крила b , середню аеродинамічну хорду $\bar{c}$, площі та плечі оперення. Виберіть AERORP на 25% MAC і розмістіть його у своїй конструктивній системі. Визначте розташування CG (будь-яка прийнятна точка з діапазону центрування — пізніше ви зможете зсунути його точковими масами).

Маса: споряджена вага + екіпаж/пасажери/вантаж як точкові маси. Щодо інерції: якщо у вас є CAD-модель, найпростіше експортувати її в інструмент кінематики (SolidWorks,

Onshape, Fusion 360), який видає вам $I_{xx,yy,zz,xy,xz,yz}$ відносно CG безпосередньо. Інакше оцініть за радіусами інерції (Roskam т. V, таблиці 9.1-9.4).

Шаци: розмістіть кожну точку контакту в конструктивній системі. Вибір пружності/демпфування — частково мистецтво, частково розрахунок: орієнтуйтеся на статичне обтиснення 2-4 дюйми під вагою на колесах і коефіцієнт демпфування близько 0,3-0,5. Швидка оцінка:

$$k = W_{\text{gear}} / \Delta_{\text{static}} \quad c = 2 \zeta \sqrt{(k \cdot W_{\text{gear}} / g)}$$

7.4 Крок 4: Силова установка

Виберіть файл двигуна, що відповідає вашому типу. Стандартна бібліотека двигунів JSBSim має гарні відправні точки: eng_io320 (160 к.с., 4-циліндровий), CFM56 (турбовентиляторний), F100-PW-229 (турбовентиляторний із форсажем), Estes_E9 (модельна ракета), DJI_E305 (електричний для дрона). Для нового двигуна скопіюйте наявний файл і відредагуйте робочий об'єм / максимальну потужність / BSFC / діапазон обертів. Поєднайте двигун із рушієм — direct для реактивних двигунів/ракет, prop_* для повітряних гвинтів.

```
<propulsion>
  <engine file="eng_io360">                                <!-- 180 hp -->
    <feed>0</feed>
    <feed>1</feed>
    <thruster file="prop_77in">
      <location unit="IN"> <x>-30</x><y>0</y><z>2</z> </location>
      <orient unit="DEG"><pitch>2</pitch></orient> <!-- thrust line up 2° -->
    </thruster>
  </engine>
  <tank type="FUEL" number="0">
    <location unit="IN"><x>0</x><y>-80</y><z>30</z></location>
    <capacity unit="LBS">300</capacity>
    <contents unit="LBS">280</contents>
  </tank>
  <tank type="FUEL" number="1">
    <location unit="IN"><x>0</x><y>80</y><z>30</z></location>
    <capacity unit="LBS">300</capacity>
    <contents unit="LBS">280</contents>
  </tank>
</propulsion>
```

7.5 Крок 5: Система керування польотом

Найпростіша можлива FCS просто відображає fcs/elevator-cmd-norm (−1..+1) на fcs/elevator-pos-rad через aerosurface_scale. Реальні літаки додають суматори балансування, обмеження швидкості, зони нечутливості, фільтри поривів, автопілоти — але починайте з мінімуму, доб'йтеся, щоб літак летів у розімкненому контурі, а потім нарощуйте складність.

7.6 Крок 6: Аеродинаміка з CFD

Це найбільш трудомісткий крок. Загальна процедура:

- **1. Визначте сітку розгортки.** Розумна дозвучова сітка — це $\alpha \in \{-6, -4, -2, 0, 2, 4, 6, 8, 10, 12, 14, 16, 18\}$ deg, $\beta \in \{-15, -10, -5, 0, 5, 10, 15\}$ deg, число Маха $\in \{0.15, 0.30, 0.50, 0.70\}$, відхилення органів керування по 5-7 точок кожне. Це приблизно 3000 випадків CFD для повної пошукової матриці; на практиці ви розв'яжете їх роздільно.
- **2. Обрахуйте усталені випадки RANS** у вашому інструменті CFD на вибір (OpenFOAM, Fluent, STAR-CCM+, SU2). Для кожного випадку витягніть $F_x, F_y, F_z, M_x, M_y, M_z$ у зв'язаній системі відносно AERORP і перетворіть на коефіцієнти, поділивши на $\bar{q} \cdot S, \bar{q} \cdot S \cdot b, \bar{q} \cdot S \cdot \bar{c}$.

- **3. Розв'яжуйте роздільно, де можливо.** Зазвичай беріть C_L , C_D , C_m з α -розгортки за $\beta=0$; беріть C_Y , C_l , C_n з β -розгортки за $\alpha=0$; беріть похідні керованості як нахил моменту залежно від відхилення в умовах балансування. Для моделей із великими α чи трансзвукових зв'язаність реальна, і повна 2-вимірна чи 3-вимірна таблиця вам справді потрібна.
- **4. Обрахуйте випадки похідних демпфування** — або вимушені коливання, або квазістатичні крен/рискання/тангаж зі сталою швидкістю. Витягніть C_{lp} , C_{mq} , C_{nr} , C_{lr} , C_{nr} , $C_{m\dot{\alpha}}$.
- **5. Табулюйте та валідуйте.** Побудуйте графік кожного коефіцієнта як функції кожної незалежної змінної. Шукайте немонотонну поведінку, якої ви не очікували — зазвичай це ознака проблем із сіткою чи збіжністю.
- **6. Перетворіть на XML JSBSim.** Формат — це проста матриця чисел у `<tableData>`. Скрипт Python у наступному розділі автоматизує це.

7.7 Крок 7: Початкові умови та скрипт димового тесту

```
<!-- reset00.xml -->
<?xml version="1.0"?>
<initialize name="reset00">
  <ubody unit="FT/SEC"> 150 </ubody>      <!-- ~100 KT -->
  <vbody unit="FT/SEC"> 0 </vbody>
  <wbody unit="FT/SEC"> 0 </wbody>
  <latitude unit="DEG"> 37.6 </latitude>
  <longitude unit="DEG">-122.4</longitude>
  <altitude unit="FT"> 5000 </altitude>
  <phi unit="DEG"> 0 </phi>
  <theta unit="DEG">2 </theta>
  <psi unit="DEG"> 90 </psi>              <!-- heading east -->
</initialize>

<!-- scripts/MyAcft_smoke.xml -->
<?xml version="1.0"?>
<runscript name="MyAcft smoke">
  <use aircraft="MyAcft" initialize="reset00"/>
  <run start="0" end="120" dt="0.00833333">
    <event name="Trim">
      <condition>simulation/sim-time-sec ge 0</condition>
      <set name="simulation/do_simple_trim" value="1"/>
      <notify/>
    </event>
    <event name="Pitch doublet">
      <condition>simulation/sim-time-sec ge 30</condition>
      <set name="fcs/elevator-cmd-norm" value="0.2" type="FG_VALUE"/>
      <notify/>
    </event>
    <event name="Pitch return">
      <condition>simulation/sim-time-sec ge 32</condition>
      <set name="fcs/elevator-cmd-norm" value="0"/>
      <notify/>
    </event>
  </run>
</runscript>
```

Запустіть це командою:

```
JSBSim --script=scripts/MyAcft_smoke.xml --logdirectivefile=...
```

7.8 Крок 8: Цикл ітерацій

- **Чи балансується?** Якщо `do_simple_trim` не вдається, перевірте домовленості про знаки (руль висоти створює правильний напрям моменту тангажа), перевірте, що $C_{m\alpha} < 0$ (статична стійкість за тангажем), і перевірте, що діапазон α ваших таблиць покриває α балансування.

- **Чи летить прямо?** Статична бічно-курсва стійкість потребує $C_{n\beta} > 0$ та $C_{l\beta} < 0$. Якщо літак звалюється в крен, перевірте ефект поперечного V.
- **Чи реагує як справжній літак?** Порівняйте власні значення фугоїда, короткоперіодичного руху, голландського кроку, режиму крену та спірального руху з опублікованими даними чи льотними випробуваннями.
- **Чи приземляється?** Налаштовуйте пружність/демпфування шасі, доки відскок не виглядатиме правильно. Якщо літак провалюється крізь ЗПС, ваша пружина заслабка або точка контакту зависока (Z надто додатне в конструктивній системі).

Розділ 8: Робочий процес CFD-to-JSBSim

У цьому розділі описано рекомендований робочий процес генерування аеродинамічних таблиць JSBSim із даних CFD, включно з конкретними випадками для обрахунку, формулами до застосування та еталонним скриптом Python для перетворення результатів на XML JSBSim.

8.1 Вибір розв'язувача CFD

- **OpenFOAM** (відкритий код) — simpleFoam для усталеного RANS; pimpleFoam з overset для рухомих сіток. Безкоштовний, але якість сітки та вибір моделі турбулентності — на вас.
- **SU2** (відкритий код) — сучасний, із підтримкою спряжених методів, добрий як для оптимізації форми, так і для аналізу.
- **ANSYS Fluent / STAR-CCM+** — комерційні, дуже зрілі. Варті ліцензії, якщо вона у вас є.
- **NASA OVERFLOW / FUN3D** — структурований/неструктурований RANS дослідницького рівня, доступний університетам США.
- **XFLR5 / AVL** (вихрова ґратка) — швидкі, на диво точні для дозвукових оцінок на ранніх етапах проєктування. Використовуйте їх для початкового наближення, перш ніж витратити бюджет на CFD.

8.2 Рекомендована матриця випадків

Для класичного дозвукового літального апарата повна модель потребує наведених нижче випадків. Кожен — це окремий прогін CFD.

Група	Розгортка	α (deg)	β (deg)	Результат
Поздовжня статика	α у чистій конфігурації, $\beta=0$	від -6 до $+18$ крок 2	0	$C_L(\alpha)$, $C_D(\alpha)$, $C_m(\alpha)$
Поздовжня із закрилками	$\alpha \times \text{flap}$	від -4 до $+14$ крок 2	0	ΔC_L , ΔC_D , ΔC_m залежно від flap
Поздовжня з рулем висоти	$\alpha \times \delta_e$	від -4 до $+14$ крок 2	0	$C_{m_{\delta e}}(\alpha, \delta_e)$
Бічна статика	β за α_{cruise}	α_{cr}	від -15 до $+15$ крок 3	$C_Y(\beta)$, $C_l(\beta)$, $C_n(\beta)$
Елерони	$\beta \times \delta_a$	α_{cr}	від -10 до $+10$	$C_{l_{\delta a}}$, $C_{n_{\delta a}}$
Руль напрямку	$\beta \times \delta_r$	α_{cr}	від -10 до $+10$	$C_{Y_{\delta r}}$, $C_{n_{\delta r}}$
Стисливість	число Маха	0	0	$\Delta C_D(M)$, $\Delta C_m(M)$
Вплив екрана	$h/\bar{c}$	α_{cr}	0	$k_{CL,ge}(h/\bar{c})$, $k_{CD,ge}(h/\bar{c})$
Демпфування тангажа	вимуш. кол. навколо Y	α_{cr}	0	C_{mq} , $C_{m_{\dot{\alpha}}}$
Демпфування крену	вимуш. кол. навколо X	α_{cr}	0	C_{lp} , C_{Yp} , C_{np}
Демпфування ристання	вимуш. кол. навколо Z	α_{cr}	0	C_{nr} , C_{Yr} , C_{lr}

Якщо у вас є трансзвукові чи надзвукові режими, додайте число Маха як третю вісь до всіх поздовжніх випадків. Для винищувача чи ударного БПЛА (UCAV), що працює на великих α , подвоюйте діапазон α та крок.

8.3 Метод вимушених коливань для похідних демпфування

Найчистіший спосіб отримати C_{mq} тощо з CFD — змусити геометрію коливатися синусоїдально навколо відповідної осі з малою амплітудою та відомою зведеною частотою, а потім підібрати синфазну та квадратурну складові моменту. Для демпфування тангажа:

$$\theta(t) = \theta_0 + \Delta\theta \cdot \sin(\omega t)$$

$$q(t) = \Delta\theta \cdot \omega \cdot \cos(\omega t)$$

$$C_m(t) = C_{m,0} + C_{m_\alpha} \cdot \Delta\alpha \cdot \sin(\omega t) + [C_{mq} + C_{m_{\dot{\alpha}}}] \cdot (\bar{c}/2V) \cdot \Delta\theta \cdot \omega \cdot \cos(\omega t)$$

Підбирайте методом найменших квадратів: коефіцієнт при $\cos$ дорівнює $(C_{mq} + C_{m_{\dot{\alpha}}}) \cdot (\bar{c}/2V) \cdot \Delta\theta \cdot \omega$, коефіцієнт при $\sin$ дорівнює $C_{m_\alpha} \cdot \Delta\alpha$. Відокремлення C_{mq} від $C_{m_{\dot{\alpha}}}$ потребує додаткових коливань занурення (α змінюється без зміни q). На практиці багато довідників просто публікують суму $(C_{mq} + C_{m_{\dot{\alpha}}})$ і розбивають її приблизно як 70/30.

Рекомендовані амплітуди та зведені частоти для транспортного літака: $\Delta\theta = 1^\circ$, $k = \omega\bar{c}/(2V) \in [0.01, 0.1]$. П'ять чи десять циклів у CFD перед виокремленням підгонки.

8.4 Квазістатичний метод (швидший)

Обрахуйте кілька усталених випадків CFD зі сталою кутковою швидкістю у зв'язаній системі, прикладеною як обертова система відліку. Побудуйте графік отриманого моменту залежно від $rb/(2V)$ (або $q\bar{c}/(2V)$, чи $rb/(2V)$); нахил — це похідна демпфування. Це значно дешевше за URANS, але справедливо лише для малих швидкостей та нестисливого потоку. Достатньо для моделі першого наближення.

8.5 Виокремлення та табулювання

Кожен випадок CFD дає вам F та M у зв'язаній системі відносно відомої точки. Перетворіть на коефіцієнти у зв'язаній системі відносно AERORP, а потім поверніть коефіцієнти сил у швидкісну вісь, якщо ваша модель JSBSim використовує LIFT/DRAG/SIDE:

$$C_L = C_Z \cos \alpha - C_X \sin \alpha$$

$$C_D = -C_X \cos \alpha - C_Z \sin \alpha$$

$$C_Y \text{ (unchanged from body)}$$

8.6 Допоміжний скрипт Python: CSV із CFD → XML JSBSim

Мінімальний генератор, що перетворює CSV зі стовпцями `alpha_deg`, `flap_deg`, `CL`, `CD`, `Cm` на блок функцій LIFT/DRAG/PITCH у JSBSim:

```
import pandas as pd

df = pd.read_csv('cfd_results.csv')
alphas = sorted(df.alpha_deg.unique())
flaps = sorted(df.flap_deg.unique())

def emit_table(name, indvar_row, indvar_col, rows, cols, values):
    out = ['<table>',
```



```

        f' <independentVar lookup="row">{indvar_row}</independentVar>',
        f' <independentVar lookup="column">{indvar_col}</independentVar>',
        ' <tableData>']
header = ' ' + ' '.join(f'{c:8.3f}' for c in cols)
out.append(header)
for r in rows:
    row = [f'{r:8.4f}'] + [f'{values[(r,c)]:8.4f}' for c in cols]
    out.append(' ' + ' '.join(row))
out += [' </tableData>', '</table>']
return '\n'.join(out)

CL = {(r,c): df[(df.alpha_deg==r)&(df.flap_deg==c)].CL.iloc[0]
      for r in alphas for c in flaps}

print('<function name="aero/coefficient/CL">')
print(' <product>')
print(' <property>aero/qbar-psf</property>')
print(' <property>metrics/Sw-sqft</property>')
alpha_rad = sorted([a*3.14159/180 for a in alphas])
print(emit_table('CL', 'aero/alpha-rad', 'fcs/flap-pos-deg',
                alpha_rad, flaps, CL))
print(' </product>')
print('</function>')

```

8.7 Валідація: від чисел CFD до відчуття польоту

Три класи валідації, у порядку зростання складності:

- **Графіки коефіцієнтів** — накладіть коефіцієнти JSBSim на вихідні дані CFD. Вони мають збігатися за побудовою; якщо ні, у вас помилка в одиницях.
- **Аналіз власних значень** — лінеаризуйте модель JSBSim на крейсерському режимі за допомогою вбудованого інструмента `python/JSBSim/utils/linearize`. Порівняйте корені фугоїда, короткоперіодичного руху, голландського кроку, крену та спірального руху з підручковими оцінками (Stevens-Lewis, розд. 5) чи опублікованими даними для подібних літаків.
- **Зіставлення маневрів** — виконайте дублети тангажа, бочки елеронами, удари рулем напрямку. Побудуйте графіки p , q , r , α , β . Порівняйте з льотними випробуваннями чи задокументованими пілотажними характеристиками (Cooper-Harper, MIL-F-8785C).

Розділ 9: Довідкові додатки

9.1 Шпаргалка з дерева властивостей

```
# Pilot inputs (writable from any external program / script)
fcs/elevator-cmd-norm      -1..+1, positive = nose-up command
fcs/aileron-cmd-norm       -1..+1, positive = roll right
fcs/rudder-cmd-norm        -1..+1, positive = yaw right
fcs/throttle-cmd-norm[n]   0..1 per engine
fcs/mixture-cmd-norm[n]    0..1 per piston engine
gear/gear-cmd-norm         0 or 1
fcs/flap-cmd-norm          0..1
fcs/speedbrake-cmd-norm    0..1

# Aero state
aero/alpha-rad / alpha-deg
aero/beta-rad / beta-deg
aero/qbar-psf
velocities/vt-fps          true airspeed
velocities/vc-kts          calibrated airspeed
velocities/mach
velocities/p-rad_sec       body-frame angular rates
velocities/p-aero-rad_sec  aero-frame angular rates

# Position and attitude
position/lat-geod-deg, position/long-gc-deg
position/h-sl-ft           altitude above MSL
position/h-agl-ft          altitude above ground
attitude/phi-rad, theta-rad, psi-rad

# Forces and moments (body frame, lbf, ft-lbf)
forces/fbx-aero, fby-aero, fbz-aero
forces/fbx-prop, fby-prop, fbz-prop
moments/l-aero, m-aero, n-aero
moments/l-total, m-total, n-total
```

9.2 Поширені компоненти FCS

Компонент	Призначення
summer	Підсумовує входи з опціональним зсувом та обмеженням.
pure_gain	$y = k * x$ (k може бути властивістю)
scheduled_gain	$k = \text{table}(\text{independent_var})$; $y = k * x$
aerosurface_scale	Відображає нормований вхід на фізичний діапазон відхилення.
kinematic	Дискретні положення з часами переходу — закритки, шасі.
lag_filter	Запізнення першого порядку: $\dot{y} = (x - y) / \tau$
lead_lag_filter	Неперервний $(a_0 + a_1 s) / (b_0 + b_1 s)$
washout_filter	Верхньочастотний: $\tau s / (\tau s + 1)$
second_order_filter	Режекторний/низькочастотний з 4 коефіцієнтами чисельника + 4 знаменника
integrator	$y = \int x \, dt$, з опціональним тригером скидання
deadband	Видає нуль у межах смуги навколо входу
switch	Умовна логіка з випадками test/default
fcs_function	Вбудовує довільний вираз-функцію як блок FCS
pid	PID(K_p, K_i, K_d) з тригером, опціональним clip-to
sensor	Додає шум, зсув, дрейф, запізнення, затримку
actuator	Обмеження швидкості, запізнення, гістерезис, зона нечутливості, режими відмов

9.3 Перехресний довідник типів двигунів

Клас двигуна	Кореневий тег XML	Типовий рушій	Ключові входи
Поршневий			дросель, суміш, магнето
Турбореактивний/Турбовентиляторний		або	дросель, форсаж вкл/викл
Турбогвинтовий			дросель, крок гвинта
Ракетний			дросель (часто 0/1)
Електричний пост. струму			дросель (PWM)
Безколекторний пост. струму			дросель, обмеження струму
Гвинт		(інтегрований)	загальний крок, циклічний крок, дросель

9.4 Карта вихідного коду

```

src/FGFDMExec.cpp / .h          main executive, model orchestration
src/initialization/FGInitialCondition.cpp
                                IC parsing and application
src/initialization/FGTrim.cpp    trim algorithm
src/input_output/FGPropertyManager.cpp
                                property tree bindings
src/input_output/FGInputType*.cpp
                                telnet/socket/QtJSBSim input drivers
src/input_output/FGOutputType*.cpp
                                CSV, FlightGear, socket output drivers
src/math/FGFunction.cpp          function-language evaluator
src/math/FGTable.cpp             N-D table interpolation
src/math/FGPropertyValue.cpp     property-bound value node
src/math/FGQuaternion.cpp        quaternion arithmetic
src/math/FGMatrix33.cpp          3x3 matrix
src/math/FGColumnVector3.cpp     3-vector
src/models/FGPropagate.cpp        state integration
src/models/FGAccelerations.cpp    Newton-Euler, ground LCP
src/models/FGAerodynamics.cpp     aero force/moment assembly
src/models/FGAuxiliary.cpp        alpha, beta, qbar, Mach
src/models/FGAtmosphere.cpp       standard atmosphere
src/models/atmosphere/FGWinds.cpp winds, gusts, turbulence
src/models/FGInertial.cpp         gravity, WGS-84, planet rotation
src/models/FGMassBalance.cpp      mass/CG/inertia composition
src/models/FGPropulsion.cpp       engine container
src/models/propulsion/FGPiston.cpp piston engine
src/models/propulsion/FGTurbine.cpp turbojet/turbofan
src/models/propulsion/FGTurboProp.cpp turboprop
src/models/propulsion/FGRocket.cpp rocket
src/models/propulsion/FGElectric.cpp electric
src/models/propulsion/FGBrushLessDCMotor.cpp
src/models/propulsion/FGPropeller.cpp propeller
src/models/propulsion/FGNozzle.cpp rocket nozzle
src/models/propulsion/FGRotor.cpp helicopter rotor
src/models/propulsion/FGTank.cpp  fuel tank
src/models/FGGroundReactions.cpp ground reactions container
src/models/FGLEGear.cpp           landing gear / contact point
src/models/flight_control/...     FCS components (summer, pid, ...)
src/models/FGFCS.cpp              flight control system orchestrator

```

9.5 Бібліографія та джерела

- Berndt, J. S. JSBSim Reference Manual (онлайн та PDF).
<https://jsbsim.sourceforge.net/documentation.html>

- Stevens B. L., Lewis F. L. Aircraft Control and Simulation, 2nd ed., Wiley, 2003. — визначальне джерело щодо рівнянь руху, які реалізує JSBSim.
- Etkin B., Reid L. D. Dynamics of Flight: Stability and Control, 3rd ed., Wiley, 1996. — класичні визначення похідних стійкості.
- Roskam, J. Airplane Design, Vols. I-VIII, DARcorp. — інженерні оцінки інерції, шасі, геометрії органів керування.
- USAF DATCOM, USAF Stability and Control DATCOM, USAF AFFDL-TR-79-3032. — довідникові оцінки похідних стійкості та демпфування; корисна перевірка адекватності CFD.
- Cooke J., Zyda M., Pratt D., McGhee R. NPSNET: Flight Simulation Dynamic Modeling Using Quaternions, 1994.
- Buss S. Accurate and Efficient Simulation of Rigid Body Rotations, UCSD, 1999.
- Catto E. Iterative Dynamics with Temporal Coherence, Crystal Dynamics, 2005.
- U.S. Standard Atmosphere, 1976, NASA-TM-X-74335.
- NASA-NESC. 2015 Flight Simulation Check Cases.
<https://nescacademy.nasa.gov/flightsim/2015>
- MIL-F-8785C, Military Specification, Flying Qualities of Piloted Airplanes.
- MIL-HDBK-516B, Department of Defense Handbook: Airworthiness Certification Criteria.

9.6 Підсумкові міркування

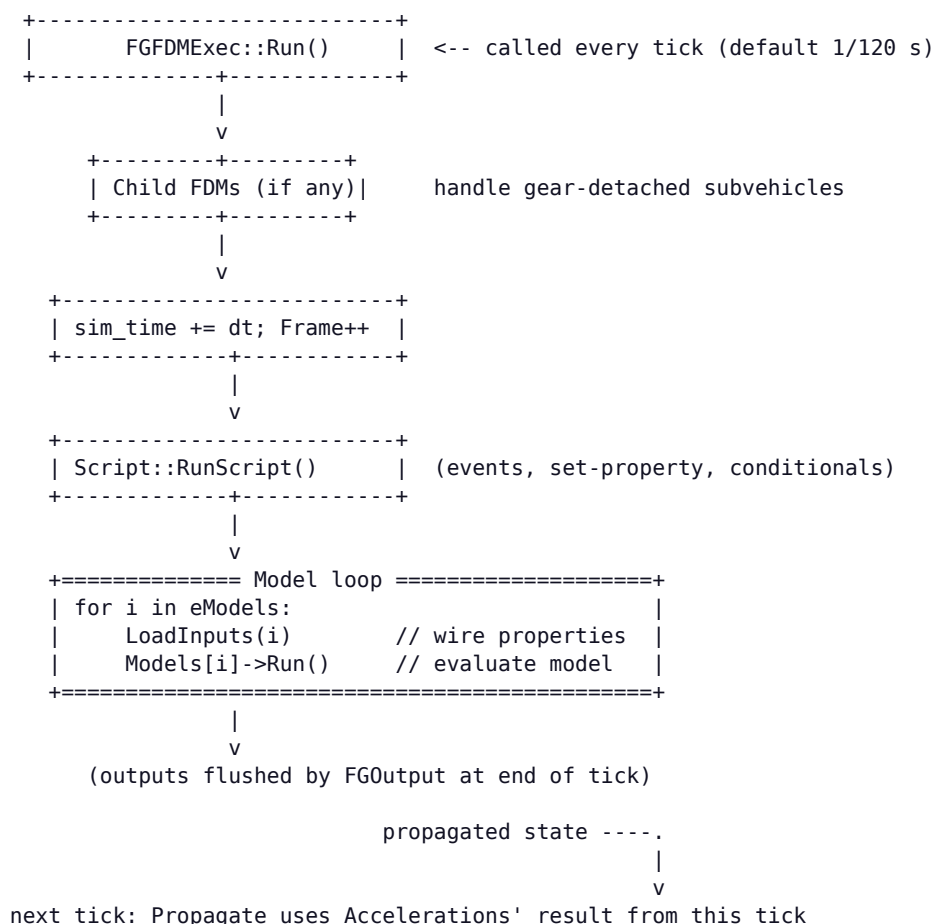
Дизайн JSBSim винагороджує інженерів, які вже мислять у термінах похідних стійкості, систем відліку та рівнянь руху: прихованої магії тут дуже мало. Більшість того, що в XML здається механічним налаштуванням, насправді є прямим вираженням фізики — функція, що множить коефіцієнт на $\dot{q} \cdot S \cdot \dot{c} \dots$, є підруничковою формулою, а не домовленістю JSBSim. Наслідок полягає в тому, що коли щось іде не так, помилка майже завжди в даних, одиницях чи знаках — а не в симуляторі. Довіряйте фреймворку, інструментуйте його (дерево властивостей робить інструментування тривіальним) та ітеруйте.

Модель хибна, але симулятор має рацію. — фольклор

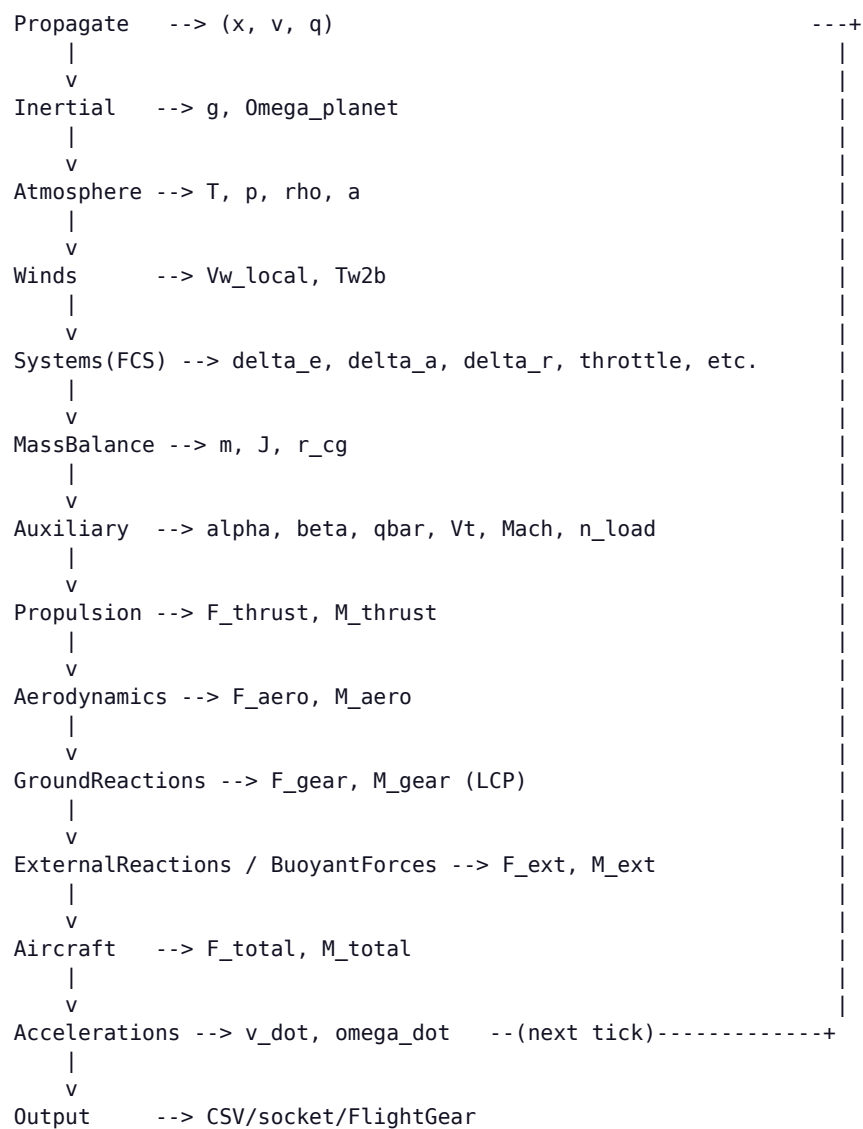
Розділ 10: Візуальні діаграми циклу FDM

JSBSim не має вбудованої графічної документації. ASCII-діаграми в цьому розділі реконструйовано з вихідного коду й вони достовірно відображають потік даних станом на v2.0.

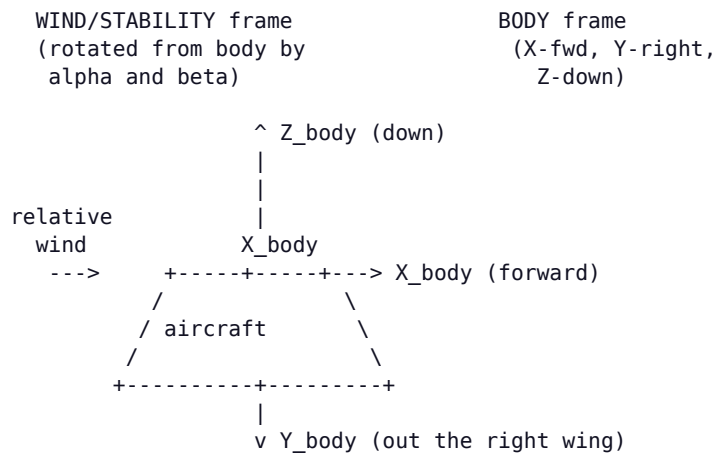
10.1 Цикл симуляції верхнього рівня



10.2 Потік даних між моделями за один такт



10.3 Схема зв'язаної, вітрової та конструктивної систем координат



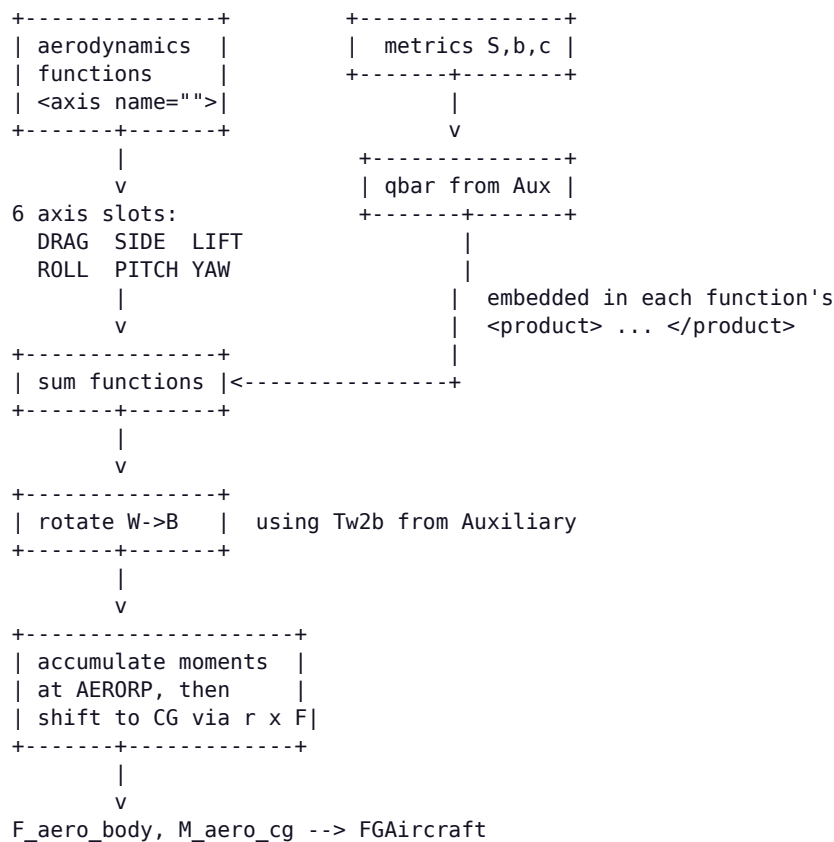
alpha = angle between V_{rel} and the body X axis in the X-Z plane
 beta = angle between V_{rel} and the body X-Z plane

STRUCTURAL frame (used in <location> elements of the XML):

X_struct = aft (typically),
 Y_struct = right,
 Z_struct = up.

JSBSim flips X and Z internally when converting to body frame.

10.4 Потік від аеродинамічних осей до сил



Розділ 11: Силова установка — довідник за типами двигунів

11.1 Поршневі двигуни

FGPiston моделює чотиритактний поршковий двигун (piston engine) з динамікою абсолютного тиску у впускному колекторі, керуванням сумішшю та дроселем, відмовами магнето й зниженням потужності з висотою. Параметри конфігурації:

```
<piston_engine name="NewEngine">
  <minmp      unit="INHG"> 10.0 </minmp>      <!-- idle MAP -->
  <maxmp      unit="INHG"> 28.5 </maxmp>      <!-- max MAP (SL) -->
  <displacement unit="IN3"> 320 </displacement>
  <maxhp>      160 </maxhp>      <!-- SL max power -->
  <bsfc>       0.45 </bsfc>      <!-- brake-spec fuel csmf -->
  <cycles>      4 </cycles>      <!-- 4 or 2-stroke -->
  <idlerpm>     600 </idlerpm>
  <maxrpm>     2700 </maxrpm>
  <maxthrottle> 1.0 </maxthrottle>
  <minthrottle> 0.05 </minthrottle>
  <sparkfaildrop> 0.1 </sparkfaildrop>
  <numboostspeeds> 1 </numboostspeeds>
  <boostoverride> 0 </boostoverride>
  <ratedpower> 160 </ratedpower>
  <ratedrpm> 2700 </ratedrpm>
  <ratedaltitude unit="FT"> 0 </ratedaltitude>
</piston_engine>
```

Вихідна потужність моделюється як параболічна залежність від MAP і лінійне масштабування за дроселем/RPM. Суміш впливає на висоту за щільністю — збіднення відносно піка зменшує потужність, збагачення відносно піка збільшує витрату палива. Модель враховує відновлення тиску набігаючого потоку на високих швидкостях.

11.2 Турбіна (турбореактивний/турбовентиляторний двигун)

FGTurbine реалізує двовальний двигун із режимами малого газу/максимальної безфорсажної тяги/форсажу та тягою за таблицями пошуку:

```
<turbine_engine name="F100">
  <milthrust unit="LBS"> 17800 </milthrust>
  <maxthrust unit="LBS"> 29100 </maxthrust> <!-- AB rating -->
  <bypassratio> 0.35 </bypassratio>
  <tsfc> 0.74 </tsfc> <!-- cruise SFC -->
  <atsfc> 2.05 </atsfc> <!-- AB SFC -->
  <bleed> 0.03 </bleed>
  <idlen1> 30.0 </idlen1>
  <idlen2> 60.0 </idlen2>
  <maxn1> 100.0 </maxn1>
  <maxn2> 100.0 </maxn2>
  <augmented> 1 </augmented> <!-- has AB -->
  <augmethod> 1 </augmethod>
  <injected> 0 </injected>

  <function name="IdleThrust">
    <table>
      <independentVar lookup="row">velocities/mach</independentVar>
      <independentVar lookup="column">atmosphere/density-altitude</independentVar>
      <tableData>
        0 20000 40000 60000
        0.0 0.0430 0.0488 0.0533 0.0593
        0.4 0.0356 0.0418 0.0466 0.0524
        0.8 0.0235 0.0312 0.0376 0.0445
        1.2 0.0070 0.0185 0.0272 0.0364
```



```

        1.6      -0.0094   0.0040   0.0156   0.0276
      </tableData>
    </table>
  </function>

  <function name="MilThrust">
    <table>...</table>      <!-- normalised thrust 0..1 -->
  </function>

  <function name="AugThrust">
    <table>...</table>      <!-- AB normalised thrust -->
  </function>
</turbine_engine>

```

Кожна таблиця тяги нормована на 1.0 для статичних умов на рівні моря (SL); модель множить її на відповідне номінальне значення (milthrust або maxthrust). Розкрутка моделюється запізненнями першого порядку; властивість `propulsion/engine[n]/n2-norm` можна подати до FCS, щоб керувати випуском закрилків або логікою реверсера тяги.

11.3 Турбогвинтовий двигун

FGTurboProp поєднує турбінне ядро з повітряним гвинтом. XML додає `maxpower` у потужності на валу (к. с.) та `betarangeend`:

```

<turboprop_engine name="PW100">
  <milthrust unit="LBS"> 0 </milthrust>
  <maxpower unit="HP"> 1200 </maxpower>
  <idlen1> 50 </idlen1>
  <maxn1> 100 </maxn1>
  <betarangeend> 0.20 </betarangeend> <!-- pitch override below this -->
  <reversemaxpower> 0.50 </reversemaxpower>
  <function name="EnginePowerVC">
    <table>...</table>
  </function>
  <function name="EnginePowerAltitude">
    <table>...</table>
  </function>
</turboprop_engine>

```

Рушій має бути `<propeller>` (окремий файл XML), а крок гвинта керується або регулятором (постійних обертів), або безпосередньо командами FCS.

11.4 Ракетні двигуни

FGRocket моделює ракету з фіксованим `Isp`. Змінна тяга підтримується через дросель і таблицю «тяга-час»; векторування тяги підтримується через `<nozzle>` з керованими кутами PYR. Придатний для прискорювачів, метеорологічних ракет і моделей ракет.

```

<rocket_engine name="SolidBooster">
  <isp> 265.0 </isp> <!-- specific impulse, sec -->
  <builduptime> 0.2 </builduptime> <!-- ignition transient -->
  <thrust_table>
    <table> <!-- F vs t in sec -->
      <independentVar>propulsion/engine[0]/thrust-time</independentVar>
      <tableData>
        0.0 1500
        0.5 3200
        3.0 3200
        4.0 1000
        4.5 0
      </tableData>
    </table>
  </thrust_table>
</rocket_engine>

```

11.5 Електричні та безколекторні двигуни постійного струму

```
<electric_engine name="E305">
  <power unit="WATTS"> 1000 </power>  <!-- max shaft power -->
</electric_engine>

<brushless_dc_motor name="OutrunnerBLDC">
  <maxvolts> 22.2 </maxvolts>  <!-- 6S LiPo -->
  <velocityconstant> 920 </velocityconstant>  <!-- Kv, rpm/V -->
  <coilresistance> 0.02 </coilresistance>  <!-- ohms -->
  <noloadcurrent> 1.0 </noloadcurrent>  <!-- A -->
</brushless_dc_motor>
```

Обидві моделі обчислюють момент на валу з дроселя (PWM) та кривої проти-ЕРС. FGBrushLessDCMotor явно відстежує споживаний струм, що корисно для моделювання ресурсу акумулятора — помножте струм на напругу на клеммах, щоб отримати миттєву потужність, а потім інтегруйте її відносно ємності бака, щоб змодельовати рівень заряду акумулятора.

11.6 Несучий гвинт вертольота

FGRotor моделює несучий або хвостовий гвинт вертольота за допомогою аналітичного підходу елемента лопаті з моделюванням індуктивного потоку, динаміки махових рухів і впливу екрана. Він складний — для першого наближення починайте з прикладів J3Cub або X15 у aircraft/, а не з нуля.

11.7 Повітряні гвинти

Повітряні гвинти розміщуються в engine/prop_*.xml і посилаються на таблиці C_T та C_P залежно від відносної ходи J :

```
<propeller name="prop_75in2f">
  <ixx> 1.67 </ixx>
  <diameter unit="IN"> 75.0 </diameter>
  <numblades> 2 </numblades>
  <gearratio> 1.0 </gearratio>
  <cp_factor> 1.0 </cp_factor>
  <ct_factor> 1.0 </ct_factor>
  <minpitch> 10.0 </minpitch>  <!-- variable-pitch range -->
  <maxpitch> 35.0 </maxpitch>
  <minrpm> 640 </minrpm>  <!-- governor range -->
  <maxrpm> 2700 </maxrpm>
  <constspeed> 1 </constspeed> <!-- 1 = constant-speed prop -->
  <table name="C_THRUST" type="internal">
    <tableData>
      0.00 0.0863
      0.10 0.0837
      0.20 0.0792
      0.30 0.0727
      0.40 0.0643
      0.50 0.0540
    </tableData>
  </table>
  <table name="C_POWER" type="internal">
    <tableData> ... </tableData>
  </table>
</propeller>
```

Використовуйте p_factor на рушії, щоб змодельовати незначний момент рискання вліво від асиметрії диска гвинта на великих α . $sense$ дорівнює $+1$ для обертання за годинниковою стрілкою (якщо дивитися з кабіни) і -1 — проти годинникової стрілки; це важливо для напрямку реактивного моменту та ефектів «критичного двигуна» на двомоторних літаках.

11.8 Паливні баки

```
<tank type="FUEL" number="0" name="LeftWingTank">
  <location unit="IN"> <x>56</x><y>-112</y><z>59.4</z> </location>
  <drain_location unit="IN">
    <x>56</x><y>-112</y><z>50</z> <!-- pickup point -->
  </drain_location>
  <type>AVGAS</type> <!-- or JET-A, RP-1, ... -->
  <capacity unit="LBS"> 185 </capacity>
  <contents unit="LBS"> 100 </contents> <!-- initial fuel -->
  <density unit="LBS/GAL"> 6.0 </density>
  <temperature> 50 </temperature> <!-- degF -->
  <standpipe unit="LBS"> 5 </standpipe> <!-- unusable bottom -->
  <unusable-volume unit="GAL"> 0.5 </unusable-volume>
  <priority> 1 </priority> <!-- 1 = consume first -->
</tank>
```

Баки додають масу та інерцію (перенесену за теоремою про паралельні осі з розташування бака до CG), які зменшуються в міру споживання палива. Кілька баків із різними пріоритетами дають змогу змодельовати реальний розклад роботи паливної системи (наприклад, «спочатку перекачувати з допоміжного бака до основного»).

Розділ 12: Компоненти системи керування польотом — довідник

Кожен компонент FCS має однакову структуру: ім'я, одну або кілька властивостей `<input>`, специфічну для компонента конфігурацію, необов'язковий `<clipto>` та властивість `<output>`, яку можуть читати інші компоненти, розташовані нижче за потоком. Вихід обчислюється в тому порядку, у якому компоненти йдуть у каналі.

12.1 Суматор / Чистий коефіцієнт підсилення / Масштабування аероповітряної поверхні

```
<summer name="trim_sum">                                <!-- y = sum(inputs) + bias -->
  <input>fcs/elevator-cmd-norm</input>
  <input>fcs/pitch-trim-cmd-norm</input>
  <bias>0.0</bias>
  <clipto> <min>-1</min><max>1</max> </clipto>
</summer>

<pure_gain name="elev_gain">                             <!-- y = k * x -->
  <input>fcs/trim_sum</input>
  <gain>2.5</gain>                                       <!-- can be a property -->
</pure_gain>

<scheduled_gain name="yaw_damper_gain"> <!-- k = table(prop) -->
  <input>velocities/r-aero-rad_sec</input>
  <table>
    <independentVar>velocities/mach</independentVar>
    <tableData>
      0.0    0.0
      0.3    1.0
      0.9    1.2
    </tableData>
  </table>
  <output>fcs/yaw-damper-cmd</output>
</scheduled_gain>

<aerosurface_scale name="elev_to_rad">
  <input>fcs/trim_sum</input>
  <gain>0.01745</gain>                                <!-- multiply input first -->
  <domain> <min>-1</min><max>1</max> </domain>         <!-- input range -->
  <range>  <min>-28</min><max>23</max> </range>         <!-- output range -->
  <output>fcs/elevator-pos-rad</output>
</aerosurface_scale>
```

12.2 Фільтри

Фільтри використовують дискретизацію за Таїніном (білінійну). Кожен має характеристичну сталу $c_1 \dots c_5$, яка відповідає підручковим коефіцієнтам.

```
<lag_filter name="y">                                    <!-- H = c1/(s+c1) -->
  <input>fcs/x</input>
  <c1>500</c1>                                           <!-- time constant 1/500 s -->
</lag_filter>

<lead_lag_filter name="y">                               <!-- H = (c1 s + c2)/(c3 s + c4) -->
  <input>fcs/x</input>
  <c1>0.5</c1> <c2>1</c2> <c3>0.05</c3> <c4>1</c4>
</lead_lag_filter>

<washout_filter name="y">                                <!-- H = s/(s+c1) -->
  <input>velocities/q-rad_sec</input>
  <c1>1.0</c1>
</washout_filter>
```

```

<second_order_filter name="y">                                <!-- H = (c1 s2 + c2 s + c3)/(c4 s2 + c5 s + c6) -->
  <input>fcs/x</input>
  <c1>0</c1><c2>0</c2><c3>4</c3>
  <c4>1</c4><c5>2.0</c5><c6>4</c6>                                <!-- 2 Hz, zeta=0.5 -->
</second_order_filter>

```

12.3 Інтегратор

```

<integrator name="alpha_int">
  <input>aero/alpha-rad</input>
  <c1>1.0</c1>                                <!-- integration gain -->
  <trigger>fcs/integrator-trigger</trigger> <!-- 0=run, 1=hold, -1=reset -->
</integrator>

```

Властивість `trigger` дає змогу реалізувати захист від інтегрального насичення звичайною логікою: утримувати під час насичення, скидати при вимкненні.

12.4 Під-регулятор

```

<pid name="AltitudeHold">
  <input>fcs/altitude-error-ft</input>
  <kp>0.05</kp>
  <ki type="ab3">0.001</ki>                                <!-- ab2/ab3 = Adams-Bashforth -->
  <kd>0.10</kd>
  <trigger>ap/altitude-hold-engaged</trigger>
  <clipto> <min>-0.5</min><max>0.5</max> </clipto>
</pid>

```

12.5 Кінематичний компонент, зона нечутливості, перемикач

```

<kinematic name="flaps">                                <!-- discrete positions w/ traverse times -->
  <input>fcs/flap-cmd-norm</input>
  <traverse>
    <setting><position> 0</position><time>0</time></setting>
    <setting><position>10</position><time>2</time></setting>
    <setting><position>30</position><time>4</time></setting>
  </traverse>
  <output>fcs/flap-pos-deg</output>
</kinematic>

<deadband name="stick_db">                                <!-- zero out small inputs -->
  <input>fcs/elevator-cmd-norm</input>
  <width>0.02</width>
</deadband>

<switch name="gear_switch">                                <!-- conditional output -->
  <default value="0"/>
  <test logic="AND" value="1">
    gear/gear-cmd-norm ge 0.5
    velocities/vc-kts le 250
  </test>
  <output>fcs/gear-extend</output>
</switch>

```

12.6 Огортання датчика та привода

Використовуйте `<sensor>` для імітації недосконалих вимірювань (запізнення, зміщення, шум) і `<actuator>` для імітації недосконалого приведення поверхні (обмеження швидкості, гістерезис, зона нечутливості, режими відмов). Обидва корисні для досліджень із внесенням несправностей.

```

<sensor name="ias_sensor">
  <input>velocities/vc-kts</input>
  <lag>0.5</lag>                                <!-- first-order lag tau -->

```

```
<bias>2</bias>          <!-- additive offset -->
<drift_rate>0.01</drift_rate> <!-- units/sec -->
<gain>1.005</gain>
<noise variation="PERCENT">0.5</noise>
<quantization name="adc12bit">
  <bits>12</bits><min>0</min><max>500</max>
</quantization>
<delay>0.1</delay>      <!-- pure time delay -->
<output>instr/ias-indicated</output>
</sensor>

<actuator name="elev_act">
  <input>fcs/elevator-cmd-rad</input>
  <lag>0.05</lag>
  <rate_limit sense="incr">0.6</rate_limit>
  <rate_limit sense="decr">0.6</rate_limit>
  <bias>0</bias>
  <deadband_width>0.0017</deadband_width>
  <hysteresis_width>0.0035</hysteresis_width>
  <fail_zero> fcs/elev-act-fail-zero </fail_zero>
  <fail_hardover> fcs/elev-act-fail-hard </fail_hardover>
  <fail_stuck> fcs/elev-act-fail-stuck </fail_stuck>
  <output>fcs/elevator-pos-rad</output>
</actuator>
```

Розділ 13: Атмосфера, вітри та Земля

13.1 Стандартна атмосфера ISA 1976

FGStandardAtmosphere обчислює US Standard Atmosphere 1976 — вісім кусково-сталих сегментів градієнта температури від рівня моря до геометричної висоти 86 км. Модель надає:

atmosphere/T-R	static temperature, Rankine
atmosphere/T-sl-R	sea-level reference temperature
atmosphere/T-dev-R	local deviation from standard
atmosphere/P-psf	static pressure, lb/ft ²
atmosphere/P-sl-psf	
atmosphere/rho-slugs_ft3	density
atmosphere/a-fps	speed of sound
atmosphere/density-altitude	
atmosphere/pressure-altitude	

Власна атмосфера. Успадкуйте від FGAtmosphere у C++ і перевизначте Calculate(double altitudeASL). Для більшості потреб достатньо стандартної моделі плюс зсуву температури (atmosphere/delta-T) — зсув T за сталого P дає зсув висоти за щільністю, у такий спосіб зазвичай моделюють ефекти висоти за щільністю (спекотно й високо).

13.2 Вітри та зсув вітру

atmosphere/wind-north-fps	steady wind from FGWinds
atmosphere/wind-east-fps	(NED frame, points TO direction wind blows toward)
atmosphere/wind-down-fps	
atmosphere/wind-mag-fps	magnitude
atmosphere/psiw-rad	wind heading
atmosphere/total-wind-north-fps	steady + turbulence + gust + shear
atmosphere/turb-rate-rad_sec	Dryden/Karman turbulence angular component

Моделі турбулентності: вибираються через atmosphere/turb-type:

0 ttNone	no turbulence
1 ttStandard	legacy white-noise model
2 ttCulp	John Culp's model (used in FlightGear)
3 ttMilspec	MIL-F-8785C Dryden
4 ttTustin	MIL-F-8785C Tustin discrete-Dryden

Для моделей за MIL-spec ви задаєте atmosphere/turbulence/milspec/severity за шкалою 0-7; модель автоматично добирає масштабні довжини та інтенсивності відповідно до специфікації.

13.3 Пориви (дискретні)

Дискретний порив «1-cosine» можна ввімкнути, задавши:

```
atmosphere/cosine-gust-start -> set to 1 to start
atmosphere/cosine-gust-frame -> 1 BODY, 2 WIND, 3 LOCAL
atmosphere/cosine-gust-duration
atmosphere/cosine-gust-magnitude-ft_sec
atmosphere/cosine-gust-startup-duration
atmosphere/cosine-gust-steady-duration
atmosphere/cosine-gust-end-duration
atmosphere/cosine-gust-X-velocity (and Y, Z in chosen frame)
```

13.4 Мікропорив

Тривимірна модель мікропориву (Vicroy + Mulgund) доступна через atmosphere/turbulence-cosine-set. Використовується для тренувальних сценаріїв повторного заходу на посадку та виходу зі зсуву вітру.

13.5 Інерціальна модель: гравітація та обертання Землі

FGInertial публікує вектор гравітації та швидкість обертання планети. Доступні дві моделі гравітації, вибираються через simulation/gravity-model:

```
0 Spherical inverse-square gravity (uniform mass distribution)
1 WGS-84 ellipsoid with J2 term (default)
```

```
simulation/gravitational-torque -> 0/1 (default 0)
  Adds tidal torque on extended bodies; relevant for spacecraft.
```

```
Planet rotation: 7.2921151467e-5 rad/s about ECI Z (sidereal day).
```

```
WGS-84 ellipsoid:
  semi-major axis a = 6378137.0 m
  semi-minor axis b = 6356752.3142 m
  flattening f      = 1/298.257223563
  GM               = 3.986004418e14 m^3/s^2
  J2               = 1.08263e-3
```

Чому це важливо навіть для польоту на малих висотах. Прискорення Коріоліса на крейсерському режимі при Mach 0.8 має порядок 0.003 m/s^2 — мале, але ненульове. JSBSim коректно його враховує, що частково пояснює, чому він проходить контрольні приклади NASA 2015. Якщо порівнювати із симуляторами, що використовують наближення плоскої Землі, очікуйте невеликих усталених відхилень курсу під час тривалих польотів у напрямку схід-захід.

Розділ 14: Орієнтація через кватерніони — вступ до коду

Читати FGPropagate і FGQuaternion значно простіше, якщо ви вже володієте кватерніонами. Цей розділ — стислий повторний огляд із позначеннями, специфічними для JSBSim.

14.1 Означення та домовленості

Одиничний кватерніон — це четвірка $q = (q_0, q_1, q_2, q_3)$ з нормою 1. JSBSim дотримується домовленості Гамільтона ($i^2 = j^2 = k^2 = ijk = -1$); FGQuaternion зберігає їх у порядку (w, x, y, z) — тобто скалярна компонента йде першою.

$$q = \cos(\theta/2) + \hat{n} \cdot \sin(\theta/2)$$

де $\hat{n}$ — вісь обертання, а θ — кут обертання. Кватерніон, який обертає вектор з інерціальної системи координат у зв'язану, — це $q_{i \rightarrow b}$, а еквівалентне пасивне обертання має вигляд

$$v_{\text{body}} = q^* \cdot v_{\text{inertial}} \cdot q$$

(де вектори піднято до чистих кватерніонів, а $\cdot$ позначає добуток Гамільтона). Саме це й попередньо обчислює FGQuaternion::GetTransformationMatrix() як матрицю обертання 3×3 для швидких перетворень між системами координат.

14.2 Кінематичне рівняння

$$\dot{q} = \frac{1}{2} \cdot q \cdot \omega_{b/i}$$

Тут $\omega_{b/i}$ — інерціальна кутова швидкість, виражена як чистий кватерніон (0, p_i , q_i , r_i). FGQuaternion::GetQDot() реалізує це за 12 операцій із рухомою комою.

```
FGQuaternion FGQuaternion::GetQDot(const FGColumnVector3& PQR) const {
    return FGQuaternion(
        -0.5*( data[1]*PQR(eP) + data[2]*PQR(eQ) + data[3]*PQR(eR)),
        0.5*( data[0]*PQR(eP) - data[3]*PQR(eQ) + data[2]*PQR(eR)),
        0.5*( data[3]*PQR(eP) + data[0]*PQR(eQ) - data[1]*PQR(eR)),
        0.5*(-data[2]*PQR(eP) + data[1]*PQR(eQ) + data[0]*PQR(eR))
    );
}
```

14.3 Перенормування

Числове інтегрування $\dot{q}$ за схемами Ейлера чи Адамса-Башфорта не зберігає одиничну норму — з часом кватерніон зсувається з одиничної 3-сфери, що рівнозначно появі паразитного розтягнення. JSBSim перенормовує його кожного такту, ділячи на модуль (FGQuaternion::Normalize()). Інтегратори Buss уникають цього, інтегруючи безпосередньо за допомогою експоненційного відображення:

$$q(t+\Delta t) = q(t) \cdot \exp(\frac{1}{2} \cdot \Delta t \cdot \omega)$$

що є точним для сталого ω і зберігає одиничну норму з точністю до рухомої коми.

14.4 Видобування кутів Ейлера

`FGQuaternion::GetEulerDeg()` повертає звичайні кути Ейлера літака (ϕ , θ , ψ) у градусах за послідовністю 3-2-1 (Z-Y-X, внутрішня). Особлива точка при $\theta = \pm 90^\circ$ неминуча — кватерніони існують саме для того, щоб уникнути особливості під час інтегрування, але вони все одно мусять проєктуватися крізь неї, коли ви зчитуєте кути Ейлера. Для високопілотажних літаків виконуйте всі обчислення у просторі кватерніонів або у просторі DCM і видобуйте кути Ейлера лише наприкінці.

Розділ 15: Повний розібраний приклад: простий літак

Цей розділ проходить кожну секцію мінімального, але придатного до польоту літака — легкого одномоторного Acme A-1 — беручи значення з геометрії за першими принципами, поршневого двигуна, повторно використаного з бібліотеки, та аеродинамічних коефіцієнтів, отриманих за CFD. Повний XML наведено частинами з коментарями.

15.1 Заголовок та довідкові дані

```
<?xml version="1.0"?>
<fdm_config name="AcmeA1" version="2.0" release="ALPHA">

<fileheader>
  <author>Engineering Team</author>
  <filecreationdate>2026-05-24</filecreationdate>
  <description>Acme A-1, single-engine 2-seat sport aircraft.</description>
</fileheader>

<metrics>
  <wingarea unit="FT2"> 140 </wingarea>      <!-- S -->
  <wingspan unit="FT"> 32 </wingspan>        <!-- b -->
  <chord unit="FT"> 4.4 </chord>             <!-- cbar -->
  <htailarea unit="FT2"> 18 </htailarea>
  <htailarm unit="FT"> 14 </htailarm>
  <vtailarea unit="FT2"> 15 </vtailarea>
  <vtailarm unit="FT"> 14 </vtailarm>
  <location name="AERORP" unit="IN">
    <x>40</x><y>0</y><z>30</z>                <!-- 25% MAC -->
  </location>
  <location name="EYEPOINT" unit="IN">
    <x>30</x><y>-10</y><z>50</z>
  </location>
  <location name="VRP" unit="IN">
    <x>0</x><y>0</y><z>0</z>
  </location>
</metrics>
```

15.2 Маса, центрування та інерція

```
<mass_balance>
  <!-- Inertias from CAD about the empty-weight CG, structural frame -->
  <ixx unit="SLUG*FT2"> 800 </ixx>
  <iyy unit="SLUG*FT2"> 1100 </iyy>
  <izz unit="SLUG*FT2"> 1700 </izz>
  <ixz unit="SLUG*FT2"> 0 </ixz>
  <emptywt unit="LBS"> 1250 </emptywt>
  <location name="CG" unit="IN">
    <x>39</x><y>0</y><z>30</z>                <!-- ~24% MAC -->
  </location>
  <pointmass name="Pilot">
    <weight unit="LBS">175</weight>
    <location unit="IN"><x>34</x><y>-12</y><z>32</z></location>
  </pointmass>
  <pointmass name="Passenger">
    <weight unit="LBS">175</weight>
    <location unit="IN"><x>34</x><y> 12</y><z>32</z></location>
  </pointmass>
  <pointmass name="Baggage">
    <weight unit="LBS"> 60</weight>
    <location unit="IN"><x>70</x><y>0</y><z>30</z></location>
  </pointmass>
</mass_balance>
```

15.3 Шасі

```

<ground_reactions>
  <contact type="BOGEY" name="NOSE">
    <location unit="IN"><x>-10</x><y>0</y><z>-20</z></location>
    <static_friction>0.8</static_friction>
    <dynamic_friction>0.5</dynamic_friction>
    <rolling_friction>0.02</rolling_friction>
    <spring_coeff unit="LBS/FT"> 1500 </spring_coeff>
    <damping_coeff unit="LBS/FT/SEC"> 500 </damping_coeff>
    <max_steer unit="DEG">15</max_steer>
    <brake_group>NONE</brake_group>
  </contact>
  <contact type="BOGEY" name="LEFT_MAIN">
    <location unit="IN"><x>55</x><y>-40</y><z>-18</z></location>
    <static_friction>0.8</static_friction>
    <dynamic_friction>0.5</dynamic_friction>
    <rolling_friction>0.02</rolling_friction>
    <spring_coeff unit="LBS/FT"> 4500 </spring_coeff>
    <damping_coeff unit="LBS/FT/SEC">1300 </damping_coeff>
    <brake_group>LEFT</brake_group>
  </contact>
  <contact type="BOGEY" name="RIGHT_MAIN">
    <location unit="IN"><x>55</x><y> 40</y><z>-18</z></location>
    <static_friction>0.8</static_friction>
    <dynamic_friction>0.5</dynamic_friction>
    <rolling_friction>0.02</rolling_friction>
    <spring_coeff unit="LBS/FT"> 4500 </spring_coeff>
    <damping_coeff unit="LBS/FT/SEC">1300 </damping_coeff>
    <brake_group>RIGHT</brake_group>
  </contact>
  <contact type="STRUCTURE" name="LEFT_TIP">
    <location unit="IN"><x>40</x><y>-192</y><z>30</z></location>
    <static_friction>0.2</static_friction>
    <dynamic_friction>0.2</dynamic_friction>
    <spring_coeff unit="LBS/FT"> 20000 </spring_coeff>
    <damping_coeff unit="LBS/FT/SEC"> 2000 </damping_coeff>
  </contact>
  <contact type="STRUCTURE" name="RIGHT_TIP">
    <location unit="IN"><x>40</x><y> 192</y><z>30</z></location>
    <static_friction>0.2</static_friction>
    <dynamic_friction>0.2</dynamic_friction>
    <spring_coeff unit="LBS/FT"> 20000 </spring_coeff>
    <damping_coeff unit="LBS/FT/SEC"> 2000 </damping_coeff>
  </contact>
</ground_reactions>

```

15.4 Силова установка

```
<propulsion>
  <engine file="eng_io360">          <!-- 180 hp 4-cyl -->
    <feed>0</feed>
    <feed>1</feed>
    <thruster file="prop_77in">
      <location unit="IN"><x>-30</x><y>0</y><z>30</z></location>
      <orient unit="DEG"><pitch>2</pitch></orient>
      <sense>1</sense>
      <p_factor>5</p_factor>
    </thruster>
  </engine>
  <tank type="FUEL" number="0">
    <location unit="IN"><x>40</x><y>-80</y><z>32</z></location>
    <capacity unit="LBS">160</capacity>
    <contents unit="LBS">120</contents>
    <type>AVGAS</type>
  </tank>
  <tank type="FUEL" number="1">
    <location unit="IN"><x>40</x><y> 80</y><z>32</z></location>
    <capacity unit="LBS">160</capacity>
    <contents unit="LBS">120</contents>
    <type>AVGAS</type>
  </tank>
</propulsion>
```

15.5 Система керування польотом

```

<flight_control name="FCS: AcmeA1">
  <channel name="Pitch">
    <summer name="pitch_sum">
      <input>fcs/elevator-cmd-norm</input>
      <input>fcs/pitch-trim-cmd-norm</input>
      <clipto><min>-1</min><max>1</max></clipto>
    </summer>
    <aerosurface_scale name="elevator">
      <input>fcs/pitch_sum</input>
      <range><min>-25</min><max>20</max></range>
      <gain>0.01745</gain>
      <output>fcs/elevator-pos-rad</output>
    </aerosurface_scale>
  </channel>

  <channel name="Roll">
    <summer name="roll_sum">
      <input>fcs/aileron-cmd-norm</input>
      <input>fcs/roll-trim-cmd-norm</input>
      <clipto><min>-1</min><max>1</max></clipto>
    </summer>
    <aerosurface_scale name="left_aileron">
      <input>fcs/roll_sum</input>
      <range><min>-20</min><max>15</max></range>
      <gain>0.01745</gain>
      <output>fcs/left-aileron-pos-rad</output>
    </aerosurface_scale>
    <aerosurface_scale name="right_aileron">
      <input>fcs/roll_sum</input>
      <range><min>-15</min><max>20</max></range>
      <gain>-0.01745</gain>
      <output>fcs/right-aileron-pos-rad</output>
    </aerosurface_scale>
  </channel>

  <channel name="Yaw">
    <summer name="yaw_sum">
      <input>fcs/rudder-cmd-norm</input>
      <input>fcs/yaw-trim-cmd-norm</input>
      <clipto><min>-1</min><max>1</max></clipto>
    </summer>
    <aerosurface_scale name="rudder">
      <input>fcs/yaw_sum</input>
      <range><min>-20</min><max>20</max></range>
      <gain>0.01745</gain>
      <output>fcs/rudder-pos-rad</output>
    </aerosurface_scale>
  </channel>

  <channel name="Flaps">
    <kinematic name="flaps">
      <input>fcs/flap-cmd-norm</input>
      <traverse>
        <setting><position> 0</position><time>0</time></setting>
        <setting><position>10</position><time>3</time></setting>
        <setting><position>20</position><time>2</time></setting>
        <setting><position>30</position><time>2</time></setting>
      </traverse>
      <output>fcs/flap-pos-deg</output>
    </kinematic>
  </channel>
</flight_control>

```

15.6 Аеродинаміка (репрезентативні значення з CFD)

```

<aerodynamics>
  <alphalimits unit="RAD">
    <min>-0.087</min><max>0.30</max>
  </alphalimits>
  <hysteresis_limits unit="RAD">
    <min>0.20</min><max>0.30</max>
  </hysteresis_limits>

  <!-- ===== LIFT ===== -->
  <axis name="LIFT">
    <function name="aero/coefficient/CLwbh">
      <description>Wing-body-tail lift vs alpha (clean)</description>
      <product>
        <property>aero/qbar-psf</property>
        <property>metrics/Sw-sqft</property>
        <table>
          <independentVar>aero/alpha-rad</independentVar>
          <tableData>
            -0.087  -0.250
            -0.044   0.100
            0.000   0.300
            0.087   0.760
            0.175   1.230
            0.262   1.500
            0.300   1.200    <!-- post-stall -->
          </tableData>
        </table>
      </product>
    </function>

    <function name="aero/coefficient/CLDf">
      <description>Lift increment due to flap</description>
      <product>
        <property>aero/qbar-psf</property>
        <property>metrics/Sw-sqft</property>
        <table>
          <independentVar>fcs/flap-pos-deg</independentVar>
          <tableData>
            0    0.00
            10   0.20
            20   0.35
            30   0.45
          </tableData>
        </table>
      </product>
    </function>

    <function name="aero/coefficient/CLde">
      <description>Lift due to elevator</description>
      <product>
        <property>aero/qbar-psf</property>
        <property>metrics/Sw-sqft</property>
        <property>fcs/elevator-pos-rad</property>
        <value>0.43</value>
      </product>
    </function>
  </axis>

  <!-- ===== DRAG ===== -->
  <axis name="DRAG">
    <function name="aero/coefficient/CD0">
      <product>
        <property>aero/qbar-psf</property>
        <property>metrics/Sw-sqft</property>
        <value>0.025</value>
      </product>
    </function>
    <function name="aero/coefficient/CDi">

```

```

    <description>Induced drag,  $CL^2 / (\pi * AR * e)$ </description>
    <product>
      <property>aero/qbar-psf</property>
      <property>metrics/Sw-sqft</property>
      <quotient>
        <property>aero/cl-squared</property>
        <value>20.4</value>      <!-- pi * AR * e ~ 20 -->
      </quotient>
    </product>
  </function>
</axis>

<!-- ===== PITCH ===== -->
<axis name="PITCH">
  <function name="aero/coefficient/Cm0">
    <product>
      <property>aero/qbar-psf</property>
      <property>metrics/Sw-sqft</property>
      <property>metrics/cbarw-ft</property>
      <value>-0.04</value>
    </product>
  </function>
  <function name="aero/coefficient/Cma">
    <description>Pitch moment slope (must be negative)</description>
    <product>
      <property>aero/qbar-psf</property>
      <property>metrics/Sw-sqft</property>
      <property>metrics/cbarw-ft</property>
      <property>aero/alpha-rad</property>
      <value>-0.5</value>
    </product>
  </function>
  <function name="aero/coefficient/Cmde">
    <product>
      <property>aero/qbar-psf</property>
      <property>metrics/Sw-sqft</property>
      <property>metrics/cbarw-ft</property>
      <property>fcs/elevator-pos-rad</property>
      <value>-1.10</value>
    </product>
  </function>
  <function name="aero/coefficient/Cmq">
    <description>Pitch damping</description>
    <product>
      <property>aero/qbar-psf</property>
      <property>metrics/Sw-sqft</property>
      <property>metrics/cbarw-ft</property>
      <property>aero/ci2vel</property>
      <property>velocities/q-aero-rad_sec</property>
      <value>-12.0</value>
    </product>
  </function>
</axis>

<!-- ===== SIDE ===== -->
<axis name="SIDE">
  <function name="aero/coefficient/CYb">
    <product>
      <property>aero/qbar-psf</property>
      <property>metrics/Sw-sqft</property>
      <property>aero/beta-rad</property>
      <value>-0.7</value>
    </product>
  </function>
</axis>

<!-- ===== ROLL ===== -->
<axis name="ROLL">
  <function name="aero/coefficient/Clb">
    <description>Dihedral effect (must be negative)</description>
    <product>

```



```

        <property>aero/qbar-psf</property>
        <property>metrics/Sw-sqft</property>
        <property>metrics/bw-ft</property>
        <property>aero/beta-rad</property>
        <value>-0.08</value>
    </product>
</function>
<function name="aero/coefficient/Clp">
    <description>Roll damping</description>
    <product>
        <property>aero/qbar-psf</property>
        <property>metrics/Sw-sqft</property>
        <property>metrics/bw-ft</property>
        <property>aero/bi2vel</property>
        <property>velocities/p-aero-rad_sec</property>
        <value>-0.46</value>
    </product>
</function>
<function name="aero/coefficient/Cllda">
    <product>
        <property>aero/qbar-psf</property>
        <property>metrics/Sw-sqft</property>
        <property>metrics/bw-ft</property>
        <property>fcs/left-aileron-pos-rad</property>
        <value>0.22</value>
    </product>
</function>
</axis>

<!-- ===== YAW ===== -->
<axis name="YAW">
    <function name="aero/coefficient/Cnb">
        <description>Weathercock stability (must be positive)</description>
        <product>
            <property>aero/qbar-psf</property>
            <property>metrics/Sw-sqft</property>
            <property>metrics/bw-ft</property>
            <property>aero/beta-rad</property>
            <value>0.06</value>
        </product>
    </function>
    <function name="aero/coefficient/Cnr">
        <description>Yaw damping</description>
        <product>
            <property>aero/qbar-psf</property>
            <property>metrics/Sw-sqft</property>
            <property>metrics/bw-ft</property>
            <property>aero/bi2vel</property>
            <property>velocities/r-aero-rad_sec</property>
            <value>-0.10</value>
        </product>
    </function>
    <function name="aero/coefficient/Cndr">
        <product>
            <property>aero/qbar-psf</property>
            <property>metrics/Sw-sqft</property>
            <property>metrics/bw-ft</property>
            <property>fcs/rudder-pos-rad</property>
            <value>-0.05</value>
        </product>
    </function>
</axis>
</aerodynamics>

</fdm_config>

```

Перевірки правдоподібності для цієї моделі:

- $C_{L\alpha} \approx 5.4/\text{rad}$ (нахил таблиці піднімальної сили поблизу $\alpha=0$). Типове значення для прямокутного крила при $AR \approx 7.3$ становить близько $4.7/\text{rad}$ — наше число достатньо

близьке.

- $C_{L_{\max}} \approx 1.5$. Прийнятно для чистого крила.
- $C_{D0} = 0.025$. Приблизно правильне значення для одномоторного літака з неприбиральним шасі.
- Статично стійкий: $C_{m\alpha} = -0.5 < 0$, $C_{n\beta} = +0.06 > 0$, $C_{l\beta} = -0.08 < 0$.
- Знак похідної за кермом висоти: $C_{m\delta_e} = -1.1$. Відхилення задньої кромки вниз (додатне δ_e) створює тангаж на пікірування — це відповідає нашій домовленості про знаки.

Розділ 16: Початкові умови, сценарії та скидання

16.1 Початкові умови (файли скидання)

Файл початкових умов (IC) задає кожну змінну стану до самоузгодженої вихідної точки для симуляції. Мінімальний набір — це положення (3), орієнтація (3), швидкість (3) — далі JSBSim обчислює α , β , $\dot{q}$, число Маха внутрішньо. Можна задати надлишково й дати JSBSim усе узгодити, але краще задавати явно.

```
<?xml version="1.0"?>
<initialize name="cruise_5000">
  <!-- Position -->
  <latitude unit="DEG"> 37.6 </latitude>
  <longitude unit="DEG">-122.4 </longitude>
  <altitude unit="FT"> 5000 </altitude>          <!-- above MSL -->

  <!-- Attitude (Euler 3-2-1) -->
  <phi unit="DEG"> 0 </phi>                      <!-- bank -->
  <theta unit="DEG"> 2 </theta>                  <!-- pitch -->
  <psi unit="DEG"> 90 </psi>                     <!-- heading -->

  <!-- Pick ONE velocity spec: -->
  <ubody unit="FT/SEC"> 150 </ubody>
  <vbody unit="FT/SEC"> 0 </vbody>
  <wbody unit="FT/SEC"> 5 </wbody>
  <!-- or: <vt unit="KTS">100</vt> with <alpha unit="DEG">2</alpha> -->
  <!-- or: <vc unit="KTS">95</vc> (calibrated airspeed) -->
  <!-- or: <mach>0.55</mach> -->
</initialize>
```

Інші корисні елементи IC:

```
<gamma unit="DEG"> 0 </gamma>          flight path angle
<vnorth>0</vnorth><veast>0</veast><vdown>0</vdown>  NED wind-rel velocity
<roc unit="FT/MIN"> 0 </roc>           rate of climb
<altitudeAGL unit="FT"> 0 </altitudeAGL> above ground level
<targetNlf> 1 </targetNlf>            target load factor for trim-pullup
```

16.2 Сценарії (подієво-орієнтовані випробування)

Сценарій (script) — це замкнений опис випробувального польотного сценарію: послідовність подій з умовами та діями над властивостями. Цикл Run крокує модель із заданим dt, доки не досягне end.

```
<?xml version="1.0"?>
<runscript name="Cruise climb">
  <description>5000 ft cruise -> trim -> climb to 8000</description>
  <use aircraft="AcmeA1" initialize="cruise_5000"/>

  <run start="0" end="300" dt="0.00833333">
    <event name="Trim">
      <condition>simulation/sim-time-sec ge 0</condition>
      <set name="simulation/do_simple_trim" value="1"/>
      <notify/>
    </event>

    <event name="Throttle up">
      <condition>simulation/sim-time-sec ge 30</condition>
      <set name="fcs/throttle-cmd-norm"
        action="FG_EXP" tc="5.0" value="1.0"/>
      <notify/>
    </event>

    <event name="Climb to 8000">
```

```

        <condition>simulation/sim-time-sec ge 60</condition>
        <set name="fcs/elevator-cmd-norm"
            action="FG_RAMP" tc="3.0" value="-0.1"/>
        <notify/>
    </event>

    <event name="Level off">
        <condition>position/h-sl-ft ge 8000</condition>
        <set name="fcs/elevator-cmd-norm" value="0"/>
        <notify>
            <property>position/h-sl-ft</property>
            <property>velocities/vc-kts</property>
        </notify>
    </event>
</run>
</runscript>

```

Типи дій для <set>:

FG_VALUE	set property to value immediately (default)
FG_DELTA	add value to current property
FG_RAMP	linear ramp over tc seconds
FG_EXP	first-order exponential approach with tc seconds

Умови: будь-який булів вираз над властивостями з використанням ge le gt lt eq nq and or not. Умови можна вкладати через <condition logic="AND|OR">.

Розділ 17: Процес валідації

Модель, яка компілюється й балансується, — це лише перший крок. Наведені нижче стандартні для галузі кроки валідації виявляють переважну більшість помилок моделювання ще до того, як вони потраплять до замовника.

17.1 Крок 1: діагностика балансування

Запустіть `simulation/do_simple_trim = 1` (поздовжнє) і перевірте, що отримані положення керма висоти та сектора газу лежать у межах реального діапазону керування. Якщо балансувальний α близький до верхньої межі за кутом атаки, то балансування не зірвалося лише завдяки везінню; зменшіть крейсерську швидкість або перевірте свою криву $C_L(\alpha)$.

17.2 Крок 2: розімкнені перехідні характеристики

Виконайте подвійні відхилення керма висоти, імпульси елеронів і поштовхи кермом напряду з балансувального стану. Побудуйте графіки p , q , r , α , β , ϕ , θ , ψ :

- **Фуґоїд**: коливання з періодом ~ 30 - 60 с, слабо демпфований ($\zeta \approx 0.05$). Проявляється як довгоперіодні коливання висоти й повітряної швидкості після збурення за тангажем.
- **Коротко-періодичний рух**: період ~ 2 - 5 с, добре демпфований ($\zeta \approx 0.5$). Проявляється як швидке коливання кутової швидкості тангажа після подвійного відхилення.
- **Крен-мода**: першого порядку, стала часу ~ 0.5 - 1.5 с. Проявляється як експоненційне згасання p після поштовху елеронами.
- **Голландський крок**: період ~ 3 - 6 с, $\zeta \approx 0.05$ - 0.3 . Зв'язане riskання-крен після поштовху кермом напряду.
- **Спіральна мода**: дуже повільна (стала часу ~ 30 - 300 с), часто трохи нестійка для звичайних літаків.

17.3 Крок 3: вилучення лінійної моделі

JSBSim постачає утиліту мовою Python (`python/JSBSim/utls/linearize.py`), яка будує лінійну модель A , B , C , D в балансувальній точці. Обчисліть власні значення матриці A і порівняйте полюси з аналітичними оцінками Etkin або Stevens-Lewis. Порядок величини має збігатися; якщо полюс опиняється не в тій півплощині, у вас помилка знака в аеродинамічних коефіцієнтах.

17.4 Крок 4: перевірка експлуатаційного діапазону

Величина	Як отримати в JSBSim	Порівнювати з
V_{S0} швидкість звалювання (чиста)	Збалансувати на мін. α , утримуючи висоту, зменшити сектор газу	РОН або проектна ціль
V_{S1} швидкість звалювання (закрилки)	Те саме, але з положенням закрилків = макс.	РОН
$V_{\max}$	Горизонтальний політ на повному газі, записати швидкість	РОН
Найбільша скоропідіймальність (V_y)	Набір висоти на різних швидкостях, знайти макс. ROC	Графік з РОН
Практична стеля	Балансування з кроком за висотою, знайти h за ROC = 100 fpm	РОН
Дальність	Крейсерський режим найбільшої економічності, інтегрувати витрату пального	РОН

17.5 Крок 5: перевірка пілотажних властивостей

Якщо вас цікавить відчуття від пілотування, оцініть модель за специфікаціями пілотажних властивостей MIL-F-8785C / MIL-HDBK-1797. Найцінніший окремий показник — це параметр C^* (коефіцієнт перевантаження, зважений за кутовою швидкістю тангажа), нанесений на граничну діаграму Cooper-Harper для крейсерського режиму. JSBSim видає всі необхідні властивості, щоб обчислити його офлайн.

17.6 Крок 6: порівняння з льотними випробуваннями

Там, де є дані льотних випробувань, виконайте той самий маневр у JSBSim з тими самими вхідними даними й накладіть графіки. Розбіжності, які ви побачите, — це рецепт для наступної ітерації CFD або підстроювання коефіцієнтів. Стережіться «гонитви за польотом»: підстроювання JSBSim під один маневр часто погіршує інші. Завжди перевіряйте модель на відкладеному наборі маневрів.

Розділ 18: Поширені помилки та способи їх діагностики

Це режими відмов, з якими найчастіше стикаються новачки, приблизно за порядком частоти. Стовець розв'язку вказує на те, що слід перевірити насамперед.

Симптом	Імовірна причина	Що перевірити першим
Літак «провалюється» крізь злітну смугу за IC	Координата Z шасі має неправильний знак або одиницю	<contact> Z має бути від'ємним, якщо шасі нижче за початок структурної системи
Літак догори дригом або обертається на 180° під час запуску	Розбіжність угод щодо ψ та курсу або невідповідність <code>negated_crossproduct_inertia</code>	Поміняйте атрибут знака інерції
Балансування одразу провалюється	Діапазон α в таблицях CFD не охоплює балансувальний α	Задайте <output> для α і запустіть нерозважливо — подивіться, який α він запитує
Балансування збігається, але літак опускає ніс і пікірує	Помилка знака $C_{m\alpha}$ (модель статично нестійка)	Побудуйте C_m від α ; нахил має бути від'ємним
Літак некеровано рискає вбік	Помилка знака $C_{n\beta}$	Нахил має бути додатним
Повільне, але стале кренення	Несиметрична точкова маса або елерон не відцентрований	Перевірте, що Y-координати точкових мас у сумі дають нуль
Тангаж коливається на 0.5-2 Гц	Недостатній C_{mq} або неправильний знак	Має бути від'ємним; величина 5-15 для авіації загального призначення
Відхилення елеронів не дає реакції за креном	Неправильний знак $C_{l\delta a}$ або властивість положення елерона не відповідає властивості аеродинамічного коефіцієнта	Простежте <code>fcs/left-aileron-pos-rad</code> аж до функції
Величезна сила опору на балансуванні	Забули відняти опорну силу опору, або коефіцієнти CFD у зв'язаній системі подано на осі LIFT/DRAG	Обчисліть $D = C_D \cdot \dot{q} \cdot S$ вручну; порівняйте з $F_{x,aero,body}$
Двигун не створює тяги	Місткість бака = 0 або неправильні індекси подачі	Перевірте <code>propulsion/tank[n]/contents-lbs</code>
Симуляція аварійно завершується з NaN	Ділення на нуль у функції (часто /Vt за нульової повітряної швидкості) або кватерніон став неединичним	Додайте <clipto> або захистіть за допомогою <ifthen>
Літак «занурюється» крізь землю після приземлення	Стала пружини занадто м'яка для цієї ваги	$k \geq W / 0.5 \text{ ft}$ для нормальних коефіцієнтів демпфування
Сильно стрибає під час посадки	Коефіцієнт демпфування занадто низький	Прагніть до $\zeta \approx 0.4-0.6$ ($c = 2\zeta\sqrt{(km)}$)
Вихід FCS застряг на clipto	Насичення; інтегратор «розкрутився»	Додайте тригер антинасичення (anti-windup) до ПІД-регулятора
FlightGear показує тремтіння літака	Розбіжність частоти виводу або розходження кроку часу FG/FDM	Задайте частоту виводу 60 Гц і FG теж на 60 Гц

Розділ 19: API мовою Python

JSBSim постачає модуль мовою Python (`jsbsim`) у PyPI та conda-forge, який обгортає бібліотеку C++. Це рекомендований спосіб керувати JSBSim із коду машинного навчання, пакетних параметричних досліджень або модульних тестів.

19.1 Швидкий старт

```
import jsbsim

fdm = jsbsim.FGFDMEExec(root_dir=None) # auto-detect aircraft path
fdm.set_debug_level(0)
fdm.load_model('c172p')
fdm.load_ic('reset00', useStoredPath=True)
fdm.run_ic() # apply initial conditions

# Trim longitudinally
fdm['simulation/do_simple_trim'] = 1

# Step the model 10 seconds at 120 Hz
for _ in range(1200):
    fdm.run()

print("alt:", fdm['position/h-sl-ft'])
print("V :", fdm['velocities/vt-fps'])
```

19.2 Читання та запис властивостей

```
# All properties accessible by string path
fdm['fcs/elevator-cmd-norm'] = -0.1 # nose-up command
fdm['fcs/throttle-cmd-norm'] = 0.8
alpha = fdm['aero/alpha-deg']
p, q, r = (fdm[f'velocities/{ax}-rad_sec'] for ax in 'pqr')

# Snapshot the full property tree (slow)
snapshot = fdm.query_property_catalog('')
```

19.3 Балансування та лінеаризація

```
from jsbsim.utils.linearize import linearize

fdm.load_model('c172p')
fdm.load_ic('cruise', useStoredPath=True)
fdm.run_ic()
fdm['simulation/do_simple_trim'] = 1

A, B, C, D, x_trim, u_trim = linearize(fdm)
import numpy as np
eigvals = np.linalg.eigvals(A)
print(sorted(eigvals, key=lambda e: e.real))
```

19.4 Пакетні параметричні дослідження

```
def sweep_alpha(cg_x_in):
    fdm = jsbsim.FGFDMEExec(None)
    fdm.load_model('AcmeA1')
    fdm.load_ic('cruise_5000', useStoredPath=True)
    fdm['inertia/pointmass-location-X-inches[0]'] = cg_x_in
    fdm.run_ic()
    fdm['simulation/do_simple_trim'] = 1
    return fdm['aero/alpha-deg'], fdm['fcs/elevator-pos-deg']

for cg in range(30, 50, 2):
    a, e = sweep_alpha(cg)
    print(f'CG={cg} in   alpha={a:.2f} elev={e:.2f}')
```


Розділ 20: Розширений довідник властивостей

Повне дерево властивостей публікує сам JSBSim за допомогою `fdm.query_property_catalog('')`. Тут згруповано найкорисніші властивості для зчитування показників та машинного навчання.

20.1 Час

<code>simulation/sim-time-sec</code>	simulation time since reset
<code>simulation/frame</code>	tick counter
<code>simulation/dt</code>	timestep size (s)
<code>simulation/integrator/rate/rotational</code>	
<code>simulation/integrator/rate/translational</code>	
<code>simulation/integrator/position/rotational</code>	
<code>simulation/integrator/position/translational</code>	

20.2 Атмосфера та вітри

<code>atmosphere/T-R, T-sl-R, delta-T</code>	
<code>atmosphere/P-psf, P-sl-psf</code>	
<code>atmosphere/rho-slugs_ft3</code>	
<code>atmosphere/a-fps</code>	speed of sound
<code>atmosphere/density-altitude</code>	
<code>atmosphere/pressure-altitude</code>	
<code>atmosphere/wind-{north,east,down}-fps</code>	steady wind in NED
<code>atmosphere/total-wind-{north,east,down}-fps</code>	wind+gust+turb
<code>atmosphere/turb-type</code>	turbulence model (0..4)
<code>atmosphere/turb-rate-rad_sec</code>	current angular turbulence

20.3 Положення

<code>position/lat-geod-deg, lat-gc-rad</code>	geodetic and geocentric lat
<code>position/long-gc-deg</code>	
<code>position/h-sl-ft, h-sl-meters</code>	altitude above MSL
<code>position/h-agl-ft</code>	altitude above ground
<code>position/geod-alt-ft</code>	
<code>position/distance-from-start-mag-mt</code>	great-circle distance
<code>position/terrain-elevation-asl-ft</code>	

20.4 Орієнтація

<code>attitude/phi-rad, theta-rad, psi-rad</code>	Euler
<code>attitude/phi-deg, theta-deg, psi-deg</code>	degrees
<code>attitude/heading-true-rad</code>	
<code>attitude/pitch-rad, roll-rad</code>	
<code>/sim/attitude/q[0..3]</code>	quaternion components (if exposed)

20.5 Швидкості

<code>velocities/vt-fps</code>	true airspeed
<code>velocities/vc-kts</code>	calibrated airspeed
<code>velocities/ve-kts</code>	equivalent airspeed
<code>velocities/vg-fps</code>	ground speed
<code>velocities/mach</code>	
<code>velocities/u-fps, v-fps, w-fps</code>	body-frame velocity
<code>velocities/u-aero-fps</code>	body-frame airmass-relative
<code>velocities/p-rad_sec, q-rad_sec, r-rad_sec</code>	body angular rates
<code>velocities/p-aero-rad_sec</code>	aero-frame angular rates
<code>velocities/vdown-fps</code>	NED down velocity (=climb rate)

20.6 Аеродинамічний стан

aero/alpha-rad, alpha-deg	
aero/beta-rad, beta-deg	
aero/alphadot-rad_sec, betadot-rad_sec	
aero/mag-alpha-rad, mag-beta-rad	alpha , beta
aero/qbar-psf, qbarUW-psf, qbarUV-psf	
aero/h_b-mac-ft	h/c for ground effect
aero/bi2vel, ci2vel	
aero/cl-squared	CL^2 from previous tick
aero/stall-hyst-norm	stall hysteresis switch (0 or 1)

20.7 Сили та моменти (зв'язана система)

forces/fbx-aero, fby-aero, fbz-aero	aerodynamic
forces/fbx-prop, fby-prop, fbz-prop	propulsion
forces/fbx-gear, fby-gear, fbz-gear	ground reactions
forces/fbx-external, fby-external, fbz-external	
forces/fbx-buoyant, fby-buoyant, fbz-buoyant	
forces/fbx-total, fby-total, fbz-total	sum (used by Accelerations)
moments/l-aero, m-aero, n-aero	
moments/l-prop, m-prop, n-prop	
moments/l-gear, m-gear, n-gear	
moments/l-total, m-total, n-total	

20.8 Прискорення

accelerations/udot-ft_sec2, vdot-ft_sec2, wdot-ft_sec2	
accelerations/pdot-rad_sec2, qdot-rad_sec2, rdot-rad_sec2	
accelerations/a-pilot-x-ft_sec2	at the EYEPOINT
accelerations/n-pilot-x-norm	load factor at EYEPOINT, g
accelerations/Nz	normal load factor

20.9 Команди та положення FCS

fcs/elevator-cmd-norm	-1..+1 from pilot/script
fcs/aileron-cmd-norm	
fcs/rudder-cmd-norm	
fcs/pitch-trim-cmd-norm	
fcs/roll-trim-cmd-norm	
fcs/yaw-trim-cmd-norm	
fcs/flap-cmd-norm	
fcs/throttle-cmd-norm[n]	
fcs/mixture-cmd-norm[n]	
fcs/elevator-pos-rad, fcs/elevator-pos-deg, fcs/elevator-pos-norm	
fcs/left-aileron-pos-rad, ...	
fcs/rudder-pos-rad, ...	
fcs/flap-pos-deg, fcs/flap-pos-norm	
fcs/speedbrake-pos-rad	
fcs/left-brake-cmd-norm, right-brake, center-brake	
fcs/steer-cmd-norm	

20.10 Силова установка

propulsion/engine[n]/thrust-lbs	
propulsion/engine[n]/fuel-flow-rate-pps	per-second
propulsion/engine[n]/fuel-flow-rate-gph	
propulsion/engine[n]/n1, n2	turbine spool speeds
propulsion/engine[n]/rpm	prop or piston rpm
propulsion/engine[n]/torque-lbsft	
propulsion/engine[n]/power-hp	piston
propulsion/engine[n]/map-inhg	manifold pressure
propulsion/engine[n]/egt-degf	exhaust temp
propulsion/engine[n]/cht-degf	cylinder head
propulsion/engine[n]/oil-pressure-psi	
propulsion/engine[n]/oil-temp-degf	
propulsion/tank[n]/contents-lbs, capacity-lbs	
propulsion/total-fuel-lbs	

20.11 Керування симуляцією

simulation/do_simple_trim	1 = longitudinal trim, 2 = ground, ...
simulation/do_trim	full trim mode
simulation/reset	reset to initial conditions
simulation/terminate	stop the script
simulation/pause	freeze the model
simulation/gravity-model	0 spherical, 1 WGS-84
simulation/gravitational-torque	spacecraft tidal torque on/off
simulation/notify-time-trigger	
simulation/randomseed	

Частина II

Розширена теорія та підґрунтя

Решта цього посібника переходить від практичних "настанов" Частини I до глибшого, підкріпленого посиланнями викладу математики, фізики, аеродинаміки, геодезії, силової установки та механізмів верифікації, які реалізує JSBSim. Читайте її послідовно, щоб побудувати повне уявлення, або користуйтеся нею як довідником, коли натрапите в Частині I на тему, яку хочете зрозуміти глибше.

Розділ 21: Математика для динаміки польоту

Кожен рядок JSBSim — це по суті дії над тривимірними векторами, матрицями обертання, кватерніонами та тензором, що пов'язує їх усі разом, — тензором інерції. Цей розділ є стислим довідником із цих об'єктів. Ми передбачаємо знання математичного аналізу та основ лінійної алгебри; усе, що понад це, побудовано з нуля.

21.1 Вектори у тривимірному просторі

Тривимірний вектор — це впорядкована трійка $\mathbf{v} = (v_x, v_y, v_z)$. Центральними є дві операції:

$$\text{Dot product: } \mathbf{a} \cdot \mathbf{b} = a_x b_x + a_y b_y + a_z b_z = |\mathbf{a}| |\mathbf{b}| \cos \theta$$

$$\text{Cross product: } (\mathbf{a} \times \mathbf{b})_i = \varepsilon_{ijk} a_j b_k; |\mathbf{a} \times \mathbf{b}| = |\mathbf{a}| |\mathbf{b}| \sin \theta$$

Скалярний добуток є скаляром; векторний добуток є вектором, перпендикулярним до обох операндів. Обидва реалізовано у вихідному коді JSBSim як перевантаження `FGColumnVector3`.

Мішаний добуток. Мішаний (скалярно-векторний) добуток $\mathbf{a} \cdot (\mathbf{b} \times \mathbf{c})$ дорівнює орієнтованому об'ємові паралелепіпеда, побудованого на трьох векторах. Подвійний векторний добуток задовольняє тотожність $\mathbf{a} \times (\mathbf{b} \times \mathbf{c}) = (\mathbf{a} \cdot \mathbf{c})\mathbf{b} - (\mathbf{a} \cdot \mathbf{b})\mathbf{c}$ — тотожність «BAC-CAB», що використовується в кінематиці твердого тіла.

Проекція. Складова вектора $\mathbf{a}$ вздовж одиничного вектора $\hat{n}$ дорівнює $\mathbf{a} \cdot \hat{n}$; векторна проекція дорівнює $(\mathbf{a} \cdot \hat{n})\hat{n}$. Це використовується для виділення складових сили опору вздовж напрямку відносного вітру, нормальних сил уздовж нормалі до еліпсоїда тощо.

21.2 Матриці обертання в SO(3)

Матриця обертання $\mathbf{R} \in \text{SO}(3)$ — це дійсна матриця 3×3 із $\mathbf{R}^T \mathbf{R} = \mathbf{I}$ (ортогональна) та $\det(\mathbf{R}) = +1$ (власна, без віддзеркалення). Звідси два наслідки:

- Обернення тривіальне: $\mathbf{R}^{-1} = \mathbf{R}^T$. JSBSim ніколи не обчислює чисельне обернення матриці обертання.
- Композиція — це множення матриць: обертання на $\mathbf{R}_1$ з наступним $\mathbf{R}_2$ дорівнює $\mathbf{R}_2 \mathbf{R}_1$. Композиція є некомутативною.

Три елементарні обертання навколо зв'язаних осей є будівельними блоками будь-якої аерокосмічної угоди про кути Ейлера:

$$R_x(\varphi) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos \varphi & \sin \varphi \\ 0 & -\sin \varphi & \cos \varphi \end{pmatrix} \quad R_y(\theta) = \begin{pmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{pmatrix} \quad R_z(\psi) = \begin{pmatrix} \cos \psi & -\sin \psi & 0 \\ \sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Зверніть увагу на угоду про знаки: ці матриці обертають вектори в активному сенсі (або, що еквівалентно, перетворюють системи координат у пасивному сенсі — JSBSim використовує пасивну угоду).

21.3 Аерокосмічна послідовність Ейлера 3-2-1

Кутове положення повітряного судна традиційно задається riskанням ψ , потім тангажем θ , потім креном φ , які застосовуються як власні обертання навколо Z, потім Y', потім X''. Об'єднана матриця, що перетворює вектор із місцевої системи NED у зв'язану систему, має вигляд

$$\mathbf{R}_n^b = \mathbf{R}_x(\varphi) \mathbf{R}_y(\theta) \mathbf{R}_z(\psi)$$

Перемножена поелементно, вона дає класичну матрицю напрямних косинусів із дев'яти елементів, наведену в кожному аерокосмічному підручнику (Stevens & Lewis, рівн. 1.4-7). Послідовність 3-2-1 має особливість при $\theta = \pm 90^\circ$, де ψ та ϕ уже не визначаються окремо, — це добре відомий ефект "складання рамок" (gimbal lock).

21.4 Тензор інерції

Для твердого тіла з неперервним розподілом маси $\rho(\mathbf{r})$ тензор інерції відносно точки дорівнює

$$\mathbf{I} = \iiint \rho(\mathbf{r}) (|\mathbf{r}|^2 \mathbf{1} - \mathbf{r} \otimes \mathbf{r}) dV$$

де $\mathbf{1}$ — одинична матриця 3×3 , а $\otimes$ — зовнішній добуток. Поелементно діагональні члени (моменти інерції)

$$I_{xx} = \int (y^2 + z^2) dm; \quad I_{yy} = \int (x^2 + z^2) dm; \quad I_{zz} = \int (x^2 + y^2) dm$$

та позадіагональні члени (відцентрові моменти інерції)

$$I_{xy} = \int xy dm; \quad I_{xz} = \int xz dm; \quad I_{yz} = \int yz dm$$

Теорема Гюйгенса-Штейнера (про паралельні осі). Якщо $\mathbf{I}_{cg}$ — тензор інерції відносно центра мас, то тензор інерції відносно точки, зміщеної на $\mathbf{d}$ від центра мас, дорівнює

$$\mathbf{I}_p = \mathbf{I}_{cg} + m (|\mathbf{d}|^2 \mathbf{1} - \mathbf{d} \otimes \mathbf{d})$$

FGMassBalance застосовує її на кожному кроці, щоб додати внесок інерції кожної точкової маси до тензора порожнього спорядженого судна.

Головні осі. Діагоналізація $\mathbf{I}$ (яка є дійсною симетричною, а отже, ортогонально діагоналізовною) дає три головні моменти інерції вздовж трьох головних осей. Для повітряних суден із симетрією відносно площини x - z $I_{xy} = I_{yz} = 0$ за побудовою; I_{xz} є ненульовим щоразу, коли верхня та нижня половини фюзеляжу масово незбалансовані (тобто завжди).

21.5 Перетворення тензора при обертанні

Вектори перетворюються як $\mathbf{v}' = \mathbf{R}\mathbf{v}$. Тензор другого порядку, як-от матриця інерції, перетворюється як

$$\mathbf{I}' = \mathbf{R} \mathbf{I} \mathbf{R}^T$$

Саме так інерція в конструктивній системі, наведена в XML, повертається у зв'язану систему під час завантаження, а інерція у зв'язаній системі повертається у швидкісну (стійкісну) чи вітрову систему, коли це потрібно для аналізу похідних стійкості.

Розділ 22: Кватерніони — теорія і практика

Кватерніони є основним робочим представленням обертань усередині FGPropagate. Читати цикл інтегрування JSBSim значно легше, якщо ви впевнено володієте відповідною алгеброю.

22.1 Означення: Н, алгебра кватерніонів

Кватерніони Гамільтона — це чотиривимірна дійсна алгебра, натягнута на 1, i, j, k з правилами множення

$$i^2 = j^2 = k^2 = ijk = -1, \quad ij = k, \quad jk = i, \quad ki = j$$

Кватерніон — це $q = q_0 + q_1i + q_2j + q_3k$, що часто записують як $(q_0, \mathbf{q})$, де q_0 — скалярна частина, а $\mathbf{q} = (q_1, q_2, q_3)$ — векторна частина.

22.2 Добуток Гамільтона

$$\mathbf{p} \cdot \mathbf{q} = (p_0q_0 - \mathbf{p} \cdot \mathbf{q}, \quad p_0\mathbf{q} + q_0\mathbf{p} + \mathbf{p} \times \mathbf{q})$$

Цей некомутативний добуток є серцем арифметики кватерніонів. Його реалізовано у FGQuaternion::operator*. **Застереження:** аерокосмічне програмне забезпечення поділене між угодою Гамільтона (яку використовують JSBSim, ROS, MATLAB Aerospace Toolbox) та угодою JPL (яку використовує космічне ПЗ JPL та NASA Goddard, із протилежним знаком члена з векторним добутком). Їх змішування міняє місцями ліво- та правобічні обертання — часте джерело помилок.

22.3 Одиничні кватерніони та обертання

Одиничний кватерніон ($|q| = 1$) параметризує обертання. Відповідність «вісь-кут» має вигляд

$$q = (\cos(\theta/2), \hat{n} \sin(\theta/2))$$

для обертання на кут θ навколо одиничної осі $\hat{n}$. Вектор $\mathbf{v}$ обертається як уявна частина виразу

$$\mathbf{v}' = q \mathbf{v} q^*$$

де $\mathbf{v}$ — чистий кватерніон $(0, \mathbf{v})$, а $q^* = (q_0, -\mathbf{q})$ — спряжений кватерніон. Еквівалентно, матриця напрямних косинусів (DCM), що відповідає q , дорівнює

$$R(q) = \begin{vmatrix} q_0^2 + q_1^2 - q_2^2 - q_3^2 & 2(q_1q_2 - q_0q_3) & 2(q_1q_3 + q_0q_2) \\ 2(q_1q_2 + q_0q_3) & q_0^2 - q_1^2 + q_2^2 - q_3^2 & 2(q_2q_3 - q_0q_1) \\ 2(q_1q_3 - q_0q_2) & 2(q_2q_3 + q_0q_1) & q_0^2 - q_1^2 - q_2^2 + q_3^2 \end{vmatrix}$$

Усередині метод FGQuaternion::GetTransformationMatrix() кешує цю DCM із дев'яти елементів, тож подальші перетворення систем зводяться до множення матриці на вектор.

22.4 Кінематичне рівняння: $\dot{q} = \frac{1}{2} q \otimes \omega$

Якщо кутова швидкість тіла (у зв'язаній системі) дорівнює ω , то кінематичне звичайне диференціальне рівняння для обертання з інерціальної системи у зв'язану має вигляд

$$\dot{q} = \frac{1}{2} q \otimes (0, \omega_{\text{body}})$$

Геометрично: у кожен мить кватерніон рухається перпендикулярно до самого себе в чотиривимірному просторі, тож $|q|$ є інваріантом при точному інтегруванні. Чисельні схеми втрачають цей інваріант і потребують або періодичної перенормалізації, або інтегратора, що зберігає структуру.

Інтегратори Бусса (Buss) реалізують дискретні відображення, що зберігають структуру. Для сталого ω на проміжку $[t, t+\Delta t]$ вираз

$$q(t+\Delta t) = q(t) \cdot \exp(\frac{1}{2} \Delta t \omega) = q(t) \cdot (\cos(|\omega|\Delta t/2), \hat{\omega} \sin(|\omega|\Delta t/2))$$

є точним і зберігає одиничну норму з точністю до похибки чисел із рухомою комою. Buss-2 доповнює його коригувальним членом від $\dot{\omega}$, забезпечуючи другий порядок точності для несталого ω .

22.5 Кватерніони проти кутів Ейлера проти DCM

Представлення	Параметри	Особливість	Вартість	Коли
Ейлер 3-2-1	3 (ϕ, θ, ψ)	$\theta = \pm 90^\circ$ (gimbal lock)	Дешеве для відображення	Індикація в кабіні, вивід
Кватерніон	4 ($q_0 \dots q_3$)	Немає	Компактний, швидкий	Інтегрування, зберігання
DCM	9 (ортогональність марнує б)	Немає	Дрейф; потрібна перенормалізація	Перетворення систем
Вектор обертання	3 ($\theta \hat{n}$)	Особлива при $\theta=2\pi$	Мінімальна	Лінеаризовані збурення

Архітектура JSBSim несе кватерніон як основний стан кутового положення (без особливості, чотири ступені вільності), кешує похідну від нього DCM (дешеві повторні перетворення систем) і надає вивід у кутах Ейлера лише на межі дерева властивостей.

Розділ 23: Чисельне інтегрування — теорія

Властивості `simulation/integrator/rate/rotational`, `position/rotational`, `rate/translational` та `position/translational` незалежно обирають чотири схеми. Знання базової чисельної теорії — це різниця між стійкою моделлю за $dt = 1/120$ с і моделлю, що розходиться, коли ви змінюєте dt .

23.1 Однокрокові методи

Для задачі Коші $\dot{y} = f(t, y)$, $y(t_0) = y_0$:

$$\text{Forward Euler: } y_{n+1} = y_n + h f(t_n, y_n)$$

Локальна похибка апроксимації $O(h^2)$, глобальна похибка $O(h)$. Умовно стійкий: для тестового рівняння $\dot{y} = \lambda y$ потрібно $|1 + h\lambda| < 1$, що для дійсного від'ємного λ означає $h < 2/|\lambda|$.

$$\text{Backward Euler: } y_{n+1} = y_n + h f(t_{n+1}, y_{n+1})$$

Неявний, безумовно А-стійкий. Потребує розв'язання нелінійного рівняння на кожному кроці. Рідко використовується в моделюванні повітряних суден у реальному часі через обчислювальну вартість.

$$\text{Trapezoidal (Heun, AM2): } y_{n+1} = y_n + (h/2)[f(t_n, y_n) + f(t_{n+1}, y_{n+1})]$$

Неявний метод другого порядку; звичайною явною версією є форма «прогноз-корекція». Метод `eTrapezoidal` у JSBSim використовує «прогноз-корекцію» з попередньою похідною як прогнозом — фактично трапецієподібний на попередній похідній.

$$\text{RK4: } k_1 = f(t_n, y_n), \quad k_2 = f(t_n + h/2, y_n + h k_1/2), \quad \dots \quad y_{n+1} = y_n + h(k_1 + 2k_2 + 2k_3 + k_4)/6$$

Класичний метод Runge-Kutta 4 є золотим стандартом для роботи симуляторів, де віддають перевагу однокроковим методам. Має четвертий порядок точності, але потребує чотирьох обчислень функції на крок — це не те, що JSBSim використовує за замовчуванням.

23.2 Багатокрокові методи: Adams-Bashforth

JSBSim реалізує явний метод Adams-Bashforth до 5-го порядку включно. Вони використовують одне обчислення функції на крок, але потребують початкових значень (які заповнюються методами нижчого порядку або повторними кроками Ейлера).

$$\begin{aligned} \text{AB2: } y_{n+1} &= y_n + h \left(\frac{3}{2} f_n - \frac{1}{2} f_{n-1} \right) \\ \text{AB3: } y_{n+1} &= y_n + h \left(\frac{23}{12} f_n - \frac{16}{12} f_{n-1} + \frac{5}{12} f_{n-2} \right) \\ \text{AB4: } y_{n+1} &= y_n + h \left(\frac{55}{24} f_n - \frac{59}{24} f_{n-1} + \frac{37}{24} f_{n-2} - \frac{9}{24} f_{n-3} \right) \\ \text{AB5: } y_{n+1} &= y_n + h \left(\frac{1901}{720} f_n - \frac{2774}{720} f_{n-1} + \frac{2616}{720} f_{n-2} - \frac{1274}{720} f_{n-3} + \frac{251}{720} f_{n-4} \right) \end{aligned}$$

Стандартні налаштування JSBSim — AB2 для поступальної швидкості, AB3 для поступального положення та прямокутний метод Ейлера для кватерніона (з подальшою перенормалізацією). Для плавної траєкторії повітряного судна це забезпечує збереження енергії та моменту імпульсу з точністю до $\sim 10^{-6}$ упродовж прогонів тривалістю в тисячі секунд за $dt = 1/120$ с.

23.3 Області стійкості та вибір кроку

Кожен інтегратор має область абсолютної стійкості в комплексній площині λh . Для рухів повітряного судна із власними значеннями λ порядку від -2 до $+0,05$ рад/с (короткоперіодичний рух, фугоїд, голландський крок) умови $h < 2/|\lambda_{\max}| \approx 0,6$ с достатньо для стійкості. Однак аеродинамічне збурення вносить швидкі рухи (смуга пропускання фільтрів запізнення СКУ, власні частоти стійок шасі 20-50 Гц), що на практиці підвищує вимогу до $h \lesssim 1/120$ с.

Жорсткість. Коли система має власні значення, що охоплюють багато порядків величини (повільні рухи польоту плюс швидкі рухи шасі/приводів), явні методи стають неефективними — вони змушені використовувати найменшу сталу часу. Неявні методи допомогли б; натомість JSBSim розв'язує жорсткі частини окремо (LCP для шасі, попередні фільтри для приводів), щоб явне інтегрування повільних рухів польоту залишалось стійким.

Розділ 24: Динаміка твердого тіла Ньютона-Ейлера

Рівняння руху повітряного судна є окремим випадком механіки твердого тіла. Цей розділ виводить їх з нуля та показує, як JSBSim реалізує кожен член.

24.1 Інерціальна форма (закони Ньютона)

$$m \mathbf{a}_i = \mathbf{F}, \quad d\mathbf{H}/dt = \mathbf{M}$$

Кількість руху $\mathbf{p} = m\mathbf{v}_i$ підпорядковується $\dot{\mathbf{p}} = \mathbf{F}$. Момент імпульсу відносно центра мас дорівнює $\mathbf{H} = \mathbf{I}\boldsymbol{\omega}$; його похідна за часом в інерціальній системі дорівнює прикладеному моменту.

24.2 Форма у зв'язаній системі через теорему про перенесення

Якщо вектор $\mathbf{q}$ виражено в системі, що обертається з кутовою швидкістю $\boldsymbol{\omega}$, то зв'язок між його похідними в інерціальній та відносно системи координат має вигляд

$$(d\mathbf{q}/dt)_{\text{inertial}} = (d\mathbf{q}/dt)_{\text{body}} + \boldsymbol{\omega} \times \mathbf{q}$$

Застосуємо до $\mathbf{v}_{\text{body}}$ та до $\mathbf{H}$:

$$m(\dot{\mathbf{v}}_{\text{body}} + \boldsymbol{\omega} \times \mathbf{v}_{\text{body}}) = \mathbf{F}_{\text{body}}$$

$$\mathbf{I}\dot{\boldsymbol{\omega}} + \boldsymbol{\omega} \times (\mathbf{I}\boldsymbol{\omega}) = \mathbf{M}_{\text{body}}$$

Це `FGAccelerations::CalculateUVWdot()` та `CalculatePQRdot()`, кожен по 12 рядків арифметики. Член перехресного зв'язку $\boldsymbol{\omega} \times (\mathbf{I}\boldsymbol{\omega})$ є джерелом гіроскопічних ефектів, зокрема знаменитої теореми про тенісну ракетку.

24.3 Урахування обертання планети

Зв'язана система на Землі, що обертається, не є строго інерціальною. Записавши інерціальне прискорення через внески у зв'язаній та дотичній до ECEF системах і застосувавши теорему про перенесення двічі, отримуємо

$$\dot{\mathbf{v}}_{\text{body}} + (\boldsymbol{\omega}_{b/i}) \times \mathbf{v}_{\text{body}} = \mathbf{F}/m - \mathbf{T}_{i \rightarrow b}(\boldsymbol{\Omega}_p \times \mathbf{r}_i) \dots$$

Повна інерціальна форма, що включає коріолісові та відцентрові члени, інтегрується в інерціальній системі; `FGPropagate` зберігає `vInertialPosition` та `vInertialVelocity` і перетворює їх у зв'язану систему для виводу.

24.4 Спрощення, специфічні для повітряних суден

- Симетрія відносно площини x-z: $I_{xy} = I_{yz} = 0$.
- Зв'язані осі координат збігаються з головними осями лише наближено; $I_{xz} \neq 0$, бо верхня та нижня половини фюзеляжу масово асиметричні.
- Усталений політ: $\dot{\mathbf{v}}_{\text{body}} = 0$, $\dot{\boldsymbol{\omega}} = 0$; $\mathbf{F}$ та $\mathbf{M}$ зводяться до нуля (умова балансування).
- Лінеаризація навколо точки балансування дає поздовжню (u , w , q , θ) та бічно-курсону (v , p , r , ϕ , ψ) розв'язані у просторі станів системи, які використовуються для аналізу стійкості.

Розділ 25: Основи механіки рідин і газів

Усі аеродинамічні коефіцієнти, які споживає JSBSim, походять із фізики, що зрештою ґрунтується на рівняннях Нав'є-Стокса. Цей розділ є містком від рівнянь суцільного середовища до інженерних величин.

25.1 Закони збереження

$$\text{Continuity: } \partial \rho / \partial t + \nabla \cdot (\rho \mathbf{V}) = 0$$

$$\text{Momentum: } \rho (\partial \mathbf{V} / \partial t + \mathbf{V} \cdot \nabla \mathbf{V}) = -\nabla p + \nabla \cdot \boldsymbol{\tau} + \rho \mathbf{g}$$

$$\text{Energy: } \rho (\partial e / \partial t + \mathbf{V} \cdot \nabla e) = -p \nabla \cdot \mathbf{V} + \Phi + \nabla \cdot (k \nabla T) + Q$$

із ньютонівським в'язким напруженням $\tau_{ij} = \mu (\partial u_i / \partial x_j + \partial u_j / \partial x_i) - (2/3) \mu \delta_{ij} \nabla \cdot \mathbf{V}$. Система замикається рівнянням стану $p = \rho R T$ (калорично досконалий газ).

25.2 Безрозмірні числа

Число	Означення	Фізичний зміст
Рейнольдса Re	$\rho V L / \mu = V L / \nu$	Інерційні / в'язкі сили; визначає пограничний шар
Маха M	$V/a, a = \sqrt{\gamma R T}$	Стисливість; $M < 0,3$ нестислива течія
Кнудсена Kn	λ / L	Застосовність суцільного середовища; $Kn < 0,01$ прийнятно
Прандтля Pr	$\mu c_p / k$	Товщина імпульсного проти теплового пограничного шару
Струхаля St	$f L / V$	Відношення нестационарного до конвективного масштабу часу
Фруда Fr	$V / \sqrt{g L}$	Інерційні / гравітаційні сили (вільна поверхня)

Типові аерокосмічні діапазони: Re для БПЛА на малій висоті становить 10^5 - 10^6 , для літака авіації загального призначення 10^6 - 10^7 , для транспортного 10^7 - 10^8 . Число Маха коливається від 0,05 (малий БПЛА) до 2,5+ (винищувач); навколосвуковий діапазон (0,8-1,2) є найскладнішим чисельно.

25.3 Пограничні шари

Prandtl (1904) помітив, що за великих Re в'язкість має значення лише в тонкому шарі товщиною δ біля поверхні. Рівняння пограничного шару є спрощеною формою рівнянь Нав'є-Стокса:

$$\partial u / \partial x + \partial v / \partial y = 0, \quad u \partial u / \partial x + v \partial u / \partial y = -(1/\rho) dp/dx + \nu \partial^2 u / \partial y^2, \quad \partial p / \partial y \approx 0$$

Тиск задається ззовні із зовнішньої незв'язкої течії. Розв'язок Блазіуса (Blasius) для плоскої пластини за нульового градієнта тиску дає закон ламінарного зростання $\delta \approx 5.0 x / \sqrt{Re_x}$ та місцеве поверхневе тертя $C_f \approx 0.664 / \sqrt{Re_x}$.

Турбулентний перехід на гладкій плоскій пластині за нульового градієнта настає поблизу $Re_x \approx 5 \times 10^5$, але вкрай чутливий до турбулентності набігаючого потоку, шорсткості та градієнта тиску. Аеродинамічні профілі масштабу БПЛА за $Re < 5 \times 10^5$ часто мають бульбашки ламінарного відриву, що домінують у полярі.

Турбулентний профіль: $u^+ = (1/\kappa) \ln y^+ + B$ у логарифмічному шарі, де $\kappa \approx 0,41$ та $B \approx 5,0$. RANS-розрахунок CFD має розв'язувати $y^+ < 1$ у першій комірці для аналізу з роздільною

здатністю біля стінки.

25.4 Відрив, зрив і закритичні режими

Потік відривається, коли дотичне напруження на стінці зникає під дією несприятливого градієнта тиску. На аеродинамічному профілі це відбувається при куті зриву α_{stall} , за яким піднімальна сила падає, а сила опору зростає. Три механізми зриву (залежно від товщини профілю):

- **Зрив із задньої кромки** (товсті профілі $>15\%$ t/c): відрив повзе вгору за потоком від задньої кромки; плавний злам піднімальної сили.
- **Зрив із передньої кромки** (середні 9-15% t/c): коротка бульбашка ламінарного відриву лопає; різкий злам піднімальної сили.
- **Зрив тонкого профілю** ($<8\%$ t/c): довга бульбашка росте та знову прилягає далі за потоком; крива піднімальної сили сплющується перед зломом.

Елемент `<hysteresis_limits>` у `<aerodynamics>` у JSBSim явно реалізує гістерезис повторного прилягання потоку після зриву.

Розділ 26: Теорія виникнення піднімальної сили

26.1 Бернуллі — не відповідь (а рівний час проходження хибний)

Рівняння Бернуллі вздовж лінії течії нестисливого незв'язкого усталеного потоку має вигляд

$$p + \frac{1}{2}\rho V^2 + \rho g z = \text{const}$$

Воно є **наслідком** збереження кількості руху, а не причиною піднімальної сили. Популярне пояснення про "рівний час проходження" — нібито повітря над верхньою поверхнею має пройти її за той самий час, що й повітря знизу, — емпірично хибне; дослід з димовими лініями показують, що повітря над верхньою поверхнею досягає задньої кромки значно раніше. Справжньою причиною асиметрії тиску є циркуляція, встановлена умовою Кутта.

26.2 Кутта-Жуковський: правильна відповідь

$$L' = \rho_{\infty} V_{\infty} \Gamma$$

Для двовимірної незв'язкої нестисливої течії навколо будь-якого замкненого тіла піднімальна сила на одиницю розмаху дорівнює добутку густини, швидкості набігаючого потоку та циркуляції $\Gamma = \oint \mathbf{V} \cdot d\mathbf{s}$. Сила опору в цій моделі дорівнює нулю (парадокс Д'Аламбера); в'язкість повертає опір і однозначно визначає Γ через умову Кутта (плавний схід потоку з гострої задньої кромки).

26.3 Теорія тонкого профілю

Для тонкого профілю похила кривої піднімальної сили дає відомий результат

$$dC_L/d\alpha = 2\pi \text{ (per radian)} \approx 0.110 / \text{deg}$$

а аеродинамічний фокус (точка нульового моменту тангажа за α) розташований на чверті хорди. Кривина зміщує кут нульової піднімальної сили $\alpha_{L=0}$, але не змінює похилу.

26.4 Крила скінченного розмаху: несна лінія Прандтля

Крило скінченного розмаху скидає за собою завихреність, що наводить скіс потоку, нахилиючи місцевий вектор піднімальної сили назад (індуктивний опір) і зменшуючи ефективний кут атаки. Теорія несної лінії Прандтля замінює крило приєднаним вихором з інтенсивністю $\Gamma(y)$ та пеленою вільних вихорів. Класичні результати:

$$C_L = \pi \cdot AR \cdot A_1, \quad C_{D,i} = C_L^2 / (\pi \cdot AR \cdot e)$$

з коефіцієнтом ефективності розмаху $e \leq 1$ ($e = 1$ для еліптичного розподілу навантаження — розподілу з мінімальним індуктивним опором). Сучасні повітряні судна використовують вінглети та оптимізацію розподілу навантаження за розмахом, щоб наблизитися до $e \approx 0,9-0,95$.

Похила кривої піднімальної сили крила скінченного розмаху:

$$dC_L/d\alpha = a_0 / (1 + a_0/(\pi \cdot AR \cdot e))$$

де $a_0 \approx 2\pi$ — похила профілю у двовимірному перерізі. Для $AR=8$, $e=0,85$ це дає $dC_L/d\alpha \approx 4,9/\text{рад}$ — приблизно на 20% менше за значення 2π , яке мало б крило нескінченного

розмаху.

26.5 Поправка на стисливість

$$C_{L,M} = C_{L,inc} / \sqrt{1 - M_{\infty}^2} \text{ (Prandtl-Glauert)}$$

Дійсна до $M \approx 0,7$. За цим значенням домінують утворення стрибка ущільнення та місцева навіколозвукова задача; M_{crit} (де місцеве число Маха вперше досягає 1) для транспортних профілів зазвичай настає при $M_{\infty} \approx 0,7-0,8$.

Розділ 27: Сила опору — повний розклад

$$C_D = C_{D,f} + C_{D,p} + C_{D,i} + C_{D,int} + C_{D,w}$$

П'ять адитивних джерел на дозвукових швидкостях. Кожне задається окремою XML-функцією в JSBSim і кожне походить від іншого фізичного механізму.

27.1 Поверхневе тертя (шкідливий опір)

Турбулентне поверхневе тертя на плоскій пластині (ESDU/Schlichting):

$$C_f \approx 0.455 / (\log_{10} Re_L)^{2.58}$$

Модифікується коефіцієнтом форми FF для кривини та коефіцієнтом інтерференції Q для з'єднань компонентів. У сумі по змочуваній площі $D_f = q \sum C_{f,i} FF_i Q_i S_{wet,i}$.

27.2 Опір форми (тиску)

Результат потовщення пограничного шару та помірного відриву, що перешкоджає повному відновленню тиску в задній частині тіла до значення в точці гальмування. Разом із поверхневим тертям це становить профільний опір. $C_{D,f} + C_{D,p}$ об'єднуються в C_{D0} для поляри.

27.3 Індуктивний опір (зумовлений піднімальною силою)

$$C_{D,i} = C_L^2 / (\pi \cdot AR \cdot e)$$

Зростає квадратично з C_L . Коефіцієнт ефективності розмаху e становить від 0,7 до 0,95 для звичайних крил; коефіцієнт ефективності Освальда e_0 (який поглинає інші ефекти, пропорційні C_L^2) використовується в стандартній параболічній полярі.

27.4 Опір інтерференції

З'єднання між компонентами (крило-фюзеляж, пілон-крило) створюють додатковий вихровий та зумовлений тиском опір. ESDU та Hoerner наводять емпіричні коефіцієнти Q, зазвичай 1,0-1,3.

27.5 Хвильовий опір

Понад критичним числом Маха M_{crit} потік місцево прискорюється до $M=1$ і утворюється стрибок ущільнення. Взаємодія стрибка з пограничним шаром дає хвильовий опір, що швидко зростає після числа Маха дивергенції опору M_{DD} :

$$\Delta C_{D,wave} \approx 20(M - M_{DD})^4$$

(правило четвертого степеня Лока). Надзвуковий мінімальний хвильовий опір досягається тілом обертання Сірса-Хаака (Sears-Haack) (правило площ, Whitcomb).

27.6 Параболічна поляра опору та максимальна аеродинамічна якість L/D

$$C_D = C_{D0} + k C_L^2, \quad k = 1/(\pi \cdot AR \cdot e)$$

Максимальна якість L/D досягається при $C_L^* = \sqrt{C_{D0}/k}$, $C_D^* = 2 C_{D0}$:

$$(L/D)_{\max} = 1 / (2 \sqrt{k \cdot C_{D0}})$$

Типові значення: планер 40-60, реактивний транспортний літак 17-22, малий БПЛА 8-14, Wright Flyer 1903 $\approx 8,3$ (Anderson).

Розділ 28: Стійкість і керованість — повна таблиця похідних

28.1 Угоди про знаки та означення

- **Поздовжня статична стійкість:** $dC_m/d\alpha < 0$. Еквівалентно розташуванню центра мас попереду нейтральної точки.
- **Курсова (флюгерна) стійкість:** $dC_n/d\beta > 0$. Праве ковзання дає момент на ніс праворуч, повертаючи ніс до вітру.
- **Бічна стійкість (ефект поперечного V):** $dC_l/d\beta < 0$. Праве ковзання дає лівий момент крену, піднімаючи навітряне крило.

28.2 Повна таблиця похідних

Сила/Момент	$\alpha / \dot{\alpha}$	Кутова швидкість	Керування
C_L	$C_{L\alpha}, C_{L\dot{\alpha}}$	C_{Lq}	$C_{L\delta e}, C_{L\delta f}$
C_D	$C_{D\alpha} (\approx 2k \cdot C_L \cdot C_{L\alpha})$	—	$C_{D\delta e}, C_{D\delta f}, C_{Dgear}$
C_Y	$C_{Y\beta}$	C_{Yp}, C_{Yr}	$C_{Y\delta a}, C_{Y\delta r}$
C_l (крен)	$C_{l\beta}$	C_{lp}, C_{lr}	$C_{l\delta a}, C_{l\delta r}$
C_m (тангаж)	$C_{m\alpha}, C_{m\dot{\alpha}}$	C_{mq}	$C_{m\delta e}$
C_n (рискання)	$C_{n\beta}$	C_{np}, C_{nr}	$C_{n\delta r}, C_{n\delta a}$

Похідні демпфування зведено до безрозмірного вигляду через $b/(2V)$ (бічні) або $\bar{c}/(2V)$ (поздовжні). Похідна за $\dot{\alpha}$ враховує запізнення скосу потоку між крилом і хвостовим оперенням. $C_{n\delta a}$ — це "зворотне рискання": позитивна команда на елерони викликає рискання у бік, протилежний до наміченого напрямку повороту, через диференціальний опір на відхилених елеронах.

28.3 Нейтральна точка та запас статичної стійкості

$$x_{NP} = x_{AC,wing} - (\bar{q} \cdot S_h / \bar{q} \cdot S) \cdot (C_{L\alpha,h} / C_{L\alpha}) \cdot l_h / \bar{c}$$

Запас статичної стійкості: $SM = (x_{NP} - x_{CG}) / \bar{c}$. У цивільних транспортних літаків $SM \approx 5-15\%$; винищувачі можуть мати від'ємний SM з активним законом керування (ослаблена статична стійкість).

28.4 Маневрена точка

Положення центра мас, за якого $dC_m/dn_z = 0$ в усталеному виході з пікірування. Розташована позаду x_{NP} приблизно на $\rho S \bar{c} C_{mq} / (4 m)$, що зміщує x_{MP} на кілька відсотків $\bar{c}$ позаду x_{NP} для типових транспортних літаків.

Розділ 29: Власні рухи повітряного судна та лінеаризація

29.1 Лінеаризація навколо балансування

Збурення Δu , Δw , Δq , $\Delta \theta$ навколо балансування з U_0 , Θ_0 :

$$\begin{aligned} m \Delta \dot{u} &= X_u \Delta u + X_w \Delta w - m g \cos \theta_0 \Delta \theta + \Delta X_{ctrl} \\ m \Delta \dot{w} &= Z_u \Delta u + Z_w \Delta w + (m U_0 + Z_q) \Delta q - m g \sin \theta_0 \Delta \theta + \Delta Z_{ctrl} \\ I_y \Delta \dot{q} &= M_u \Delta u + M_w \Delta w + M_{\dot{w}} \Delta \dot{w} + M_q \Delta q + \Delta M_{ctrl} \\ \Delta \dot{\theta} &= \Delta q \end{aligned}$$

Бічно-курсва підсистема — це $[\Delta v, \Delta p, \Delta r, \Delta \phi]$, керована $Y_v, Y_p, Y_r, L_v, L_p, L_r, N_v, N_p, N_r$. У першому наближенні за симетричного балансування ці дві підсистеми розв'язуються.

29.2 Форма у просторі станів

$$\dot{x} = Ax + Bu, \quad y = Cx + Du$$

Власні значення A дають рухи; власні вектори дають форму руху (які стани беруть участь у кожному русі). Властивість `simulation/do_linearization` у JSBSim виділяє A, B, C, D у точці балансування та записує їх у вивід `log4cpp`. Модуль Python надає те саме через `jsbsim.utils.linearize`.

29.3 Поздовжні рухи

Короткоперіодичний рух — швидке коливання тангажа, переважно рух Δw та Δq , слабко залежний від Δu . Сильно задемпфований ($\zeta \approx 0,3-0,7$), ω_n зазвичай 1-5 рад/с для транспортних літаків, 5-15 рад/с для винищувачів.

$$\omega_{n,sp}^2 \approx Z_{\alpha} M_q / V_0 - M_{\alpha}$$

$$2\zeta_{sp} \omega_{n,sp} \approx -(M_q + M_{\dot{\alpha}} + Z_{\alpha} / V_0)$$

Фуґоїд — повільний обмін кінетичної та потенціальної енергії: Δu та $\Delta \theta$ коливаються, Δw та Δq майже нульові. Слабко задемпфований ($\zeta \approx 0,05$) та повільний.

$$\omega_{n,ph} \approx \sqrt{2 \cdot g / V_0} \quad (\text{Lanchester})$$

$$\zeta_{ph} \approx (1/\sqrt{2}) \cdot (C_D/C_L)$$

Для пасажирського літака за $V_0 = 250$ м/с $\omega_{ph} \approx 0,055$ рад/с — період ≈ 115 с. $L/D = 17$ дає $\zeta_{ph} \approx 0,041$.

29.4 Бічно-курсві рухи

Рух крену — першого порядку, сильно задемпфоване експоненційне згасання кутової швидкості крену:

$$\tau_{roll} \approx -I_x / (\bar{q} \cdot S \cdot b \cdot C_{lp} \cdot b / (2V))$$

Типове $\tau_{roll} = 0,3-1,5$ с. Сприймається пілотом як "реакція за кутовою швидкістю".

Спіральний рух — першого порядку, дуже повільний, часто трохи нестійкий. Власне значення близьке до нуля. Визначається відношенням ефекту поперечного V до флюгерної стійкості; стійкість вимагає

$$C_{lp} \cdot C_{nr} > C_{nr} \cdot C_{lr}$$

Голландський крок — зв'язане коливання рискання-крену; повітряне судно "виляє хвостом", водночас погойдуючи крилами зі зсувом фази 90° . Помірно задемпфований ($\zeta \approx 0,05-0,3$), $\omega_n \approx 0,5-3$ рад/с для транспортних літаків.

$$\omega_{n,DR}^2 \approx (\bar{q} \cdot S \cdot b / I_z) C_{n\beta}$$

$$2 \zeta_{DR} \omega_{n,DR} \approx -(\bar{q} \cdot S \cdot b^2 / (2V \cdot I_z)) C_{nr}$$

Голландський крок потребує демпфера рискання на транспортних повітряних суднах; власний рух зазвичай задемпфований надто слабко для комфорту пілота. СКУ в JSBSim керує цим за допомогою демпфера рискання з програмованим коефіцієнтом підсилення.

Розділ 30: Аеродинаміка профілю

30.1 Сімейство профілів NASA

4-значні (NASA 2412): максимальна кривина 2% хорди, положення максимальної кривини 0,4с, максимальна товщина 12%.

5-значні (NASA 23012): розрахунковий $C_L = 0,3$, положення максимальної кривини 0,15с, максимальна товщина 12%.

6-значні (NASA 64₂-415, "ламінарний"): положення мінімального тиску на 0,4с, напівширина «ковша» малого опору за $CL = 0,2$, розрахунковий $C_L = 0,4$, товщина 15%.

30.2 Типові характеристики

Профіль	$C_{L,max}$	α_{stall}	$C_{d,min}$	$C_{m,ac}$
NACA 0012	1.45 (Re 3×10^6)	14°	0.0060	0.000
NACA 2412	1.55	15°	0.0065	-0.045
NACA 23012	1.70	18°	0.0070	-0.014
NACA 65-415	1.50	16°	0.0045	-0.075
Selig S1223 (малі Re)	2.20 (Re 2×10^5)	10°	0.0150	-0.27

30.3 Вплив числа Рейнольдса на БПЛА

Нижче за $Re \approx 5 \times 10^5$ домінують бульбашки ламінарного відриву, підвищуючи $C_{d,min}$ та знижуючи $C_{L,max}$. БПЛА за $Re \approx 10^5$ - 3×10^5 потребують спеціалізованих профілів для малих Re: Eppler 387, SD7037, Selig S1223. Аеродинамічні таблиці JSBSim для БПЛА мають відповідати експлуатаційному Re — не екстраполуйте від Re 10^6 в аеродинамічній трубі до польотного Re 10^5 .

30.4 Критичне число Маха

$$C_p^* = (2/\gamma M_\infty^2) [((1 + (\gamma-1)/2 \cdot M_\infty^2)/(1 + (\gamma-1)/2))^{\gamma/(\gamma-1)} - 1]$$

M_{crit} розв'язується одночасно з поправкою Prandtl-Glauert $C_p = C_{p,min,inc} / \sqrt{1 - M_{crit}^2}$. Число Маха дивергенції опору $M_{DD} > M_{crit}$ визначається там, де $dC_D/dM = 0,1$.

Надкритичні профілі (серія NASA SC(2) Віткомба) підвищують M_{DD} на 0,05-0,10 за тієї самої товщини або дозволяють на 30-60% більшу товщину за того самого M_{DD} — що дає змогу робити конструктивно легші крила.

Розділ 31: Методи CFD — теорія і практика

31.1 Панельні методи

Для незв'язкої нестисливої течії (або з поправкою P-G) навколо довільних форм метод Хесса-Сміта (Hess-Smith) розподіляє джерела та диполі по панелях тіла і забезпечує умову непротікання. Складність $O(N^2)$ за кількістю панелей. Метод дипольної решітки (Albano-Rodden 1969) поширюється на нестационарну коливальну течію і є основним інструментом аналізу флатера.

31.2 Вихрова решітка

Крила представлено підковоподібними вихорами на плоскій серединній поверхні, з вільними нитками, орієнтованими вздовж набігаючого потоку. Обчислює C_L , індуктивний опір, розподіл навантаження за розмахом. AVL Марка Дрели (Mark Drela) є канонічним інструментом з відкритим кодом для попереднього проєктування та оцінювання похідних стійкості. Обмеження: без товщини, без в'язких ефектів, без стисливості понад P-G, без відриву.

31.3 Моделі турбулентності RANS

- **Spalart-Allmaras (1992)**: одне рівняння перенесення для модифікованої турбулентної в'язкості $\tilde{\nu}$. Широко вживається у зовнішній аеродинаміці. Допускає вищі y^+ завдяки пристінній лінеаризації.
- **k- ϵ** : два рівняння. Погано працює за несприятливих градієнтів тиску та відриву; зазвичай потребує пристінних функцій.
- **k- ω SST (Menter, 1994)**: поєднує k- ω біля стінок з k- ϵ у віддаленому полі через зрощувальну функцію F1. Найкраща універсальна модель RANS для течій з відривом / несприятливим градієнтом тиску. Промисловий стандарт для механізації крила, крил, гвинтокрилих апаратів.

Роздільна здатність біля стінки: $y^+ < 1$ у першій комірці, 30-40 комірок уперек пограничного шару, коефіцієнт зростання за нормаллю до стінки $< 1,2$.

31.4 LES, DES, гібридні методи

LES розв'язує вихори аж до масштабу сітки; $y^+ \approx 1$ у всіх трьох напрямках, $\Delta x^+, \Delta z^+ \sim 50$. Вартість $\sim Re^{2.5}$ для LES з роздільною здатністю біля стінки — неприйнятна за польотних Re .

DES/DDES/IDDES (Spalart 1997): RANS у приєднаних пограничних шарах, LES у зонах відриву. Доступний за вартістю для течій з масштабним відривом (великі α , закритичні режими, відділення вантажу).

31.5 Валідаційні задачі

AGARD AR-303 та AR-138, NASA Turbulence Modeling Resource (turbmodels.larc.nasa.gov), механізація крила DLR-F6/F11, NASA Common Research Model (CRM), семінари HiLiftPW, семінари AIAA Drag Prediction Workshops 1-7. CFL3D, FUN3D та OVERFLOW є еталонними кодами.

Розділ 32: Вимушені коливання та похідні демпфування

32.1 Постановка задачі

Накладаємо синусоїдальний рух із частотою ω . Для тангажа:

$$\alpha(t) = \alpha_0 + \Delta\alpha \sin(\omega t), \quad q(t) = \omega \Delta\alpha \cos(\omega t)$$

Часозалежний (time-accurate) CFD-розрахунок проходить ~ 5 циклів, щоб розсіяти початкові перехідні процеси; дані беруть із циклів 3-5. Найкраща практика: ≥ 100 кроків за часом на цикл, подвійне крокування за часом (dual time-stepping) із 30-50 внутрішніми ітераціями.

32.2 Зведена частота

$$k = \omega L_{\text{ref}} / (2 V_{\infty})$$

Відношення нестационарного й конвективного часових масштабів. $k \rightarrow 0$ відповідає квазістационарному режиму (лише статичні похідні); $k > 0.05$ враховує запізнення циркуляції та ефекти приєднаної маси. Типові вимушені коливання в CFD: $k = 0.02-0.10$.

32.3 Виділення похідних

$$C_m(t) \approx C_{m0} + C_{m\alpha} \Delta\alpha + (C_{mq} + C_{m\dot{\alpha}})(\bar{c}/(2V)) \Delta\alpha \cdot \omega \cdot \cos(\omega t) / \sin(\omega t) \dots$$

Інтегрування за Фур'є на одному циклі:

$$\begin{aligned} C_{m\alpha} &\approx (1/(\pi \Delta\alpha)) \int_0^{2\pi/\omega} C_m(t) \sin(\omega t) dt \\ C_{mq} + C_{m\dot{\alpha}} &\approx (1/(\pi k \Delta\alpha)) \int_0^{2\pi/\omega} C_m(t) \cos(\omega t) dt \end{aligned}$$

Чисте демпфування C_{mq} та запізнення за $\dot{\alpha}$ $C_{m\dot{\alpha}}$ неможливо розділити з одного коливання лише за тангажем; для цього потрібен вертикальний рух (plunge — зміна α за сталого q) або комбінований графік. Багато довідників публікують суму $(C_{mq} + C_{m\dot{\alpha}})$ і розбивають її приблизно у співвідношенні 70/30.

Розділ 33: Геодезія та еліпсоїд WGS-84

JSBSim інтегрує свої рівняння руху в інерціальній системі координат і перетворює результати до еліпсоїда WGS-84 для виведення. Цей розділ є довідником щодо геодезичних сталих і домовленостей.

33.1 Визначальні сталі WGS-84

Символ	Значення	Опис
a	6 378 137.0 м (точно)	Велика піввісь
$1/f$	298.257 223 563 (точно)	Обернене стиснення
GM	$3.986\,004\,418 \times 10^{14} \text{ м}^3/\text{с}^2$	Гравітаційний параметр Землі
ω	$7.292\,115 \times 10^{-5} \text{ рад/с}$	Кутова швидкість обертання Землі
b	6 356 752.3142 м	Мала піввісь, $b = a(1-f)$
e^2	$6.694\,379\,990\,14 \times 10^{-3}$	Перший ексцентриситет ² = $2f - f^2$
e'^2	$6.739\,496\,742\,28 \times 10^{-3}$	Другий ексцентриситет ² = $e^2/(1-e^2)$
J_2	$1.082\,626\,7 \times 10^{-3}$	Друга зональна гармоніка (ненормована)
R_V	6 371 000.79 м	Середній (об'ємний) радіус
γ_e	9.780 325 м/с ²	Нормальна сила тяжіння на екваторі
γ_p	9.832 185 м/с ²	Нормальна сила тяжіння на полюсі
g_0	9.806 65 м/с ²	Стандартна сила тяжіння (за визначенням)
Зоряна доба	86 164.0905 с	Тривалість зоряної доби

33.2 Геодезична та геоцентрична широта

$$\tan(\varphi_c) = (1 - e^2) \tan(\varphi_g)$$

Вони відрізняються до 11.55 кутової хвилини $\approx 0.19^\circ$ поблизу $\varphi = 45^\circ$ — що відповідає 21.4 км відстані в напрямку північ-південь по поверхні Землі. В авіації завжди використовують геодезичну широту (GPS, карти, автопілоти). Внутрішні обчислення JSBSim іноді послуговуються геоцентричною; на виведенні присутні обидві.

33.3 Означення висоти

- **Еліпсоїдальна (геодезична) висота h** — знакова відстань до еліпсоїда WGS-84 уздовж нормалі до еліпсоїда. Власне виведення GPS.
- **Ортометрична висота H** — висота над геоїдом (еквіпотенціаль середнього рівня моря). Вода тече від високих H до низьких. Її використовують на картах.
- **Хвиля геоїда N** — $h = H + N$. У глобальному масштабі $N \in [-105 \text{ м}, +85 \text{ м}]$ відносно WGS-84. EGM96/EGM2008 дають глобальні моделі геоїда.
- **Барометрична висота (pressure altitude)** — висота в ISA, на якій виникає вимірний тиск. Висотомір із налаштуванням 29.92 inHg.
- **Висота за густиною (density altitude)** — те саме поняття для густини. Прогнозує характеристики літака та двигуна.
- **Геопотенціальна висота** — $H_{\text{geopot}} = R \cdot H_{\text{geom}} / (R + H_{\text{geom}})$. Використовується моделлю стандартної атмосфери, щоб гідростатичне рівняння мало сталу g .

33.4 Радіуси кривини

$$M(\varphi) = a(1-e^2) / (1 - e^2 \sin^2\varphi)^{3/2} \quad (\text{меридіан})$$

$$N(\varphi) = a / \sqrt{1 - e^2 \sin^2\varphi} \quad (\text{перший вертикал})$$

На екваторі $M = a(1-e^2)$, $N = a$; на полюсі $M = N = a/\sqrt{1-e^2} \approx 6\,399\,593.6$ м. Відстань на радіан: північ-південь M ; схід-захід $N \cos \varphi$. Одна кутова хвилина широти на екваторі дорівнює 1843 м; одна морська миля становить точно 1852 м за визначенням.

Розділ 34: Перетворення координат докладно

34.1 Геодезичні → ECEF (замкнена форма)

```
x = (N + h) cos(φ) cos(λ)
y = (N + h) cos(φ) sin(λ)
z = (N (1 - e²) + h) sin(φ)
where N(φ) = a / sqrt(1 - e² sin²φ)
```

Три множення та один квадратний корінь. JSBSim застосовує це на кожному такті, щоб відобразити проінтегроване інерціальне положення в геодезичну трійку (широта, довгота, висота) для виведення.

34.2 ECEF → геодезичні (ітерація Боврінга)

```
p = sqrt(x² + y²)
φ_0 = atan2(z, p (1 - e²))
repeat:
  N_i = a / sqrt(1 - e² sin² φ_i)
  h_i = p / cos(φ_i) - N_i
  φ_{i+1} = atan2(z, p (1 - e² N_i / (N_i + h_i)))
until |φ_{i+1} - φ_i| < 1e-12
λ = atan2(y, x)
```

Збіжність за 3 ітерації до 10^{-11} рад ≈ 0.06 мм будь-де на Землі. Гейккінен (Heikkinen, 1982) та Олсон (Olson, 1996) дають безітераційні замкнені альтернативи; метод Вермея (Vermeille, 2011) точний до нанометрів у межах 5000 км від еліпсоїда.

34.3 Поворот ECEF → ECI

$$\begin{bmatrix} x_{ECI} \\ y_{ECI} \\ z_{ECI} \end{bmatrix} = \begin{bmatrix} \cos(\theta) & -\sin(\theta) & 0 \\ \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} x_{ECEF} \\ y_{ECEF} \\ z_{ECEF} \end{bmatrix}$$

θ — це середній зоряний час за Гринвічем (GMST). Для прецизійних задач його доповнюють прецесією P , нутацією N , рухом полюсів W : $GCRF = P^T N^T R^T W^T \cdot ITRF$. Для авіасимуляції достатньо чистого повороту.

34.4 ECEF → NED у точці (ϕ_0, λ_0)

$$R_{NED \leftarrow ECEF} = \begin{bmatrix} -\sin(\phi_0)\cos(\lambda_0) & -\sin(\phi_0)\sin(\lambda_0) & \cos(\phi_0) \\ -\sin(\lambda_0) & \cos(\lambda_0) & 0 \\ -\cos(\phi_0)\cos(\lambda_0) & -\cos(\phi_0)\sin(\lambda_0) & -\sin(\phi_0) \end{bmatrix}$$

ENU отримують перестановкою перших двох рядків та зміною знаку третього. JSBSim кешує обидві матриці — NED-із-ECEF та ECEF-із-NED — на кожному такті.

34.5 NED → зв'язана (3-2-1 Тейта-Брайана)

$$R_n^b = R_x(\phi) R_y(\theta) R_z(\psi)$$

У розгорнутому вигляді:

$$R_{b \leftarrow n} = \begin{bmatrix} c\theta c\psi & c\theta s\psi & -s\theta \\ s\phi s\theta c\psi - c\phi s\psi & s\phi s\theta s\psi + c\phi s\psi & s\phi c\theta \\ c\phi s\theta c\psi + s\phi s\psi & c\phi s\theta s\psi - s\phi s\psi & c\phi c\theta \end{bmatrix}$$

Сингулярність при $\theta = \pm 90^\circ$ — це канонічне складання рамок (gimbal lock); саме тому JSBSim інтегрує кватерніонну форму і лише експортує кути Ейлера.

Розділ 35: Обертання Землі та сила тяжіння

35.1 Кутова швидкість обертання Землі та фіктивні сили

Обертова система ECEF є неінерціальною. Закон Ньютона набуває коріолісового та відцентрового доданків:

$$\mathbf{a}_{\text{inertial}} = \mathbf{a}_{\text{ECEF}} + 2 \boldsymbol{\Omega} \times \mathbf{v} + \boldsymbol{\Omega} \times (\boldsymbol{\Omega} \times \mathbf{r})$$

На крейсерському режимі М 0.8 (~250 м/с) на середніх широтах коріолісове прискорення становить ~0.036 м/с² (0.0037 g) — мале, але за нехтування ним воно накопичується у значні похибки курсу й траєкторії на масштабах годин польоту. Відцентровий доданок на екваторі становить ~0.034 м/с² назовні, що саме й пояснює, чому виміряна поверхнева сила тяжіння на екваторі (9.780 м/с²) менша за чисте гравітаційне притягання (9.814 м/с²). Відцентрову складову традиційно включають у модель сили тяжіння, тож у рівняннях руху явно лишається тільки коріолісова.

35.2 Сферична сила тяжіння (проста)

$$g(r) = GM / r^2 \text{ (радіальний напрямок назовні)}$$

На рівні моря $r = 6\,378$ км, $g \approx 9.798$ м/с². Достатня для багатьох авіасимуляцій; усталений варіант JSBSim.

35.3 Нормальна сила тяжіння за Сомільяною (поверхня WGS-84)

$$\gamma(\varphi) = \gamma_e (1 + k \sin^2 \varphi) / \sqrt{1 - e^2 \sin^2 \varphi}$$

де $\gamma_e = 9.7803254$ м/с² та $k = (b \gamma_p - a \gamma_e) / (a \gamma_e) \approx 0.001932$. Поправка вільного повітря (free-air) на висоту:

$$\gamma(\varphi, h) \approx \gamma(\varphi) [1 - (2/a)(1 + f + m - 2f \sin^2 \varphi) h + (3/a^2) h^2]$$

де $m = \omega_E^2 a^2 b / GM \approx 3.45 \times 10^{-3}$.

35.4 Збурення сили тяжіння J2

Відхилення поля сили тяжіння Землі від сферичної маси у головному порядку:

$$a_{J2} = -(3 J2 GM a^2) / (2 r^4) \times [(1 - 5 \sin^2 \varphi_c) \hat{x}, (1 - 5 \sin^2 \varphi_c) \hat{y}, (3 - 5 \sin^2 \varphi_c) \hat{z}]$$

Для літаків на висоті нижче 20 км у польотах тривалістю до 24 годин достатньо моделі Сомільяни з поправкою на висоту. Для ракет-носіїв, ракет і орбітального входу в атмосферу потрібні J2 (а часто й J3, J4).

Розділ 36: Магнітне поле та навігація

36.1 Світова магнітна модель (WMM 2025)

WMM — це спільний стандарт NCEI / British Geological Survey / NGA для головного геомагнітного поля. Поточна модель — WMM2025, чинна до 31 грудня 2029 року. Поле подається рядом сферичних гармонік (стандартний степінь/порядок 12, 133 для високороздільної WMMHR2025), де кожен коефіцієнт Гаусса змінюється лінійно в часі протягом епохи моделі.

У заданій точці (ϕ , λ , h , t) модель видає три геоцентричні компоненти (X на північ, Y на схід, Z униз), з яких:

- Повна напруженість $F = \sqrt{X^2 + Y^2 + Z^2}$
- Горизонтальна напруженість $H = \sqrt{X^2 + Y^2}$
- Магнітне схилення $D = \text{atan2}(Y, X)$ — кут від справжньої півночі до магнітної півночі, додатний на схід
- Магнітне нахилення $I = \text{atan2}(Z, H)$ — кут від горизонталі

Перетворення справжнього курсу на магнітний: $\psi_{\text{mag}} = \psi_{\text{true}} - D$. Магнітні компаси також потребують картки девіації (специфічної для планера) і мають похибки повороту на північ через нахилення.

36.2 GPS у WGS-84

Сузір'я GPS передає ефемериди в ECEF WGS-84 та супутниковий час у GPST. Приймач розв'язує систему з чотирма невідомими (3 координати ECEF + зсув годинника) з рівнянь псевдодальності

$$\rho_i = |\mathbf{r}_{\text{sat},i} - \mathbf{r}_{\text{rx}}| + c \cdot \Delta t_{\text{rx}} + \varepsilon_i$$

методом зважених найменших квадратів або розширеним фільтром Калмана. Розв'язок у ECEF потім перетворюється на (ϕ , λ , h) замкненими або ітераційними алгоритмами з попереднього розділу. Сучасні системи GNSS із підтримкою IHC досягають субметрової точності в динамічних режимах.

36.3 Відстань по великому колу за гаверсинусом

$$\begin{aligned} a &= \sin^2((\phi_2 - \phi_1)/2) + \cos(\phi_1) \cos(\phi_2) \sin^2((\lambda_2 - \lambda_1)/2) \\ c &= 2 \cdot \text{atan2}(\sqrt{a}, \sqrt{1-a}) \\ d &= R_E \cdot c \end{aligned}$$

де $R_E \approx 6\,371$ км. Початковий пеленг:

$$\theta_i = \text{atan2}(\sin(\Delta\lambda) \cos(\phi_2), \cos(\phi_1) \sin(\phi_2) - \sin(\phi_1) \cos(\phi_2) \cos(\Delta\lambda))$$

Похибки нижче 0.3% порівняно зі сплюснутою Землею. Ітераційний метод Вінсенті (Vincenty) (або варіант Карні, Karney) дає геодезичні відстані з субміліметровою точністю на WGS-84.

Розділ 37: Системи відліку часу в аерокосмічній галузі

Мають значення п'ять годинників. Чітке розуміння їх відрізняє симуляцію, що бездоганно узгоджується з GPS, ефемеридами та моделями магнітного поля, від тієї, що ні.

Шкала	Означення	Застосування
TAI	Міжнародний атомний час. Рівномірні секунди SI.	Основа; без розривів.
UTC	= TAI – N високосних секунд; UT1–UTC <0.9 с.	Цивільний час. UTC = TAI – 37 с (2026).
UT1	Час обертання Землі (середнє Сонце на меридіані → полудень).	Дрейфує; передається як DUT1 = UT1–UTC.
GPST	Атомний; розпочався від UTC 1980-01-06.	Супутниковий час GPS; = TAI – 19 с.
TT	= TAI + 32.184 с (за визначенням).	Ефемериди Сонячної системи.
GMST	Середній зоряний час за Гринвічем.	Поворот ECEF↔ECI; +3хв 56.6 с/добу відносно UTC.

Формула GMST (наближена, у градусах):

$$GMST = 280.46061837 + 360.98564736629 \cdot d + 0.000387933 \cdot T^2 - T^3 / 38710000$$

with d = days from J2000.0, T = centuries from J2000.0.

Розділ 38: Стандартна атмосфера докладно

Клас FGStandardAtmosphere у JSBSim реалізує стандартну атмосферу США 1976 року (NASA-TM-X-74335). Стандартна атмосфера ICAO ідентична до висоти 32 км.

38.1 Опорні значення на рівні моря

Величина	Символ	Значення
Температура	T_0	288.15 K (15°C)
Тиск	p_0	101 325 Pa
Густина	ρ_0	1.225 kg/m ³
Швидкість звуку	a_0	340.294 m/s
Динамічна в'язкість	μ_0	1.7894×10^{-5} Pa·s
Питома газова стала	R	287.058 J/(kg·K)
Показник адіабати	γ	1.40
g_0	g_0	9.806 65 m/s ²

38.2 Структура шарів (від 0 до 86 км)

Шар	Нижня висота (км)	Верхня висота (км)	Вертикальний градієнт L (K/км)
Тропосфера	0	11	-6.5
Тропопауза	11	20	0
Стратосфера 1	20	32	+1.0
Стратосфера 2	32	47	+2.8
Стратопауза	47	51	0
Мезосфера 1	51	71	-2.8
Мезосфера 2	71	84.852	-2.0

У межах кожного неізотермічного шару гідростатичне рівняння

$$dp/dh = -\rho g, \quad p = \rho RT, \quad T(h) = T_b + L(h - h_b)$$

інтегрується до

$$p(h) = p_b \cdot [T_b / (T_b + L(h - h_b))]^{g_0 / (R \cdot L)}$$

Для ізотермічних шарів ($L = 0$) розв'язок є експоненційним:

$$p(h) = p_b \cdot \exp[-g_0(h - h_b) / (R T_b)]$$

38.3 Похідні величини

$$a = \sqrt{\gamma RT} \quad (\text{швидкість звуку})$$

$$M = V/a \quad (\text{число Маха})$$

$$\mu(T) = \mu_{\text{ref}} (T/T_{\text{ref}})^{1.5} (T_{\text{ref}} + S) / (T + S)$$

(закон Сазерленда, із $S = 110.4$ K для повітря). Число Рейнольдса $Re = \rho V L / \mu$. Повний тиск/тиск гальмування та температура:

$$p_t = p(1 + (\gamma - 1)/2 \cdot M^2)^{\gamma / (\gamma - 1)}$$

$$T_t = T(1 + (\gamma - 1)/2 \cdot M^2)$$

Розділ 39: Вітер, турбулентність і пориви

39.1 Профіль вітру поблизу поверхні

$$u(z) = (u_*/\kappa) \ln(z/z_0) \quad (\text{логарифмічний закон, } \kappa \approx 0.41)$$

$$u(h) = u_{\text{ref}} (h/h_{\text{ref}})^\alpha \quad (\text{степеневий закон, } \alpha \approx 1/7 \text{ над відкритою місцевістю})$$

39.2 Дискретні пориви (MIL-F-8785C, 1-косинус)

$$V_{\text{gust}}(t) = (V_m/2) (1 - \cos(\pi t/T_m))$$

Використовується для сертифікації пілотажних характеристик. T_m лежить у діапазоні від 0.5 с (малий БПЛА) до кількох секунд (транспортні літаки); V_m типово 5-15 м/с.

39.3 Неперервна турбулентність — СЩП за Драйденом

$$\Phi_u(\omega) = \sigma_u^2 \cdot (2L_u/V) / (1 + (L_u\omega/V)^2)$$

$$\Phi_v(\omega) = \sigma_v^2 \cdot (L_v/V) \cdot (1 + 3(L_v\omega/V)^2) / (1 + (L_v\omega/V)^2)^2$$

$$\Phi_w(\omega) = \sigma_w^2 \cdot (L_w/V) \cdot (1 + 3(L_w\omega/V)^2) / (1 + (L_w\omega/V)^2)^2$$

Масштаби довжини $L_{u,v,w}$ та інтенсивності $\sigma_{u,v,w}$ задаються за смугами висот у MIL-F-8785C / MIL-HDBK-1797. Імовірності слабкої/помірної/сильної інтенсивності зменшуються з висотою; турбулентність вище тропопаузи трапляється рідко.

СЩП за фон Карманом — це альтернатива (точніша на високих частотах), яку часто застосовують в аналізі флатера.

39.4 Мікропорив (Вікрой, Vicroy)

Мікропорив (microburst) моделюється як трикомпонентне поле течії: горизонтальний відтік + спадний потік + вихрове кільце. Аналітична модель Вікроя параметризує профіль радіального відтоку радіусом ядра, висотою вихрового кільця та піковою швидкістю відтоку. Використовується для тренувальних сценаріїв виходу зі зсуву вітру.

Розділ 40: Теорія силової установки

40.1 Поршневі двигуни — цикл Отто

Чотиритактний двигун: впуск, стиснення (адіабатичне), згоряння (за сталого об'єму), розширення (адіабатичне), випуск. Ідеальний ККД циклу Отто $\eta = 1 - (1/r)^{\gamma-1}$ зі ступенем стиснення r . Питома витрата палива (BSFC) типово 0.4-0.5 lb/(hp·hr) для безнаддувних двигунів на авіаційному бензині.

Характеристики масштабуються з абсолютним тиском у впускному колекторі (MAP), частотою обертання (RPM) та сумішшю. Клас FGPISTON використовує параболичну залежність потужності від MAP і лінійне масштабування за дроселем/RPM. Зниження характеристик з висотою (derating) враховує зменшення густини довкілля (якщо немає нагнітача чи турбонаддуву).

40.2 Турбіна — цикл Брайтона

Цикл із неперервним потоком: стиснення, згоряння ($\approx$ за сталого тиску), розширення. Ідеальний ККД циклу Брайтона $\eta = 1 - (1/\pi)^{(\gamma-1)/\gamma}$ зі ступенем підвищення тиску π . У сучасних турбовентиляторних двигунах $\pi \approx 30-50$, TSFC 0.30-0.50 lb/(lbf·hr) на крейсерському режимі, 1.5-2.5 з увімкненим форсажем (AB).

Ступінь двоконтурності: чистий турбореактивний ≈ 0 ; військовий із низькою двоконтурністю ≈ 0.3 ; комерційний із високою $\approx 10-12$ (GE9X, Trent XWB). Висока двоконтурність = більша масова витрата на нижчій швидкості = тихіше й ефективніше на дозвукових швидкостях.

Двовальна архітектура: N1 (ротор низького тиску, вентилятор) та N2 (ротор високого тиску, компресор + турбіна). N1 — це основний параметр режиму тяги на більшості комерційних двигунів.

40.3 Ракета — $F = \dot{m} V_e + (p_e - p_a)A_e$

$$I_{sp} = F / (\dot{m} \cdot g_0) \text{ (секунди)}$$

Питомий імпульс вимірює паливну ефективність. Типові значення: тверде паливо 250 с, RP-1/LOX 350 с, LH2/LOX 450 с, іонний >3000 с. Рівняння Ціолковського:

$$\Delta v = I_{sp} g_0 \ln(m_0/m_f)$$

Виведення на орбіту вимагає $\Delta v \approx 9-10$ км/с з урахуванням втрат на силу тяжіння та опір. Багатоступеневість необхідна, бо m_0/m_f експоненційно залежить від Δv .

40.4 Електродвигуни — BLDC

Безщітковий двигун постійного струму описується (на фазу):

$$V = K_e \cdot \omega + I \cdot R, \quad \tau = K_t \cdot I, \quad K_t = 1/K_v \text{ in SI units}$$

де K_v в rpm/V (хобі-домовленість). Потужність $P = V \cdot I = \omega \cdot \tau + I^2 \cdot R$; другий доданок — резистивні втрати. ККД $\eta = \omega \cdot \tau / (V \cdot I)$ досягає максимуму за проміжних навантажень ($\sim 50-70\%$ номінального струму).

Акумулятор: елементи LiPo номінально 3.7 V (4.2 V макс., 3.0 V мин.); C-рейтинг обмежує піковий розряд (напр., 25 C $\times$ 5000 mAh = 125 A макс.). Рівень заряду інтегрує струм за часом: $SoC(t) = SoC(0) - \int I/Q \cdot dt$.

Розділ 41: Теорія повітряного та несного гвинта

41.1 Повітряний гвинт — імпульсна теорія + елемент лопаті

Імпульсна теорія (Фруда): ідеальний диск прискорює потік від V до $V+2v_i$, створюючи тягу

$$T = 2 \rho A (V + v_i) v_i$$

де v_i — індуктивна швидкість на диску. Потужність $P = T (V + v_i)$; ККД $\eta = TV/P \rightarrow 1$ лише за нульового v_i (нескінченна площа диска, межа активного диска).

Теорія елемента лопаті аналізує кожен елемент лопаті як 2-D профіль із локальними α та $V_{\text{resultant}}$. Клас FGPropeller у JSBSim використовує табличні криві $C_T(J)$ та $C_P(J)$, де відносна хода (advance ratio)

$$J = V / (n \cdot D), \quad n \text{ in rev/s, } D = \text{diameter}$$

$$\text{Thrust} = C_T(J) \cdot \rho \cdot n^2 \cdot D^4$$

$$\text{Power} = C_P(J) \cdot \rho \cdot n^3 \cdot D^5$$

$$\text{Efficiency } \eta_p = J \cdot C_T / C_P$$

41.2 Р-фактор та ефекти, спричинені гвинтом

- **Р-фактор:** за ненульового α спадна лопать має вищий локальний кут атаки, ніж висхідна, що створює бічний зсув тяги, який спричиняє ризикання літака.
- **Обертання струменя гвинта:** струмінь гвинта закручується по спіралі, асиметрично потрапляючи на вертикальне оперення і створюючи момент ризикання.
- **Гіроскопічна прецесія:** літак на режимі тангажа прикладає вхідний момент відносно поперечної осі; обертаний гвинт реагує прецесійним ризиканням, і навпаки.
- **Реактивний момент:** третій закон Ньютона — двигун, що крутить гвинт в один бік, крутить літак в інший. Напрямок (за/проти годинникової стрілки з кабіни) визначає, у який саме.

41.3 Гвинти зі сталою частотою обертання

Регулятор частоти обертання змінює крок лопаті, щоб утримувати задану RPM незалежно від положення дроселя. Це вмикається через `<constspeed>1</constspeed>` у XML гвинта в JSBSim. Нижче діапазону регулятора гвинт повертається до фіксованого кроку на `<minpitch>`. Бета-діапазон (нижче польотного малого газу) та реверс (від'ємний крок) підтримуються на турбогвинтових двигунах.

41.4 Несний гвинт вертольота

Несний гвинт — це повітряний гвинт, що також створює піднімальну силу. Тяга на висінні (імпульсна теорія):

$$T = 2 \rho A v_i^2, \quad v_i = \sqrt{T/(2\rho A)}$$

Поступальна піднімальна сила: у горизонтальному польоті приплив стає асиметричним; наступаюча лопать бачить $V + \omega \cdot r$, відступаюча — $\omega \cdot r - V$. Відступаюча лопать наближається до зриву на високій поступальній швидкості — це межа V_{NE} вертольота.

Циклічний крок: крок лопаті змінюється раз за оберт, щоб нахилити диск несного гвинта, створюючи сили в будь-якому напрямку. **Загальний крок:** усі лопаті змінюють крок разом, керуючи величиною тяги. Клас FG_Rotor у JSBSim реалізує елемент лопаті + імпульсну теорію з динамікою махання; приклади X-15 та Pterosaur є гарною відправною точкою.

Розділ 42: Обробка сигналів для САК

42.1 Фільтри в неперервному часі

Аперіодична ланка: $H(s) = c_1/(s + c_1)$
Ланка випередження-запізнення: $H(s) = (c_1s + c_2)/(c_3s + c_4)$
Розмивна (washout) ланка: $H(s) = s/(s + c_1)$
Другого порядку: $H(s) = (c_1s^2 + c_2s + c_3)/(c_4s^2 + c_5s + c_6)$

Усі чотири — це компоненти-фільтри JSBSim із розділу 12. Коефіцієнти $c_{1..6}$ беруться безпосередньо з проєкту в неперервній s-площині.

42.2 Дискретизація: білінійне (Тастіна) перетворення

$$s \leftarrow (2/T) \cdot (z - 1)/(z + 1)$$

Перетворення Тастіна відображає всю ліву півплощину (стійкий неперервний випадок) в одиничний круг (стійкий дискретний), тож стійкість зберігається. Частоти спотворюються: $\omega_d = (2/T) \tan(\omega_a T/2)$. Для критичних частот полюсів/нулів «передспотворення» (prewarping) (заміна $2/T$ на $\omega_0/\tan(\omega_0 T/2)$) зберігає їхнє розташування точно.

42.3 Дискретизація ПІД-регулятора

$$u(z) = K_p \cdot e(z) + K_i \cdot T/2 \cdot (1 + z^{-1})/(1 - z^{-1}) \cdot e(z) + K_d \cdot (1 - z^{-1})/T \cdot e(z)$$

(trapezoidal)
(backward Euler)

Елемент <pid> у JSBSim пропонує чотири методи інтегрування (rect = зворотний метод Ейлера, trap = трапецій, ab2/ab3 = Адамса-Бешфорта). Елемент <trigger> забезпечує захист від насичення інтегратора (anti-windup): за ненульового значення інтеграл заморожується, за від'ємного — скидається.

42.4 Правила налаштування ПІД-регулятора

Метод	Kp	Ki	Kd
Циглера-Нікольса (за коливаннями)	$0.6 \cdot K_u$	$1.2 \cdot K_u / T_u$	$0.075 \cdot K_u \cdot T_u$
Лямбда-налаштування (FOPDT, $\lambda = \tau$ замкнутого контуру)	$\tau/(K(\lambda+\theta))$	Kp/τ	0 (без D)
ІМС PI	$(2\tau+\theta)/(2K(\lambda+\theta))$	$Kp/(\tau+\theta/2)$	—

Розділ 43: Лінійна задача доповнюваності та контактне тертя

43.1 Формулювання LCP

Лінійна задача доповнюваності (LCP) шукає $\mathbf{z} \in \mathbb{R}^n$ такий, що

$$\mathbf{w} = \mathbf{M}\mathbf{z} + \mathbf{q}, \quad \mathbf{w} \geq 0, \quad \mathbf{z} \geq 0, \quad \mathbf{w}^T \mathbf{z} = 0$$

тобто для кожного i або $w_i = 0$, або $z_i = 0$. Контакт із тертям природно є LCP: z_i — це контактний імпульс на i -му контакті, w_i — відносна швидкість за нормаллю до поверхні; або контакт є ($z > 0, w = 0$), або його немає ($z = 0, w > 0$).

43.2 LCP шасі в JSBSim

Кожен контакт шасі додає:

- Обмеження непроникнення за нормаллю до злітно-посадкової смуги.
- Обмеження кулонівського тертя по дотичній до смуги (поздовжній / бічний / опір коченню).
- Гальмівний момент від САК як додаткове обмеження конуса тертя.

Метод `FGAccelerations::CalculateFrictionForces()` у JSBSim розв'язує отриману LCP проєкційною ітерацією Гаусса-Зейделя (Catto 2005), до 50 внутрішніх ітерацій на такт. Це той самий алгоритм, що застосовується в сучасних ігрових фізичних рушіях (Bullet, ODE, Box2D).

Чому LCP, а не просто явне обчислення сил? Бо жорстка система пружина-демпфер-тертя є безумовно нестійкою за явного інтегрування, якщо допускати взаємопроникнення. Формулювання LCP проєктує контакт на допустимий многовид обмежень на кожному такті, що є неявно стійким незалежно від жорсткості.

Розділ 44: Алгоритм балансування зсередини

Клас FGTrim знаходить стан літака та положення органів керування, що задовольняють умову усталеного польоту. Його реалізовано як композицію одновимірних секансних пошуковців коренів, координованих зовнішнім циклом.

44.1 FGTrimAxis: елементарна комірка

Кожен FGTrimAxis поєднує змінну стану, яку треба обнулити (напр. $\dot{w}$), зі змінною керування, яку треба варіювати (напр. α). За двома пробними значеннями керування з відповідними значеннями стану секансне оновлення таке:

$$x_{n+1} = x_n - f(x_n) \cdot (x_n - x_{n-1}) / (f(x_n) - f(x_{n-1}))$$

JSBSim застосовує релаксацію 0.9, щоб гасити коливання:

$$x_{n+1} = x_n - 0.9 f(x_n) (\Delta x / \Delta t)$$

Якщо секансний крок виходить за межі інтервалу, JSBSim повертається до бісекції на цій осі. Кожна вісь збігається до допуску: $1e-3$ для поступального $\ddot{x}$, $1e-4$ (у $10\times$ жорсткіше) для кутового $\dot{\omega}$.

44.2 Режими балансування

Режим (TrimMode)	Обнулювані стани	Варійовані органи керування
tLongitudinal	$\dot{u}=0, \dot{w}=0, \dot{q}=0$	дросель, α , кермо висоти
tFull	$+ \dot{v}=0, \dot{p}=0, \dot{r}=0$, утримання ψ	$+ \phi$, елерон, кермо напрямку, β
tFullWingsLevel	tFull, але $\phi=0$; β розв'язує $\dot{v}=0$	дросель, α , кермо висоти, елерон, кермо напрямку, β
tGround	$\dot{w}=0, \dot{q}=0, \dot{p}=0$	висота (AGL), θ , ϕ
tPullup	поздовжній, n_z = ціль	α , дросель, кермо висоти
tTurn	координований віраж із заданим креном	дросель, кермо висоти, кермо напрямку
tTurnFull	координований віраж + кутові швидкості крену/рискання	повний набір
tCustom	побудований користувачем через AddState/RemoveState	обраний користувачем

44.3 Зовнішній цикл та збіжність

Усталені значення: SetMaxCycles(60) проходів по списку осей, SetMaxCyclesPerAxis(100) ітерацій на внутрішню вісь. У межах кожного циклу осі баланшуються у фіксованому порядку. Якщо всі осі одночасно досягають своїх допусків після повного циклу, балансування оголошується збіжним, а simulation/trim-completed встановлюється в 1; інакше часова історія відновлюється і виникає помилка.

Розділ 45: Контрольні приклади верифікації NASA NESCS 2015

JSBSim був єдиним учасником із відкритим кодом у програмі верифікації руху з 6 ступенями вільності Інженерно-безпекового центру NASA (NASA/TM-2015-218675). Шістьма іншими інструментами були внутрішні симулятори NASA (LaSRS++, SES, Marvin, POST-II, OSIRIS, MAVERIC). NESCS дійшов висновку, що всі сім симуляторів узгоджувалися до придатного для публікації ступеня на більшості прикладів, а решта відмінностей була пояснена й могла бути зменшена.

45.1 Приклади атмосферного польоту

#	Приклад	Що перевіряє
1	Скинута куля, без опору	Вільне падіння в однорідному полі сили тяжіння
2	Цеглина, що перекидається, без опору	Рівняння Ньютона-Ейлера, тензор інерції
3	Цеглина, що перекидається + аеродинамічне демпфування	Додає демпфувальні моменти до твердотілого прикладу 2
4	Скинута куля, плоска Земля	Перевіряє вибір моделі сили тяжіння
5	Скинута куля, обертова сферична Земля	Додає коріолісову складову
6	Скинута куля, обертова еліпсоїдальна Земля	Додає геодезію WGS-84
7-8	Скинута куля, сталий / змінний вітер	Профілі зсуву вітру та поривів
9-10	Балістика на схід / на північ	Коріолісові компоненти
11	Дозвукове балансування F-16	Алгоритм балансування, власна структура короткоперіодичного руху
12	Надзвукове балансування F-16	Трансзвукова / надзвукова аеродинаміка, балансування на $M > 1$
13.1-13.4	Збурювальні маневри F-16	Дублет за висотою, ступінчаста зміна швидкості, ступінчаста зміна курсу
15-16	Глобальні польоти F-16	Над Північним полюсом, навколо екватора
17	Двоступенева ракета від рівня моря до орбіти	Перехід атмосферний → орбітальний

Реалізації тестових прикладів NASA для JSBSim розташовані за адресою github.com/open-aerospace/jsbsim-nasa-test-cases; їх запуск — рекомендований спосіб верифікувати збірку JSBSim проти опублікованих еталонних траєкторій. Власні значення короткоперіодичного руху та руху голландського кроку F-16 збігаються між симуляторами до третьої значущої цифри; нев'язки балансування нижчі за допуски на вісь $1e-3 \text{ ft/s}^2$ та $1e-4 \text{ rad/s}^2$, усталені в JSBSim.

Розділ 46: Повне дерево властивостей

Далі наведено майже повний перелік простору імен властивостей JSBSim, узагальнений з довідника API Doxygen, онлайн-посібника та викликів PropertyManager->Tie() у вихідному коді кожної моделі.

46.1 simulation/

simulation/sim-time-sec	cumulative simulation time
simulation/dt	integration step
simulation/frame	tick counter
simulation/do_simple_trim	0=long, 1=full, 2=ground, 3=pullup, 4=custom, 5=turn, 6=none
simulation/do_linearization	write nonzero to dump A,B,C,D
simulation/trim-completed	1 after successful trim
simulation/reset	reset to IC
simulation/pause	freeze model
simulation/terminate	stop script
simulation/integrator/rate/rotational	
simulation/integrator/rate/translational	
simulation/integrator/position/rotational	
simulation/integrator/position/translational	
simulation/gravity-model	0=spherical, 1=WGS-84
simulation/gravitational-torque	0/1 spacecraft tidal
simulation/randomseed	seeds random/urandom functions
simulation/disperse	Monte-Carlo enable
forces/hold-down	hold aircraft fixed at IC

46.2 ic/

ic/u-fps, ic/v-fps, ic/w-fps	body-axis velocity
ic/vn-fps, ic/ve-fps, ic/vd-fps	NED velocity
ic/vt-fps, ic/vt-kts	true airspeed
ic/vc-kts	calibrated airspeed
ic/ve-kts	equivalent airspeed
ic/vg-kts	ground speed
ic/mach	
ic/phi-deg, ic/theta-deg, ic/psi-true-deg	
ic/alpha-deg, ic/beta-deg, ic/gamma-deg	
ic/lat-gc-deg, ic/lat-geod-deg	
ic/long-gc-deg	
ic/h-sl-ft, ic/h-agl-ft, ic/terrain-elevation-ft	
ic/vw-dir-deg, ic/vw-mag-fps	
ic/vw-north-fps, ic/vw-east-fps, ic/vw-down-fps	
ic/p-rad_sec, ic/q-rad_sec, ic/r-rad_sec	
ic/roc-fpm, ic/roc-fps	
ic/targetNlf	load factor target

46.3 position/, attitude/

position/h-sl-ft, h-sl-meters
 position/h-agl-ft
 position/lat-gc-deg, lat-gc-rad, lat-geod-deg
 position/long-gc-deg, long-gc-rad
 position/radius-to-vehicle-ft
 position/distance-from-start-lat-mt
 position/distance-from-start-lon-mt
 position/distance-from-start-mag-mt
 position/terrain-elevation-asl-ft
 position/epa-rad Earth position angle
 position/eci-x-ft, eci-y-ft, eci-z-ft
 position/ecef-x-ft, ecef-y-ft, ecef-z-ft
 attitude/phi-rad, theta-rad, psi-rad
 attitude/phi-deg, theta-deg, psi-deg
 attitude/heading-true-rad
 attitude/roll-rad, pitch-rad

46.4 velocities/

velocities/u-fps, v-fps, w-fps body
 velocities/u-aero-fps, v-aero-fps, w-aero-fps body, airmass-rel
 velocities/p-rad_sec, q-rad_sec, r-rad_sec body rates
 velocities/p-aero-rad_sec, q-aero-rad_sec, r-aero-rad_sec
 velocities/pi-rad_sec, qi-rad_sec, ri-rad_sec inertial rates
 velocities/vt-fps, vt-kts
 velocities/vc-fps, vc-kts calibrated
 velocities/ve-fps, ve-kts equivalent
 velocities/vg-fps ground
 velocities/mach, machU
 velocities/h-dot-fps climb rate
 velocities/v-north-fps, v-east-fps, v-down-fps
 velocities/eci-velocity-mag-fps

46.5 aero/, atmosphere/

aero/qbar-psf, qbarUW-psf, qbarUV-psf
 aero/alpha-rad, alpha-deg, beta-rad, beta-deg
 aero/alphadot-rad_sec, betadot-rad_sec
 aero/mag-beta-rad, mag-alpha-rad
 aero/Re Reynolds number
 aero/bi2vel, ci2vel $b/(2V)$, $cbar/(2V)$
 aero/alpha-wing-rad incidence adjusted
 aero/h_b-cg-ft, h_b-mac-ft ground-effect ratios
 aero/stall-hyst-norm stall hysteresis
 aero/cl-squared
 aero/coefficient/<axis>/<name> every defined coefficient
 atmosphere/T-R, T-sl-R, delta-T
 atmosphere/rho-slugs_ft3, rho-sl-slugs_ft3
 atmosphere/P-psf, P-sl-psf
 atmosphere/a-fps, a-sl-fps
 atmosphere/delta, theta, sigma
 atmosphere/density-altitude, pressure-altitude
 atmosphere/wind-north-fps, wind-east-fps, wind-down-fps
 atmosphere/total-wind-fps, psiw-rad
 atmosphere/turbulence/milspec/severity 0..7

46.6 forces/, moments/, accelerations/

forces/fbx-aero-lbs, fby-aero-lbs, fbz-aero-lbs
 forces/fwx-aero-lbs, fwy-aero-lbs, fwz-aero-lbs wind-axis
 forces/fbx-prop-lbs, fby-prop-lbs, fbz-prop-lbs
 forces/fbx-gear-lbs, fby-gear-lbs, fbz-gear-lbs
 forces/fbx-total-lbs, fby-total-lbs, fbz-total-lbs
 forces/hold-down
 moments/l-aero-lbsft, m-aero-lbsft, n-aero-lbsft
 moments/l-prop-lbsft, m-prop-lbsft, n-prop-lbsft
 moments/l-gear-lbsft, m-gear-lbsft, n-gear-lbsft
 moments/l-total-lbsft, m-total-lbsft, n-total-lbsft
 accelerations/pdot-rad_sec2, qdot-rad_sec2, rdot-rad_sec2
 accelerations/udot-ft_sec2, vdot-ft_sec2, wdot-ft_sec2
 accelerations/Nx, Ny, Nz load factors (g)
 accelerations/n-pilot-x-norm, n-pilot-y-norm, n-pilot-z-norm
 accelerations/a-pilot-x-ft_sec2, a-pilot-y, a-pilot-z
 accelerations/gravity-ft_sec2

46.7 fcs/, gear/

fcs/elevator-cmd-norm, aileron-cmd-norm, rudder-cmd-norm
 fcs/flap-cmd-norm, speedbrake-cmd-norm, spoiler-cmd-norm
 fcs/pitch-trim-cmd-norm, roll-trim-cmd-norm, yaw-trim-cmd-norm
 fcs/steer-cmd-norm
 fcs/throttle-cmd-norm[n], mixture-cmd-norm[n]
 fcs/throttle-pos-norm[n], mixture-pos-norm[n]
 fcs/advance-cmd-norm[n], feather-cmd-norm[n]
 fcs/magneto-cmd[n], starter-cmd[n]
 fcs/elevator-pos-rad, -deg, -norm
 fcs/left-aileron-pos-..., right-aileron-pos-...
 fcs/rudder-pos-...
 fcs/flap-pos-deg, flap-pos-norm
 fcs/speedbrake-pos-rad, spoiler-pos-rad
 fcs/wing-fold-pos-norm
 fcs/left-brake-cmd-norm, right-brake-cmd-norm, center-brake-cmd-norm
 gear/gear-cmd-norm, gear-pos-norm, tailhook-pos-norm
 gear/unit[i]/compression-ft, WOW, wheel-speed-fps,
 static-friction-coeff, pos-x, pos-y, pos-z

46.8 propulsion/, inertia/, metrics/

propulsion/engine[i]/thrust-lbs, power-hp, rpm
 propulsion/engine[i]/fuel-flow-rate-pps, fuel-flow-rate-gph
 propulsion/engine[i]/n1, n2
 propulsion/engine[i]/egt-degF, oil-pressure-psi, oil-temperature-degF
 propulsion/engine[i]/bsfc-lbs_hphr
 propulsion/engine[i]/mp-osi, volumetric-efficiency
 propulsion/engine[i]/set-running start flag
 propulsion/tank[i]/contents-lbs, capacity-gal_us
 propulsion/total-fuel-lbs, refuel
 inertia/empty-weight-lbs, weight-lbs, mass-slugs
 inertia/cg-x-in, cg-y-in, cg-z-in
 inertia/ixx-slugs_ft2, iyy-, izz-, ixy-, ixz-, iyz-
 metrics/Sw-sqft, bw-ft, cbarw-ft
 metrics/Sh-sqft, lh-ft, Sv-sqft, lv-ft
 metrics/aero-rp-x-in, aero-rp-y-in, aero-rp-z-in

Розділ 47: Мова функцій — повний довідник операторів

Кожен оператор, придатний для використання всередині блоку `<function>` у `<aerodynamics>`, `<flight_control>`, `<system>` чи `<external_reactions>`, узагальнений з довідника Doxygen FGFunction. Скорочення: `<v>` = `<value>`, `<p>` = `<property>`, `<t>` = `<table>`.

Оператор	Опис
sum	Додає всі безпосередні дочірні елементи.
difference	Перший дочірній елемент мінус сума решти дочірніх.
product	Перемножує всі дочірні елементи.
quotient	Перший дочірній елемент, поділений на другий.
pow	Перший дочірній елемент, піднесений до другого.
sqrt	Квадратний корінь.
exp	е в степені дочірнього елемента.
ln, log2, log10	Натуральний логарифм, за основою 2, за основою 10.
abs, sign	Абсолютне значення, знак.
sin, cos, tan	Тригонометричні (аргумент у радіанах).
asin, acos, atan	Обернені тригонометричні; результат у радіанах.
atan2	atan2(Y, X); діапазон –п..п.
toradians, todegrees	Перетворення кутових одиниць.
pi	Стала π. Вживається як .
lt, le, gt, ge, eq, neq	Повертає 1, якщо відношення виконується, інакше 0.
and, or, not	Булеві. and/or приймають n дочірніх елементів.
ifthen	ifthen(умова, true, false). Усталене false = 0.
switch	switch(індекс, v0, v1, ...); індекс від нуля.
min, max, avg	Агрегування за дочірніми елементами.
floor, ceil, integer, fraction	Операції округлення.
mod, fmod, roundmultiple	Остача, остача з рухомою комою, округлення до кратного.
random	Гаусів; атрибути seed, mean, stddev.
urandom	Рівномірний; атрибути seed, lower, upper.
value, v	Числовий літерал.
property, p	Читання властивості (рядкове тіло).
table, t	1-D, 2-D або 3-D таблиця.
interpolate1d	1-D вбудована інтерполяція.
rotation_alpha_local, rotation_beta_local, rotation_gamma_local	Спеціальні повороти (6 аргументів).
rotation_bf_to_wf, rotation_wf_to_bf	Повороти зв'язана↔вітрова (7 аргументів).

Розділ 48: Процедури верифікації та валідації

48.1 Ієрархія V&V

- **Верифікація** (verification): чи правильно ми розв'язуємо рівняння? Перевірка проти аналітичних розв'язків, проти опублікованих еталонних траєкторій (NESC), проти власних попередніх результатів симулятора після зміни коду.
- **Валідація** (validation): чи розв'язуємо ми правильні рівняння? Порівняння з льотними випробуваннями, аеродинамічною трубою, паспортними числами реального літака. Валідація завжди проводиться проти зовнішнього еталона.
- **Аналіз чутливості**: збурити кожен параметр (центр мас, I_{yy} , $C_{m\alpha}$ тощо) і кількісно оцінити зсув відгуку. Виявляє параметри, у які найбільше варто вкладатися.

48.2 Контрольний список верифікації (на кожен реліз)

1. Build and run unit tests (CMake/CTest).
2. Run NESC atmospheric cases 1-17. Compare against open-aerospace/jsbsim-nasa-test-cases reference CSVs. Acceptable error: $<1e-4$ in normalised state.
3. Run all aircraft in aircraft/ through their reset00 IC and do_simple_trim=0. Every aircraft must trim within 60 cycles.
4. For each trimmed aircraft, run a pitch doublet and check short-period damping ratio is in $[0.2, 0.9]$; phugoid in $[0.005, 0.1]$.
5. Run forced-oscillation about Y at ± 1 deg, $k=0.05$, on the F-16 cruise condition; extract $C_{mq} + C_{m\alpha}$; compare to NASA-2015 reference -22.0 ± 1.0 1/rad.

48.3 Валідація за льотними даними

Коли наявні дані льотних випробувань (ступінчасті відгуки з оцінкою за шкалою Купера-Гарпера, відпускання штурвала, дублети), процедура така:

- Точно відтворити умови балансування: вагу, центр мас, висоту, число Маха, стан палива. Задokumentувати конфігурацію літака (шасі, закрилки, підвіски).
- Подати на симулятор записані команди пілота (цифрові траси штурвала/важеля газу).
- Побудувати графіки p , q , r , α , β , n_z , висоти, IAS у накладенні із сигналами льотних випробувань. Обчислити середньоквадратичну (RMS) похибку для кожного каналу.
- Уточнювати аеродинамічні коефіцієнти для мінімізації похибки методом найменших квадратів (least-squares system ID) — але завжди проти відкладеного валідаційного маневру, щоб уникнути перенавчання.

48.4 Пілотажні характеристики (MIL-F-8785C / MIL-HDBK-1797)

Сучасна оцінка пілотажних характеристик прогнозує пілотну оцінку за шкалою Купера-Гарпера з розімкнених і замкнених параметрів. Ключові межі:

- Короткоперіодична ω_n відносно n/α (межі Категорії A/B/C, Рівня 1/2/3).
- Коефіцієнт демпфування фугоїда (Рівень 1: $\zeta > 0.04$).
- Стала часу режиму крену τ_{roll} (Рівень 1: < 1 с).
- Голландський крок $\zeta \cdot \omega_n > 0.15$ (Рівень 1).
- Смуга пропускання та закид (dropback) (сучасні критерії; доповнюють MIL-F-8785C).

Частина III

Від CFD до літаючої моделі: робочий процес від OpenFOAM до JSBSim

Найвибагливіша частина створення нового літального апарата — це наповнення блоку <aerodynamics> числами, які насправді відповідають вашому планеру. Частина III — це наскрізний, відтворюваний рецепт генерування цих чисел за допомогою відкритого пакета обчислювальної гідрогазодинаміки (CFD) OpenFOAM та їх перенесення в JSBSim. Ми розглядаємо підготовку геометрії та побудову сітки, стаціонарне отримання статичних коефіцієнтів сил і моментів, три взаємодоповнювальні методики для динамічних похідних (за кутовими швидкостями), точне відображення кожного коефіцієнта мовою функцій/таблиць JSBSim, скриптований конвеєр, що автоматизує всю розгортку, та цикл верифікації, який зводить CFD із балансуванням, власними модами, даними з аеродинамічної труби та льотними даними. Усе тут спирається на механізми аеродинаміки, функцій/таблиць та балансування, описані в Частинах I і II.

Розділ 49: Конвеєр від CFD до FDM: від OpenFOAM до моделі JSBSim

Аеродинамічна модель JSBSim — це, по суті, таблиця безрозмірних коефіцієнтів як функцій стану потоку (кута атаки, кута ковзання, числа Маха, відхилень керм) плюс кілька динамічних похідних, які відображають нестационарну реакцію на кутові швидкості. Дані з аеродинамічної труби — це золотий стандарт, але для нового чи модифікованого планера кампанія CFD на основі осереднених за Рейнольдсом рівнянь Нав'є-Стокса (RANS) в OpenFOAM — це найдоступніший спосіб отримати повний, узгоджений набір даних ще до того, як з'явиться будь-яке залізо. Цей розділ окреслює всю кампанію: що потрібно JSBSim, як модель розкладається на складові та яка матриця розрахунків CFD її породжує.

49.1 Навіщо будувати аеродинамічну модель з CFD

- **Не потрібно залізо.** Ви можете охарактеризувати планер за CAD-моделлю за місяці до того, як отримаєте час в аеродинамічній трубі чи виконаєте перший політ.
- **Повна спостережуваність.** CFD повертає повне поле тиску та дотичних напружень, тож ви можете розкласти сили за компонентами (крило, хвостове оперення, фюзеляж, гондולי) саме так, як того потребує складова модель JSBSim.
- **Довільні умови.** Великий кут атаки, кут ковзання, відхилення керм, вплив землі та польотні числа Рейнольдса/Маха, які важко чи дорого відтворити в трубі.
- **Відкритість і відтворюваність.** OpenFOAM поширюється за GPL, піддається скриптуванню та запускається на ноутбуці чи на HPC-кластері з ідентичними словниками — природна пара для відкритої, керованої даними філософії JSBSim.

CFD не замінює аеродинамічну трубу чи льотні випробування; вона наповнює модель наперед, тож дорогі дані, які ви все ж збираєте, витрачаються на коригування моделі, а не на створення її з нуля.

49.2 Що потрібно JSBSim: перелік необхідних коефіцієнтів

Кожна величина нижче — це безрозмірний коефіцієнт. У JSBSim кожен із них стає одним чи кількома елементами `<function>`, що сумуються в `<axis>` (див. розділ про аеродинаміку та `FGAerodynamics.cpp:57-69` щодо відображення назви осі в індекс). Правий стовпець називає експеримент CFD, який його дає.

Коефіцієнт	Фізичний зміст	Вісь JSBSim	Експеримент CFD
C_L	Піднімальна сила за α , закрилком, Махом	LIFT (вітрова)	Стаціонарна RANS-розгортка за α
C_D	Поляра опору за α , Махом	DRAG (вітрова)	Стаціонарна RANS-розгортка за α
C_Y	Бічна сила за β	SIDE (вітрова)	Стаціонарна RANS-розгортка за β
C_l	Момент крену за β , δa , δr	ROLL (зв'язана)	Розгортка за β + розрахунки керм
C_m	Момент тангажа за α , δe , Махом	PITCH (зв'язана)	Розгортка за α + розрахунки руля висоти
C_n	Момент рискання за β , δa , δr	YAW (зв'язана)	Розгортка за β + розрахунки керм
C_{Lq} , C_{mq}	Піднімальна сила/тангаж від швидкості тангажа q	LIFT, PITCH	Вимушені коливання тангажа / обертання
C_{lp} , C_{np}	Крен/рискання від швидкості крену p	ROLL, YAW	Вимушені коливання крену / стаціонарний крен

C_{lr}, C_{nr}	Крен/рискання від швидкості рискання γ	ROLL, YAW	Вимушені коливання рискання / стаціонарне рискання
$C_{L\dot{\alpha}}, C_{m\dot{\alpha}}$	Піднімальна сила/тангаж від $\dot{\alpha}$ (запізнення скосу потоку)	LIFT, PITCH	Коливання вертикального переміщення (плунжерні)
$\Delta C_{(\dots)}/\delta$	Прирости від ефективності керм	відповідні осі	Розрахунки з відхиленою геометрією

49.3 Складова (покомпонентна) модель і квазістаціонарне припущення

JSBSim сумує незалежні внески `<function>` у кожен вісь, тож природна модель — це складова: базова крива плюс адитивні прирости. Наприклад, для моменту тангажа:

$$C_m = C_{m0} + C_m(\alpha) + C_{m\delta e} \cdot \delta e + C_{mq} \cdot (\dot{c}/2V) \cdot q + C_{m\dot{\alpha}} \cdot (\dot{c}/2V) \cdot \dot{\alpha}$$

Ця лінійна суперпозиція точна лише тоді, коли внески незалежні. На практиці базові криві табулюються нелінійно (щоб охопити зрив потоку та стисливість), члени керм та кутових швидкостей трактуються як прирости навколо локальної робочої точки, а сильні зв'язки (напр. залежність ефективності керм від α) переносяться як двовимірні таблиці. Квазістаціонарне припущення — що миттєва сила залежить лише від миттєвого стану плюс члени кутових швидкостей першого порядку — це те, що робить достатньою скінченну таблицю. Воно справджується для зведених частот звичайного польоту; воно порушується для швидких маневрів, динамічного зриву та аеропружного флатера, які потребують нестационарних моделей поза межами табличного каркаса JSBSim.

49.4 Матриця плану експерименту

Плануйте кампанію як структуровану розгортку, щоб кожен розрахунок CFD відповідав відомому вузлу таблиці. Практична базова матриця для звичайного літака:

- **Розгортка за α при $\beta=0$:** напр. від -8° до $+20^\circ$ із кроком 2° , дрібнішим поблизу зриву. Дає $C_L(\alpha)$, $C_D(\alpha)$, $C_m(\alpha)$.
- **Розгортка за β при кількох репрезентативних α :** від 0° до $\pm 15^\circ$. Дає $C_Y(\beta)$, $C_l(\beta)$, $C_n(\beta)$ — статичну бічно-коліїну стійкість.
- **Розрахунки керм:** перебудуйте сітку з кожною відхиленою поверхнею (руль висоти, елерон, кермо напряду, закрилок) на 2-3 кутах, щоб отримати лінійні та насичувані прирости.
- **Розгортка за Махом** (якщо потік стисливий/трансзвуковий): повторіть розгортку за α на кількох числах Маха, щоб наповнити вимір таблиці за стисливістю.
- **Динамічні розрахунки:** випадки вимушених коливань чи обертання в одній чи кількох робочих точках, щоб отримати похідні за кутовими швидкостями (наступні розділи).

Мінімальний набір даних для звичайного літака — порядку 20-40 статичних розрахунків плюс кілька динамічних; трансзвуковий, багатомановий набір із повним набором керм може сягати кількох сотень. Скрипуйте це (див. розділ про автоматизацію).

49.5 Знерозмірювання: міст між CFD і JSBSim

Обидва світи говорять коефіцієнтами, але ви маєте узгодити характерні величини, інакше числа не перенесуться. CFD повідомляє $C_F = F / (\frac{1}{2}\rho V^2 S_{ref})$ та $C_M = M / (\frac{1}{2}\rho V^2 S_{ref} l_{ref})$. JSBSim відновлює силу як коефіцієнт $\times aero/qbar \cdot area (= \bar{q} \cdot S)$, а момент — із додатковим множником розмаху чи хорди, де $\bar{q} = \frac{1}{2}\rho V^2$ (FGAerodynamics.cpp:163).

Ключові правила узгодження — помиліться в них, і вашу модель буде непомітно неправильно змасштабовано:

- S_{ref} = площа крила, яку ви задаєте в `<metrics><wingarea>` (властивість `metrics/Sw-sqft`). Використовуйте той самий `Aref` у `forceCoeffs` в OpenFOAM.
- I_{ref} = середня аеродинамічна хорда $\bar{c}$ для тангажа, розмах b для крену/рискання. JSBSim множить тангаж на `metrics/cbarw-ft`, а крен/рискання на `metrics/bw-ft`.
- **Точка зведення моментів** = `CofR` в OpenFOAM має дорівнювати `<location name="AERORP">` в JSBSim, інакше ви маєте перенести моменти (див. розділ про відображення в XML).
- **Знерозмірювачі кутових швидкостей.** JSBSim формує $b/(2V)$ як `aero/bi2vel`, а $\bar{c}/(2V)$ як `aero/ci2vel` (FGAerodynamics.cpp:159-160, 618-619); ваші похідні за кутовими швидкостями з CFD мають використовувати ту саму умову половини хорди/половини розмаху.

49.6 Дорожня карта Частини III

- **Геометрія та побудова сітки** — герметичний STL, область, `snappyHexMesh`, y^+ , кермові поверхні.
- **Статичні коефіцієнти** — `simpleFoam/rhoSimpleFoam`, функційний об'єкт `forceCoeffs`, розгортки за α/β .
- **Динамічні похідні** — обертова система, вимушені коливання, вертикальне переміщення, індикальний метод; отримання C_{mq} , C_{lp} , C_{nr} , ...
- **Відображення в XML** — перетворення набору даних на повний блок `<aerodynamics>`.
- **Автоматизація** — драйвер на Python, що виконує розгортку та видає таблиці JSBSim.
- **Верифікація** — балансування, власні моди та порівняння з трубою/польотом.

Розділ 50: Геометрія та побудова сітки для літаків в OpenFOAM

Розрахункова сітка визначає, чи будуть ваші коефіцієнти фізикою, чи числовим шумом. Цей розділ охоплює підготовку герметичної геометрії, визначення розмірів області, генерацію тілоконформної сітки за допомогою blockMesh + snappyHexMesh, досягнення правильного пристінкового розрізнення для вашої моделі турбулентності та особливу проблему відхилених крмових поверхонь.

50.1 Герметична геометрія та виділення особливостей

Експортуйте планер як тріангульовану поверхню (STL/OBJ) у вигляді єдиної, замкненої оболонки без самоперетинів, з іменованими областями (wing, fuselage, elevator, ...), щоб патчі можна було інтегрувати за силами окремо для покомпонентної складової моделі. Помістіть файл у constant/triSurface/. Виділіть гострі кромки (задні кромки, щілини крмових поверхонь), щоб побудовник сітки прив'язувався до них:

```
# constant/triSurface/ contains aircraft.stl (named solids)
surfaceFeatureExtract      # reads system/surfaceFeatureExtractDict
                           # -> writes aircraft.eMesh (feature edges)
surfaceCheck aircraft.stl  # verify closed & non-degenerate
```

Тримайте геометрію в метрах та узгодженою з характерною площею, яку ви оголосите; OpenFOAM не залежить від одиниць, але ваші Aref/lRef та метрики JSBSim мають відповідати тим самим фізичним розмірам.

50.2 Розрахункова область

Області зовнішньої аеродинаміки мають бути достатньо великими, щоб межа набіжного поля не впливала на розв'язок. Емпіричні правила, відмірювані від літака:

- Вхід вище за потоком: 10-20 середніх хорд (або >5 довжин тіла).
- Вихід нижче за потоком: 20-30 хорд, щоб дати слідів розвинутих.
- Бічне/вертикальне набіжне поле: 10-20 хорд (або 5-10 розмахів).
- Для симетричного випадку ($\beta=0$, без асиметрії крену/рискання) розітніть область по центральній площині патчем symmetry і будуйте сітку лише для половини літака — удвічі менше комірок за того самого розрізнення.

50.3 Фонова сітка: blockMesh

blockMesh будує прямокутну фонову гексаедричну сітку та називає патчі набіжного поля. snappyHexMesh потім вирізає з неї літак. Скететний blockMeshDict:

```
scale 1;                                // STL already in metres
vertices ( (-60 -40 -40) (90 -40 -40) (90 40 -40) (-60 40 -40)
           (-60 -40 40) (90 -40 40) (90 40 40) (-60 40 40) );
blocks ( hex (0 1 2 3 4 5 6 7) (150 80 80) simpleGrading (1 1 1) );
boundary
(
    inlet    { type patch;    faces ((0 4 7 3)); }
    outlet   { type patch;    faces ((1 2 6 5)); }
    farfield { type patch;    faces ((0 1 5 4)(3 7 6 2)(4 5 6 7)); }
    symmetry { type symmetryPlane; faces ((0 3 2 1)); } // y = 0 plane
);
```

50.4 Тілоконформна сітка: snappyHexMesh

snappyHexMesh працює у три фази: зубчасте різання (подрібнення та вилучення комірок усередині тіла), прив'язка (зсув граничних вузлів на STL та особливі кромки) та додавання шарів (вставлення призматичних комірок пограничного шару). Основні параметри словника:

```
castellatedMeshControls
{
    maxLocalCells 2000000; maxGlobalCells 50000000;
    refinementSurfaces
    {
        aircraft { level (5 6);          // min/max surface refinement
                    patchInfo { type wall; } }
    }
    features ( { file "aircraft.eMesh"; level 6; } ); // snap to edges
    refinementRegions
    {
        wakeBox { mode inside; levels ((1e15 3)); } // refine the wake
    }
    locationInMesh (50 5 5);           // a point in the FLUID, not the body
}
snapControls { nSmoothPatch 3; tolerance 2.0; nSolveIter 50; }
addLayersControls
{
    relativeSizes true;
    layers { aircraft { nSurfaceLayers 12; } }
    expansionRatio 1.2;
    finalLayerThickness 0.4;           // fraction of adjacent cell
    minThickness 0.1;
}
```

Запустіть snappyHexMesh -overwrite, потім checkMesh. Вимагайте неортогональності нижче $\sim 65^\circ$, скошеності нижче ~ 4 і щоб запитані шари справді росли (читайте журнал snappyHexMesh: покриття «Layer addition» близько 100% на несучих поверхнях).

50.5 Пристінкове розрізнення та y^+

Висота першої комірки задає пристінкову координату $y^+ = u_\tau y / \nu$, а ваш підхід до турбулентності диктує цільове значення:

- **Пристінково-розрізнена (низькорейнольдсова) k- ω SST:** $y^+ < 1$ на несучих поверхнях, з 30-40 комірками поперек пограничного шару та коефіцієнтом зростання < 1.2 . Потрібна для надійних опору, відриву та зриву.
- **Пристінкові функції** (високорейнольдсові): $30 < y^+ < 300$. Дешевше, прийнятно для піднімальної сили при безвідривному обтіканні та моменту тангажа, але ненадійно поблизу зриву та для розкладу опору.

Оцініть висоту першої комірки за кореляцією поверхневого тертя плоскої пластини: $C_f \approx 0.026/Re_x^{1/7}$, дотичне напруження на стінці $\tau_w = \frac{1}{2}C_f\rho U^2$, динамічна швидкість $u_\tau = \sqrt{(\tau_w/\rho)}$, потім $\Delta y_1 = y^+\nu/u_\tau$. Завжди підтверджуйте досягнуте y^+ за розв'язком (postProcess -func yPlus) і перебудуйте сітку, якщо воно завелике.

50.6 Незалежність від сітки

Запустіть щонайменше три систематично подрібнені сітки (напр. з кількостями комірок у співвідношенні близько 2) за фіксованої умови та підтвердьте, що C_L , C_D , C_m збігаються. Кількісно оцініть це за допомогою індексу збіжності сітки (Roache) та екстраполяції Річардсона; повідомляйте асимптотичне значення, а не значення на найдрібнішій сітці. Опір значно чутливіший до сітки, ніж піднімальна сила, тож домагайтеся збіжності за опором.

50.7 Кермові поверхні та рухома геометрія

Розрахунки ефективності керм та динамічних похідних потребують геометрії у відхиленому чи рухомому стані. Три підходи, у порядку зростання вартості:

- **Окремі статичні сітки** — моделюйте кожне відхилення як власний STL та сітку. Найпростіший і найнадійніший спосіб для приростів від керм; просто перезапустіть розгортку з відхиленою геометрією та візьміть різницю коефіцієнтів.
- **Деформування сітки** (displacementLaplacian / RBF) — деформуйте єдину сітку для малих відхилень чи коливань; уникає перебудови, але погіршує якість комірок при великому русі.
- **Накладені (overset) або ковзні сітки AMI** — тілоконформна сітка компонента рухається крізь фонову сітку. Потрібна для великих обертань (відхилення поверхні на повний хід, гвинти) та для динамічних розрахунків з вимушеними коливаннями.

Розділ 51: Статичні аеродинамічні коефіцієнти з OpenFOAM

Зі збіжною сіткою статичні коефіцієнти отримують із серії стаціонарних RANS-розв'язків, по одному на кожну умову потоку, кожен з яких постоброблюється функційним об'єктом forceCoeffs. Цей розділ уточнює вибір розв'язувача, граничні умови, спосіб задання кута атаки та кута ковзання, точний словник forceCoeffs та спосіб зчитування розгортки.

51.1 Вибір розв'язувача

Режим	Розв'язувач	Примітки
$M < 0.3$ (нестисливий)	simpleFoam	Стаціонарний SIMPLE; стала густина; найшвидший. Більшість робіт для АЗП/БПЛА.
$0.3 \leq M < 0.7$	rhoSimpleFoam	Стаціонарний стисливий; враховує зміну густини.
Трансзвуковий ($M \sim 0.7-1.2$)	rhoSimpleFoam	Стисливий із вловлюванням стрибків; потребує дрібнішої сітки, обережної релаксації.
Потрібна часова точність	pimpleFoam / rhoPimpleFoam	Нестаціонарний; використовується для розрахунків динамічних похідних.

Щодо моделі турбулентності, **k- ω SST** (Menter) — це робоча конячка зовнішньої аеродинаміки: вона добре поводить себе при несприятливих градієнтах тиску та відриві й інтегрується до стінки, коли $y^+ < 1$. Spalart-Allmaras — надійна однорівнянна альтернатива для безвідривного обтікання.

51.2 Граничні умови

Типове нестисливе налаштування (поля в θ):

Патч	U	p	k / ω
inlet	freestreamVelocity	freestreamPressure / zeroGradient	fixedValue (турб. притік)
outlet	freestream / inletOutlet	freestreamPressure	inletOutlet
farfield	freestream	freestreamPressure	inletOutlet
aircraft	noSlip	zeroGradient	kqRWallFunction / omegaWallFunction
symmetry	symmetryPlane	symmetryPlane	symmetryPlane

Умови freestream/freestreamVelocity перемикаються між поведінкою входу та виходу на основі локального потоку, тож те саме набігне поле працює для будь-якого напрямку потоку — зручно для розгортки за α/β .

51.3 Задання кута атаки та кута ковзання

Чистий підхід — тримати сітку нерухомою та **повертати вектор швидкості набіжного потоку**. У зв'язаних осях, де x спрямована назад уздовж фюзеляжу, y — на правий борт, z — угору, набігний потік зі швидкістю V при куті атаки α та куті ковзання β дорівнює:

$$\mathbf{U} = V \cdot (\cos\alpha \cdot \cos\beta, -\sin\beta, \sin\alpha \cdot \cos\beta)$$

```
// 0/include/freestreamConditions (use #include in 0/U, controlDict)
Uinf      68.0;           // m/s
alphaDeg   5.0;
betaDeg    0.0;
// 0/U
internalField uniform (67.74 0 5.93); // = Uinf*(cosA, 0, sinA), beta=0
boundaryField { inlet { type freestreamVelocity;
```

```
freestreamValue $internalField; } ... }
```

Принципово важливо, щоб `liftDir` та `dragDir` у `forceCoeffs` були задані для тих самих α/β , щоб піднімальна сила повідомлялася перпендикулярно до відносного вітру, а опір — уздовж нього. Для розгортки у площині тангажа: `dragDir = (cos $\alpha$ , 0, sin $\alpha$ )`, `liftDir = (-sin $\alpha$ , 0, cos $\alpha$ )`.

51.4 Функційний об'єкт `forceCoeffs`

Додайте це до `system/controlDict` в розділ `functions { }` (синтаксис ESI / openfoam.com; форк openfoam.org майже ідентичний). Він інтегрує сили тиску та в'язкісні сили по іменованих патчах і нормує їх.

```
functions
{
    forceCoeffs1
    {
        type            forceCoeffs;
        libs             ("libforces.so");
        writeControl     timeStep; writeInterval 1;
        patches          (aircraft);           // or (wing fuselage tail ...)
        rho              rhoInf;               // incompressible: name + value
        rhoInf           1.225;                // kg/m^3
        magUInf          68.0;                 // m/s (freestream speed)
        lRef             1.40;                 // mean aerodynamic chord [m]
        Aref             16.17;                // reference (wing) area [m^2]
        // wind axes for alpha = 5 deg, beta = 0:
        liftDir          (-0.0872 0 0.9962);
        dragDir          ( 0.9962 0 0.0872);
        pitchAxis        (0 1 0);
        CofR             (1.07 0 0);           // == JSBSim AERORP, in metres
        // optional (newer versions): which coefficients to write
        coefficients      (Cd Cl CmPitch Cs CmRoll CmYaw);
    }
}
```

Результати потрапляють у `postProcessing/forceCoeffs1/<startTime>/` як `coefficient.dat` (новіше) чи `forceCoeffs.dat` (старіше), по одному рядку на кожен запис, зі стовпцями, що включають `Cd` (опір), `Cl` (піднімальна сила), `Cs` (бічна сила), та коефіцієнти моментів `CmRoll`, `CmPitch`, `CmYaw`. Усі вони нормуються на $\frac{1}{2}\rho \cdot \text{magUInf}^2 \cdot \text{Aref}$ (моменти $\times \text{lRef}$).

Зауваження щодо знаку та осей. Коефіцієнти моментів OpenFOAM беруться відносно `CofR` у системі сітки; ROLL/PITCH/YAW у JSBSim — це моменти у зв'язаних осях відносно AERORP. Задайте `CofR=AERORP` та перевірте знак кожного коефіцієнта на відомому випадку, перш ніж довіряти йому — перевернута вісь чи зсув точки зведення моментів — це найпоширеніша помилка в усьому конвеєрі (детально розглянуто в розділі про відображення в XML).

51.5 Збіжність та осереднення

- Запускайте до стагнації нев'язок (зазвичай від $1e-4$ до $1e-6$) та виходу коефіцієнтів на плато — стежте за `coefficient.dat` наживо, а не лише за нев'язками.
- Поблизу зриву чи для погано обтічних тіл стаціонарний розв'язувач може виходити на граничний цикл; перейдіть на нестаціонарний розв'язувач та осередніть за часом, або осередніть останні кілька сотень ітерацій SIMPLE.
- Використовуйте узгоджену нижню релаксацію та кілька тисяч ітерацій; трансзвукові випадки потребують поступового нарощування числа Куранта / релаксації.

51.6 Виконання розгортки

Клонуйте збіжний базовий випадок для кожної умови, відредагуйте набігний потік та liftDir/dragDir, запустіть і зберіть фінальний рядок коефіцієнтів. Результат — це сировина для таблиць JSBSim:

- **Розгортка за α** $\rightarrow C_L(\alpha), C_D(\alpha), C_m(\alpha)$. Нахил C_L за α має бути близько $2\pi \cdot AR / (AR + 2)$ на радіан; нахил C_m за α має бути від'ємним для статичної стійкості.
- **Розгортка за β** $\rightarrow C_Y(\beta), C_l(\beta)$ (ефект поперечного V, бажано <0), $C_n(\beta)$ (флюгерна стійкість, бажано >0).
- **Розрахунки керм** $\rightarrow \Delta C$ на градус руля висоти/елерона/керма напряду/закрилка.
- **Перевірка поляри опору** $\rightarrow$ підженіть $C_D = C_{D0} + C_L^2 / (\pi e AR)$, щоб перевірити осмисленість опору при нульовій піднімальній силі та коефіцієнта Освальда.

Розділ 52: Динамічні похідні стійкості з OpenFOAM

Статичні розгортки не охоплюють реакцію планера на кутові швидкості та на швидкість зміни кута набігання. Ці динамічні похідні (похідні демпфування) — C_{mq} , C_{lp} , C_{nr} , C_{lr} , C_{np} , $C_{m\dot{\alpha}}$, ... — задають короткоперіодичну, голландського кроку, кренову та спіральну поведінку. Їх важче отримати з CFD, бо вони потребують або обертової системи відліку, або справді часо-точного руху. У широкому вжитку три методи.

52.1 Похідні та їхній зміст

Похідна	Зв'язує	Знак для стійкого звичайного літака
C_{mq}	момент тангажа $\leftarrow$ швидкість тангажа q	< 0 (демпфування тангажа)
$C_{m\dot{\alpha}}$	момент тангажа $\leftarrow \dot{\alpha}$ (запізнення скосу потоку)	< 0
C_{Lq}	піднімальна сила $\leftarrow$ швидкість тангажа q	> 0
C_{lp}	момент крену $\leftarrow$ швидкість крену p	< 0 (демпфування крену)
C_{nr}	момент рискання $\leftarrow$ швидкість рискання r	< 0 (демпфування рискання)
C_{lr}	момент крену $\leftarrow$ швидкість рискання r	> 0
C_{np}	момент рискання $\leftarrow$ швидкість крену p	< 0 (несприятлива)

52.2 Безрозмірна кутова швидкість та зведена частота

Похідні за кутовими швидкостями визначаються відносно безрозмірних кутових швидкостей: $\hat{q} = q \cdot \bar{c} / (2V)$ для тангажа, та $\hat{p}, \hat{r} = p, r \cdot b / (2V)$ для крену/рискання — це точно `ci2vel` та `bi2vel` у JSBSim. Для коливальних випробувань ключовий параметр подібності — зведена частота:

$$k = \omega \cdot \bar{c} / (2V)$$

Обирайте k малим (~ 0.01 - 0.1) та амплітуду малою (1 - 2°), щоб отримані похідні були квазістаціонарними значеннями, яких очікують таблиці JSBSim; більше k зондує справді нестаціонарну аеродинаміку.

52.3 Метод 1: стаціонарне обертання в неінерціальній системі

Чисті обертові похідні (C_{mq} без частини $\dot{\alpha}$, C_{lp} , C_{nr}) можна отримати зі стаціонарного розв'язку в обертовій системі відліку, уникаючи будь-якого руху сітки. Літак перебуває в системі, що обертається зі сталою кутовою швидкістю Ω ; для випадку швидкості тангажа потік слідує по дузі кола, а стаціонарний момент дає C_{mq} безпосередньо. В OpenFOAM це налаштовується через MRF-зону або джерело обертання `fvOptions`:

```
// constant/fvOptions (impose a body rate omega about the CofR)
rotationSource
{
    type            rotorDiskSource; // or a coded/MRF source
    // For derivative work an SRF (single rotating frame) solver
    // (SRFSimpleFoam) with SRFProperties is the classic route:
}
```

```
// constant/SRFProperties
SRFModel    rpm;
axis        (0 1 0);           // pitch about y
rpm         rpm_value;         // omega = q (rad/s) -> rpm
```

Запустіть дві-три кутові швидкості, що охоплюють нуль, побудуйте графік коефіцієнта моменту відносно безрозмірної кутової швидкості та візьміть нахил:

$$C_{mq} = \partial C_m / \partial \dot{q}, \quad \dot{q} = q \cdot \bar{c} / (2V)$$

Це дешево (стаціонарно) та чисто для обертової частини, але не охоплює внесок $\dot{\alpha}$ (запізнення скошу потоку), який потребує руху.

52.4 Метод 2: вимушені коливання

Коливайте літак синусоїдально навколо CofR за допомогою нестационарного розв'язувача (pimpleFoam + динамічна сітка) і вилучайте похідні з фази відгуку моменту. Канонічний примітив OpenFOAM — це oscillatingRotatingMotion (див. навчальний приклад pimpleFoam/RAS/wingMotion):

```
// constant/dynamicMeshDict
dynamicFvMesh    dynamicMotionSolverFvMesh;
motionSolver     solidBody;
solidBodyMotionFunction oscillatingRotatingMotion;
oscillatingRotatingMotionCoeffs
{
    origin        (1.07 0 0);      // CofR == AERORP
    axis          (0 1 0);          // pitch
    omega         6.2832;           // angular FREQUENCY of oscillation [rad/s]
    amplitude     (0 2 0);          // degrees about each axis (here 2 deg pitch)
}
```

Для чистого коливання тангажа навколо CG за нерухомого набіжного потоку кут атаки та швидкість тангажа жорстко зчеплені ($q = \dot{\alpha}$), тож квазістаціонарний момент дорівнює:

$$C_m(t) = C_{m0} + C_{m\alpha} \cdot \alpha(t) + (C_{mq} + C_{m\dot{\alpha}}) \cdot (\bar{c}/2V) \cdot \dot{\alpha}(t)$$

При $\alpha(t) = \alpha_0 + \Delta\alpha \cdot \sin(\omega t)$ синфазна (синусна) компонента C_m дає статичний нахил $C_{m\alpha}$, а протифазна (косинусна) компонента дає сумарне демпфування. Вилучаючи їх як перші коефіцієнти Фур'є за один період T :

$$C_{m\alpha} = (2/(\Delta\alpha T)) \int C_m(t) \sin(\omega t) dt$$

$$C_{mq} + C_{m\dot{\alpha}} = (2V/(\bar{c} \cdot \omega \cdot \Delta\alpha T)) \int C_m(t) \cos(\omega t) dt$$

Рівнозначно, побудуйте графік C_m відносно α за цикл: площа та напрямок обходу петлі кодують демпфування. Петля, що обходиться за годинниковою стрілкою (енергія відбирається), означає додатне демпфування ($C_{mq} + C_{m\dot{\alpha}} < 0$). Відкиньте перші 1-2 цикли як пусковий перехідний процес і осередніть за кількома чистими циклами.

52.5 Метод 3: вертикальне переміщення та індикальний відгук

Вимушений тангаж дає лише суму $C_{mq} + C_{m\dot{\alpha}}$. Щоб їх розділити, запустіть **коливання вертикального переміщення (плунжерні)**: переміщуйте літак вертикально так, щоб змінювався кут атаки (даючи $\dot{\alpha}$) за нульової швидкості тангажа ($q = 0$). Це виокремлює $C_{m\dot{\alpha}}$; відніміть її від суми вимушеного тангажа, щоб відновити C_{mq} :

$$C_{mq} = (C_{mq} + C_{m\dot{\alpha}})_{pitch} - (C_{m\dot{\alpha}})_{plunge}$$

Індикальний (ступінчастий) метод — це третій шлях: задайте стрибок α чи q та запишіть перехідне наростання моменту; похідні впливають з індикальних функцій відгуку (теорія Тобака/Вагнера). Він найзагальніший, але найвибагливіший до постобробки.

52.6 Бічно-коліїні похідні

Той самий механізм застосовується до крену та рискання. Вимушені коливання крену навколо зв'язаної осі x дають C_{lp} (та C_{nr}); вимушені коливання рискання навколо z дають C_{nr} (та C_{lp}). Знерозмірювач перемикається з $\bar{c}/(2V)$ на $b/(2V)$ (`bi2vel` у JSBSim). Стаціонарне обертання в неінерціальній системі знову є дешевим шляхом для чистих частин за кутовими швидкостями.

52.7 Порівняння методів

Метод	Дає	Вартість	Застереження
Стаціонарна обертова система	чисті C_{mq} , C_{lp} , C_{nr}	низька (стаціонарна)	пропускає запізнення $\dot{\alpha}$
Вимушені коливання	$C_{mq} + C_{m\dot{\alpha}}$ тощо	висока (нестационарна + рухома сітка)	потребує розділення Фур'є
Вертикальне переміщення	лише $C_{m\dot{\alpha}}$	висока	у парі з вимушеним тангажем
Індикальний / ступінчастий	усі, найзагальніший	висока	складна постобробка

Прагматичний рецепт: використовуйте стаціонарну обертову систему для основної маси обертових похідних, додайте один випадок вимушеного тангажа + один випадок вертикального переміщення, щоб розрізнити C_{mq} та $C_{m\dot{\alpha}}$, та вдавайтеся до оцінок DATCOM/AVL для будь-якої похідної, яку бюджет не може охопити.

Розділ 53: Зведення даних CFD у XML літака JSBSim

Саме тут кампанія окупається: перетворення набору даних CFD на повний блок `<aerodynamics>`. Добра новина в тому, що каркас JSBSim відображається майже один-до-одного на безрозмірні коефіцієнти — значення похідної потрапляє прямо в `<value>`, а крива коефіцієнта потрапляє прямо в `<table>`.

53.1 Вибір системи осей та точки зведення

Узгодьте осі JSBSim з тим, що природно повідомляє CFD. Сили з `forceCoeffs` — це піднімальна/опір/бічна (вітрові осі); моменти — це крен/тангаж/рискання у зв'язаних осях. Отже:

- Сили → `<axis name="LIFT">`, "DRAG", "SIDE" (вітрова система — стандартна, коли використовуються LIFT/DRAG; FGAerodynamics.cpp:453-460).
- Моменти → `<axis name="ROLL">`, "PITCH", "YAW" (зв'язана система за замовчуванням; FGAerodynamics.cpp:450-452).
- Задайте `CoFR` в OpenFOAM рівним `<metrics><location name="AERORP">`, щоб точка зведення моментів збігалася; інакше перенесіть моменти (нижче).

53.2 Головна таблиця відображення

Кожен коефіцієнт CFD стає `<function>` = (знерозмірювачі) × (коефіцієнт). Знерозмірювачі — це властивості JSBSim; коефіцієнт — це ваша `<table>` чи `<value>` з CFD.

Коефіцієнт	Вісь	Функція JSBSim = добуток
$C_L(\alpha)$	LIFT	<code>aero/qbar-area</code> × таблиця $C_L(\alpha)$
$C_D(\alpha)$	DRAG	<code>aero/qbar-area</code> × таблиця $C_D(\alpha)$
$C_Y(\beta)$	SIDE	<code>aero/qbar-area</code> × таблиця $C_Y(\beta)$
$C_l(\beta)$	ROLL	<code>aero/qbar-area</code> × <code>bw-ft</code> × таблиця
$C_m(\alpha)$	PITCH	<code>aero/qbar-area</code> × <code>sbarw-ft</code> × таблиця
$C_n(\beta)$	YAW	<code>aero/qbar-area</code> × <code>bw-ft</code> × таблиця
C_{mq}	PITCH	<code>aero/qbar-area</code> × <code>sbarw-ft</code> × <code>ci2vel</code> × <code>q-aero-rad_sec</code> × значення
C_{lp}	ROLL	<code>aero/qbar-area</code> × <code>bw-ft</code> × <code>bi2vel</code> × <code>p-aero-rad_sec</code> × значення
C_{nr}	YAW	<code>aero/qbar-area</code> × <code>bw-ft</code> × <code>bi2vel</code> × <code>r-aero-rad_sec</code> × значення

53.3 Ключове розуміння: значення і є похідною

Оскільки JSBSim будує безрозмірну кутову швидкість внутрішньо (`b/2V` та `ḃ/2V` через `bi2vel/ci2vel`, FGAerodynamics.cpp:159-160) і повертає розмірність через $\dot{q} \cdot S$ та розмах/хорду, стала, яку ви розміщуєте в `<value>` похідної за кутовою швидкістю, — це точно підручникова безрозмірна похідна. Модель c172 робить це наочним — її функція демпфування крену — це буквально $\dot{q} S \cdot b \cdot (b/2V) \cdot p \cdot C_{lp}$ з $C_{lp} = -0.47$ (aircraft/c172x/c172x.xml:1025-1034):

```
<function name="aero/coefficient/Clp">
  <description>Roll moment due to roll rate (roll damping)</description>
  <product>
    <property>aero/qbar-area</property>      <!-- qbar * Sw      -->
    <property>metrics/bw-ft</property>        <!-- span b      -->
    <property>aero/bi2vel</property>          <!-- b/(2V)       -->
    <property>velocities/p-aero-rad_sec</property> <!-- roll rate p -->
```

```

    <value>-0.47</value>                                <!-- Clp from CFD      -->
  </product>
</function>

```

Тож ваші отримані з OpenFOAM C_{lp} , C_{mq} , C_{nr} , ... йдуть прямо в гніздо `<value>`. Жодного додаткового масштабування.

53.4 Статичні криві як таблиці

Табулюйте кожен базовий коефіцієнт відносно його первинної змінної, додаючи виміри стовпця/таблиці для зв'язків (напр. β , Махом, закрилком). Крива піднімальної сили з розгортки за α , помножена до сили:

```

<axis name="LIFT">
  <function name="aero/coefficient/CL_basic">
    <description>Lift from CFD alpha-sweep</description>
    <product>
      <property>aero/qbar-area</property>
      <table>
        <independentVar lookup="row">aero/alpha-rad</independentVar>
        <tableData>
          -0.140  -0.62
          -0.087  -0.18
          0.000   0.27
          0.087   0.95
          0.175   1.42
          0.262   1.61  <!-- approaching stall -->
          0.300   1.45  <!-- post-stall drop  -->
        </tableData>
      </table>
    </product>
  </function>
</axis>

```

Для набору даних із залежністю від Маха додайте `<independentVar lookup="column">velocities/mach</independentVar>` та надайте матрицю; для $\alpha/\beta/\text{Маха}$ додайте третій вимір `lookup="table"` з блоками `<tableData breakPoint="...">`.

53.5 Прирости від керм

Взяття різниці між розрахунком із відхиленою геометрією та чистим базовим розрахунком дає ΔC на кожну поверхню; табулюйте відносно відхилення, щоб охопити нелінійність/насичення:

```

<function name="aero/coefficient/Cm_de">
  <description>Pitch moment due to elevator (CFD increments)</description>
  <product>
    <property>aero/qbar-area</property>
    <property>metrics/cbarw-ft</property>
    <table>
      <independentVar lookup="row">fcs/elevator-pos-rad</independentVar>
      <tableData>
        -0.35  0.38
         0.00  0.00
         0.35 -0.41
      </tableData>
    </table>
  </product>
</function>

```

53.6 Умови знаків та осей: підводні камені

Більшість невдач тут — це облік, а не фізика. Узгодьте ці моменти, перш ніж довіряти моделі:

- **Знак опору.** C_D з CFD додатний (сила, що протидіє вітру). Вісь DRAG у JSBSim уже спрямована вздовж відносного вітру, тож додатне значення — це правильний опір — не змінюйте знак.
- **Напрямки зв'язаних осей.** Підтвердьте, що ваша зв'язана система CFD (напрямки x , y , z) відповідає умові структурної-до-зв'язаної JSBSim; перевернута у чи z непомітно інвертує крен/рискання чи тангаж.
- **Перенесення точки зведення моментів.** Якщо $C_{ofR} \neq AERORP$, зсуньте момент тангажа. За нормальної сили C_N , осьової сили C_A та $AERORP$ на відстані Δx назад і Δz вище за C_{ofR} :

$$C_{m,AERORP} = C_{m,CofR} + (C_N \cdot \Delta x - C_A \cdot \Delta z) / \bar{c}$$

- **Радіани проти градусів.** Незалежні змінні таблиць використовують власну одиницю властивості — `aero/alpha-rad` у радіанах, `aero/alpha-deg` у градусах, `fcs/elevator-pos-rad` у радіанах. Узгодьте вузли ваших CFD-таблиць із властивістю, на яку ви посилаєтеся.
- **Узгодженість характерних площі/довжини.** S , b , $\bar{c}$ в `<metrics>` мають дорівнювати $Aref/lRef$, що використовуються в `forceCoeffs`.

53.7 Розв'язаний приклад: повний набір осей, отриманий з CFD

Збираючи все до купи — компактний, але повний скелет `<aerodynamics>`, наповнений цілковито з CFD (базові криві скорочено):

```
<aerodynamics>
  <axis name="DRAG">
    <function name="aero/coefficient/CD0">          <!-- CD vs alpha -->
      <product><property>aero/qbar-area</property>
        <table><independentVar lookup="row">aero/alpha-rad</independentVar>
          <tableData> -0.09 0.025
                      0.00 0.022
                      0.17 0.060
                      0.26 0.140 </tableData></table>
        </product>
      </function>
    </axis>
    <axis name="LIFT">
      <function name="aero/coefficient/CLa">          <!-- CL vs alpha -->
        <product><property>aero/qbar-area</property>
          <table><independentVar lookup="row">aero/alpha-rad</independentVar>
            <tableData> -0.09 -0.18
                        0.00 0.27
                        0.17 1.42
                        0.26 1.61 </tableData></table>
          </product>
        </function>
        <function name="aero/coefficient/CLq">          <!-- lift due to q -->
          <product><property>aero/qbar-area</property>
            <property>aero/ci2vel</property>
            <property>velocities/q-aero-rad_sec</property>
            <value>3.9</value></product>
          </function>
        </axis>
        <axis name="SIDE">
          <function name="aero/coefficient/CYb">          <!-- side force vs beta -->
            <product><property>aero/qbar-area</property>
              <property>aero/beta-rad</property><value>-0.31</value></product>
            </function>
          </axis>
          <axis name="ROLL">
            <function name="aero/coefficient/Clb">          <!-- dihedral effect -->
              <product><property>aero/qbar-area</property><property>metrics/bw-ft</property>
                <property>aero/beta-rad</property><value>-0.089</value></product>
```

```

</function>
<function name="aero/coefficient/Clp">          <!-- roll damping -->
  <product><property>aero/qbar-area</property><property>metrics/bw-ft</property>
    <property>aero/bi2vel</property>
    <property>velocities/p-aero-rad_sec</property>
    <value>-0.47</value></product>
</function>
</axis>
<axis name="PITCH">
  <function name="aero/coefficient/Cma">          <!-- pitch stiffness -->
    <product><property>aero/qbar-area</property><property>metrics/cbarw-ft</property>
      <property>aero/alpha-rad</property><value>-1.8</value></product>
  </function>
  <function name="aero/coefficient/Cmq">          <!-- pitch damping -->
    <product><property>aero/qbar-area</property><property>metrics/cbarw-ft</property>
      <property>aero/ci2vel</property>
      <property>velocities/q-aero-rad_sec</property>
      <value>-12.4</value></product>
  </function>
</axis>
<axis name="YAW">
  <function name="aero/coefficient/Cnb">          <!-- weathercock -->
    <product><property>aero/qbar-area</property><property>metrics/bw-ft</property>
      <property>aero/beta-rad</property><value>0.065</value></product>
  </function>
  <function name="aero/coefficient/Cnr">          <!-- yaw damping -->
    <product><property>aero/qbar-area</property><property>metrics/bw-ft</property>
      <property>aero/bi2vel</property>
      <property>velocities/r-aero-rad_sec</property>
      <value>-0.15</value></product>
  </function>
</axis>
</aerodynamics>

```

Кожне <value> вище — це безрозмірна похідна стійкості прямо з CFD; кожна <table> — це розгортка CFD. Додавайте прирости від керм, вплив землі та виміри за Махом, як того вимагають дані.

Розділ 54: Автоматизація конвеєра від OpenFOAM до JSBSim

Реальна кампанія — це десятки-сотні розрахунків; робити це вручну — загрожує помилками та невідтворювано. Цей розділ накидає скриптований конвеєр: створіть шаблон базового випадку за матрицею умов, запустить розв'язувач, розберіть coefficient.dat та видайте XML `<function>`/`<table>` для JSBSim.

54.1 Шаблонування випадків за матрицею умов

Тримайте один збіжний baseCase/ та клонуйте його для кожної умови, переписуючи лише набігний потік та liftDir/dragDir. Мінімальний драйвер на Python:

```
import math, shutil, subprocess, pathlib, re

def wind_dirs(alpha_deg, beta_deg=0.0):
    a, b = math.radians(alpha_deg), math.radians(beta_deg)
    drag = (math.cos(a)*math.cos(b), -math.sin(b), math.sin(a)*math.cos(b))
    lift = (-math.sin(a), 0.0, math.cos(a))
    return drag, lift

def make_case(alpha, Uinf=68.0, base="baseCase"):
    case = pathlib.Path(f"run_a{alpha:+05.1f}")
    if case.exists(): shutil.rmtree(case)
    shutil.copypath(base, case)
    drag, lift = wind_dirs(alpha)
    U = (Uinf*math.cos(math.radians(alpha)), 0.0,
         Uinf*math.sin(math.radians(alpha)))
    # patch 0/U internalField and forceCoeffs liftDir/dragDir
    sub(case/"0"/"U", r"internalField\s+uniform \([^\)]*\)",
        f"internalField    uniform ({U[0]:.4f} {U[1]:.4f} {U[2]:.4f})")
    fc = case/"system"/"controlDict"
    sub(fc, r"liftDir\s+([^\)]*\)", f"liftDir ({lift[0]:.4f} {lift[1]:.4f} {lift[2]:.4f})")
    sub(fc, r"dragDir\s+([^\)]*\)", f"dragDir ({drag[0]:.4f} {drag[1]:.4f} {drag[2]:.4f})")
    return case

def sub(path, pattern, repl):
    t = path.read_text()
    path.write_text(re.sub(pattern, repl, t))

for alpha in range(-8, 22, 2):
    case = make_case(alpha)
    subprocess.run(["simpleFoam", "-case", str(case)], check=True)
```

54.2 Розбір коефіцієнтів

Зчитуйте останній рядок coefficient.dat (пропускаючи заголовок #) для кожного випадку:

```
import numpy as np, glob, os

def last_coeffs(case):
    f = sorted(glob.glob(f"{case}/postProcessing/forceCoeffs1*/coefficient.dat"))[-1]
    rows = [l for l in open(f) if not l.startswith("#")]
    cols = np.array(rows[-1].split(), dtype=float)
    # column order (ESI): time Cd Cs Cl CmRoll CmPitch CmYaw ...
    return dict(Cd=cols[1], Cs=cols[2], Cl=cols[3],
                Cl_roll=cols[4], Cm=cols[5], Cn=cols[6])

data = {}
for case in sorted(glob.glob("run_a*")):
    alpha = float(case.split("_a")[1])
    data[alpha] = last_coeffs(case)
```

54.3 Видача таблиць JSBSim

Нарешті, відформатуйте розібрану розгортку як `<function>` JSBSim з `<table>`. Зверніть увагу на перетворення вузлів за α у радіани, щоб відповідати `aero/alpha-rad`:

```
def emit_lift(data):
    rows = "\n".join(f"      {math.radians(a):8.4f} {data[a]['CL']:8.4f}"
                     for a in sorted(data))
    return f"""<function name="aero/coefficient/CL_cfd">
<description>Lift from OpenFOAM alpha-sweep</description>
<product>
  <property>aero/qbar-area</property>
  <table>
    <independentVar lookup="row">aero/alpha-rad</independentVar>
    <tableData>
{rows}
    </tableData>
  </table>
</product>
</function>"""

print(emit_lift(data))  # paste into <axis name="LIFT"> ... </axis>
```

54.4 Інструментарій та відтворюваність

- **PyFoam, foamlib** та **openfoamparser** надійно читають/записують словники OpenFOAM та файли постобробки — віддавайте перевагу їм перед регулярними виразами для робочих конвеєрів.
- **pandas/numpy** для зберігання розгортки, підгонки поляри (C_{D0} , число Освальда e) та оцінки нахилів ($C_{L\alpha}$, $C_{m\alpha}$).
- **Модуль JSBSim для Python** (`import jsbsim`) дозволяє тому самому скрипту збалансувати та випробувати згенеровану модель для негайної верифікації.
- Тримайте `Allrun/Allclean`, зафіксуйте версію OpenFOAM та архівуйте журнали і вивід `checkMesh`, щоб набір даних можна було перевірити та перезапустити.

Розділ 55: Верифікація: замикання циклу CFD-JSBSim-політ

Модель, що завантажується без помилок, — це не валідована модель. Останній крок — підтвердити, що літак JSBSim відтворює фізику CFD, осмислено поводить себе при балансуванні та у своїх динамічних модах і — там, де дані існують, — узгоджується з аеродинамічною трубою та польотом. Сприймайте це як цикл: кожна розбіжність вказує назад на конкретну таблицю, знак чи точку зведення.

55.1 Статичні перевірки: чи балансується там, де каже CFD

- Збалансуйте модель (FGTrim, поздовжній режим) та підтвердьте, що балансувальні α і руль висоти є фізичними та відповідають робочій точці CFD.
- Відновіть $C_m(\alpha)$ з JSBSim, розгортаючи α за фіксованих керм та зчитуючи `aero/coefficient/*` чи `moments/m-aero-lbsft`; нахил має відповідати $C_{m\alpha}$ з CFD.
- Знайдіть **нейтральну точку** (де $dC_m/dC_L = 0$) та підтвердьте, що **статичний запас** (НТ мінус CG, у % MAC) є додатним і узгодженим зі значенням, отриманим із CFD.

55.2 Динамічні перевірки: лінеаризуйте та порівняйте моди

Збалансуйте, застосуйте малі збурення (чи скористайтесь утилітою лінеаризації) та вилучіть власні значення, потім порівняйте з аналітичними модами, які прогнозують похідні (див. розділ про власні моди):

- **Короткоперіодична** частота/демпфування, зумовлені $C_{m\alpha}$ та $C_{mq} + C_{m\dot{\alpha}}$ — пряма перевірка похідної демпфування тангажа, яку ви вилучили.
- **Фуґоїдна** — низька частота, слабко демпфована; чутлива до опору та піднімальної сили, отже, до статичної поляри.
- **Голландський крок** від $C_{n\beta}$, C_{nr} , $C_{l\beta}$; **аперіодичний крен** від C_{lp} ; **спіраль** від $C_{l\beta}$, C_{nr} , C_{lr} , $C_{n\beta}$.
- Мода, що є нестійкою, коли не мала б бути, або частота, хибна на порядок, майже завжди простежується до хибного знаку чи відсутньої похідної за кутовою швидкістю.

55.3 Порівняння з незалежними даними

Ранжуйте свою впевненість: льотні випробування > аеродинамічна труба > високоточна CFD > панельні методи/VLM (AVL, XFLR5) > емпіричні (DATCOM). Використовуйте дешевші методи, щоб обмежити CFD, та дорогі дані, щоб її скоригувати:

- Перехресно перевірте $C_{L\alpha}$, $C_{m\alpha}$, $C_{l\beta}$, C_{nr} , ... з оцінками AVL (вихорова решітка) та USAF DATCOM — вони мають збігатися за знаком та грубою величиною.
- Там, де існують дані з аеродинамічної труби чи польоту, налаштовуйте отримані з CFD таблиці так, щоб вони збігалися (спершу поправки за Рейнольдсом та станом балансування).
- Поєднуйте джерела явно: напр. CFD для нелінійної піднімальної сили на великих α , AVL для лінійних похідних, DATCOM для похідної, яку не охопив жоден розрахунок. Документуйте походження кожного числа.

55.4 Застереження щодо екстраполяції

- **Число Рейнольдса.** Запускайте CFD при польотному Re ; піддослідне (зменшене) Re зсуває C_{D0} , максимальну піднімальну силу та зривний α .
- **Мах.** Нестисливі коефіцієнти недійсні за $M \sim 0.3-0.5$; додайте вимір таблиці за Махом для швидких літаків.
- **За межами даних.** JSBSim продовжує таблиці, утримуючи кінцеве значення сталим (без екстраполяції); забезпечте, щоб ваші таблиці охоплювали всю передбачену область, особливо зазривний режим та великий кут ковзання.
- **Лише абсолютно тверде тіло.** CFD на CAD ігнорує аеропружну деформацію та нестационарні/відривні ефекти поза межами квазістационарної моделі.

55.5 Цикл ітерацій

Верифікація рідко буває за один прохід. Здоровий робочий процес такий: побудувати таблиці з CFD → збалансувати → перевірити моди → порівняти з еталонними даними → виявити найгіршу розбіжність → додати чи скоригувати відповідальний розрахунок CFD чи таблицю → повторити. Оскільки кожен коефіцієнт — це ізольована, спостережувана `<function>` у дереві властивостей, JSBSim робить цей цикл швидким: ви можете спостерігати за кожним внеском наживо та точно вказати, який саме член хибний.

Польотна модель ніколи не буває завершеною — лише поступово менш хибною. CFD дає вам правдоподібну першу модель; балансування, власні моди та реальні дані підказують, куди витратити наступний розрахунок.

Частина IV

Побудова високодостовірної моделі динаміки польоту (FDM): дані, підсистеми та налаштування

Частини I-III дають вам рушій, математику та спосіб генерувати аеродинамічні дані. Однак достовірність криється в деталях: звідки беруться дані та як ви їх поєднуєте, як будуються й налаштовуються підсистеми маси, силової установки, шасі, керування польотом і датчиків, як відтворюються звалювання та штопор, як ви пишете сценарії випробувань і звіряєте результат із опублікованими льотними та пілотажними характеристиками. Частина IV — це практична половина посібника, дистиляція вихідного коду JSBSim, довідкового керівництва, вікі FlightGear та форумів розробників у конкретний рецепт планера, який упізнав би льотчик-випробувач. Кожен розділ про підсистему прив'язаний до вихідного коду, тож ви точно знаєте, який елемент XML керує яким рядком коду.

Розділ 56: Джерела аеродинамічних даних і сходинки достовірності

Модель JSBSim настільки реалістична, наскільки реалістичні числа в її таблицях. Перш ніж щось налаштовувати, ви маєте вирішити, звідки беруться коефіцієнти, бо саме це задає стелю достовірності (fidelity). Методи утворюють сходи: кожна сходинка коштує більше і повертає більше правди. Хороша модель зазвичай поєднує сходинки — дешеві методи для більшої частини діапазону, дорогі — там, де дешеві методи не працюють.

56.1 Сходинки достовірності

Метод	Достовірність	Зусилля	Найкраще підходить для
Aeromatic / Aeromatic++	низька	хвилини	перша літаюча модель зі специфікацій рівня РОН
USAF Digital DATCOM	низька-середня	години	напівемпіричні похідні для звичайних компонувань; експорт XML
Вихрова решітка (AVL, XFLR5)	середня	години	лінійні похідні, ефективність керування, нейтральна точка (приєднана течія)
Панельний + в'язкий (VSPAero)	середня	години-дні	поляри та похідні на основі геометрії з моделі OpenVSP
RANS CFD (OpenFOAM)	середня-висока	дні-тижні	нелінійні великі α , стисливість, опір (див. Частина III)
Аеродинамічна труба	висока	тижні+	надійні статичні дані + дані вимушених коливань на реальній моделі
Льотні випробування (system ID)	найвища	програма	істина; використовується для корекції всього вищезазначеного

56.2 Зчитування даних із РОН або сертифіката типу

Керівництво з льотної експлуатації (РОН), документ даних сертифіката типу та виробничий тривид — це найдешевші реальні дані, які ви будь-коли отримаєте. Вони фіксують геометрію (розмах, площу, хорду, довжини), маси та діапазон CG, а також опорні точки характеристик, до яких можна налаштовуватися: швидкості звалювання (чиста конфігурація та з закрилками), V_x/V_y і швидкопідйомність, крейсерську TAS і витрату палива, практичну стелю, граничну швидкість і дистанції зльоту/посадки. Кожна з них — це обмеження, яке має відтворювати готова FDM.

56.3 Digital DATCOM

USAF Stability and Control Digital DATCOM — це напівемпіричний код: з геометричного опису він повертає поздовжні коефіцієнти C_D , C_L , C_m , C_n , C_A та ключові похідні $dC_L/d\alpha$, $dC_m/d\alpha$, $dC_Y/d\beta$, $dC_n/d\beta$, $dC_l/d\beta$. Сучасні збірки експортують таблицю даних XML, яку безпосередньо споживають JSBSim і FlightGear, а Aerospace Toolbox у MATLAB може імпортувати її вивід. Він швидкий та ідеальний для звичайних компонувань, але це інтерполяція по базі даних результатів з аеродинамічної труби, тож він найслабший саме там, де планери стають цікавими: великі α , незвичні форми в плані, сильна інтерференція.

56.4 Методи вихрової решітки та панельні методи

AVL Марка Дрели та керований графічним інтерфейсом **XFLR5** розв'язують задачу вихрової решітки за секунди й дають чудові лінійні похідні, ефективність керування та нейтральну точку для приєднаної течії — ідеально для більшої частини крейсерського діапазону та для перехресної перевірки CFD. **VSPAero**, прив'язаний до геометрії OpenVSP, додає активний диск і нарощування в'язкого опору й має задокументований спільнотою робочий процес для створення аеродинамічних таблиць JSBSim. Їхня спільна сліпа зона — це відривання течії: жоден з них не моделює звалювання чи зазвалювальний режим, тож вище лінійного діапазону їх необхідно доповнювати даними CFD, труби чи емпіричними даними.

56.5 Аеродинамічна труба та льотні випробування

Дані з аеродинамічної труби — статичні розгортки плюс запуски вимушених коливань для похідних демпфування — це класичне джерело високої достовірності. Льотні випробування — це найвищий авторитет: через ідентифікацію параметрів (підгонку похідних до виміряних відгуків на здвоєний імпульс/розгортку) ви видобуваєте реальні похідні стійкості й керованості та використовуєте їх для корекції моделі. Невизначеність цих похідних можна обмежити розгортками Монте-Карло, щоб показати, що модель залишається коректною в усьому правдоподібному діапазоні.

56.6 Поєднання джерел і подібні літаки

Реальні моделі гібридні: лінійне ядро з AVL/DATCOM, нелінійні піднімальна сила і опір на великих α із CFD чи даних труби, похідні демпфування з вимушених коливань і фінальне калібрування за льотними випробуваннями. Коли для вашого літака немає даних, запозичте їх у подібного — зі схожою конфігурацією, видовженням і призначенням — і масштабуйте за геометрією; це набагато краще за здогад і є стандартною практикою спільноти JSBSim. Хоч би якою була суміш, фіксуйте походження кожного числа; саме цей слід аудиту дозволяє пізніше налагодити модель, що поводить неправильно.

Колективна мудрість спільноти зафіксована в документах на кшталт "A Journal for the Creation and Refinement of a JSBSim Aircraft Flight Model" — прочитайте один із них, перш ніж братися за свій перший планер.

Розділ 57: Aeromatic: первинне створення літаючої моделі

Aeromatic — це найшвидший шлях від чистого аркуша до літака, який літає. Він запитує мінімальний набір специфікацій і генерує правдоподібні файли конфігурації JSBSim, використовуючи спрощувальні припущення. Існує два різновиди: оригінальний вебінструмент (PHP) на jsbsim.sourceforge.net та потужніша програма командного рядка на C++ `aeromatic++`, що постачається в `utils/aeromatic++/`. Для будь-чого серйознішого за іграшку використовуйте версію на C++.

57.1 Що ви йому подаєте

Що кращі ваші вхідні дані, то менше Aeromatic здогадується. Надайте якомога більше з PОН:

- Клас літака (планер, легкий одномоторний, транспортний, винищувач, ...) та повну масу.
- Геометрію: розмах крила, площу крила, довжину; наявність керівних поверхонь.
- Тип двигуна та потужність/тягу; кількість двигунів; гвинт чи реактивний.
- Опорні точки характеристик там, де інструмент їх приймає (крейсерська, V_{stall}).

57.2 Що він створює

Повний стартовий набір: **файл літака** (метрики, маса і центрування, параметричний блок аеродинаміки, реакції опори та базова система керування польотом), **файл двигуна** і — для гвинтових літаків — **файл рушія/гвинта**. Аеродинаміка будується з підручникових співвідношень (наближено еліптична крива піднімальної сили, параболічна поляра опору, оцінки похідних за видовженням і об'ємом хвостового оперення), і саме тому результат літає, але ще не відповідає жодному конкретному планеру.

57.3 Припущення та обмеження

- **Один тип двигуна на модель.** Змішану силову установку (напр. поршневий + реактивний) доводиться додавати вручну згодом.
- **Узагальнені похідні.** Похідні демпфування й керування — це емпіричні правила; заради достовірності замініть їх значеннями з DATCOM/CFD/льотних випробувань.
- **Лінійна, симетрична аеродинаміка.** Стандартна вісь riskання не залежить від кута атаки, тож щойно створена модель Aeromatic не буде реалістично штопорити, доки ви не додасте члени $C_n(\alpha, \beta)$ (див. розділ про звалювання/штопор).

57.4 Робочий процес уточнення та поширені пастки

Ставтеся до виводу Aeromatic як до риштування: допомогтеся, щоб модель балансувалася й літала, а потім замінійте таблиці й похідні джерело за джерелом, повторно перевіряючи після кожної зміни. Wiki FlightGear застерігає, що "легко зробити зміни, які призведуть до нелітаючої FDM" — двома класичними помилками є:

- **Порушення лівої/правої симетрії** при переміщенні розташувань — зміщене шасі, бак чи точкова маса вносять фантомний нахил крену/riskання.
- **Зміщення CG надто далеко від AERORP** — велике зміщення CG відносно аеродинамічного орієнтира псує нарощування моменту тангажа й запас поздовжньої стійкості, часто породжуючи незбалансовуваний чи розбіжний літак.

Розділ 58: Достовірність маси, центрування, інерції та палива

Ніщо не впливає на те, як літає літак, більше, ніж розташування його маси. CG задає запас поздовжньої стійкості й балансування; тензор інерції задає обертальний відгук і частоти власних рухів; вигорання палива переміщує CG у польоті. Зробіть цей розділ неправильно — і жодне аеродинамічне налаштування не зробить модель правильною на відчуття.

58.1 Блок `mass_balance`

Порожні інерції задаються відносно CG порожнього літака у конструктивній системі координат; JSBSim додає точкові маси та паливо автоматично. Зверніть увагу на знакову угоду для `ixz`: JSBSim очікує фактичний відцентровий момент інерції (для більшості літаків додатний зв'язок «на кабрування» — це від'ємний `ixz`).

```
<mass_balance>
  <ixx unit="SLUG*FT2"> 948 </ixx>
  <iyy unit="SLUG*FT2"> 1346 </iyy>
  <izz unit="SLUG*FT2"> 1967 </izz>
  <ixz unit="SLUG*FT2"> 0 </ixz>          <!-- XZ product of inertia -->
  <emptywt unit="LBS"> 1500 </emptywt>
  <location name="CG" unit="IN"> <x>41.0</x><y>0</y><z>36.5</z> </location>
  <pointmass name="pilot">
    <weight unit="LBS">180</weight>
    <location unit="IN"> <x>36</x><y>-14</y><z>40</z> </location>
  </pointmass>
</mass_balance>
```

58.2 Оцінювання тензора інерції

Якщо у вас є CAD, беріть інерції прямо з нього відносно CG. Інакше оцініть за радіусами інерції: $I = m \cdot k^2$, де k — частка розмаху (крен), довжини (тангаж) та їх поєднання (рискання). Roskam та USAF DATCOM наводять у таблицях безрозмірні радіуси інерції за класом літака — це набагато краща відправна точка, ніж здогад. Правило перевірки для звичайних літаків: $I_{zz} > I_{yy} > I_{xx}$ і $I_{zz} \approx I_{xx} + I_{yy}$.

58.3 Точкові маси та компонування завантаження

Екіпаж, пасажери, корисне навантаження та підвіски — це елементи `<pointmass>`; JSBSim підсумовує їхню масу, зміщує CG та доповнює тензор інерції їхніми внесками за теоремою про паралельні осі. Моделюйте випадки завантаження, які вас цікавлять (передній CG, задній CG, максимальна повна маса) — керованість помітно різниться між ними, а саме у випадку заднього CG запаси стійкості стають тонкими.

58.4 Паливні баки та зміщення CG у польоті

Кожен `<tank>` має розташування, ємність і вміст; модель силової установки накопичує $\Sigma(\text{положення бака} \times \text{вміст бака})$ у бюджет маси (FGPropulsion.cpp:579-586), тож вигорання палива переміщує CG у реальному часі. Важелі достовірності баків:

- `<capacity>` / `<contents>` у LBS або KG; `<density>` або іменований `<type>` (AVGAS, JET-A).
- `<priority>` та список `<feed>` двигуна задають, які баки спорожнюються першими — моделюйте реальне керування паливом, щоб відтворити міграцію CG.
- `<unusable>` та `<standpipe>` резервують паливо, яке не можна спалити чи злити — важливо для достовірності дальності та тривалості польоту (FGTank.h:113-142).

Модель, яка чудово балансується з повним паливом і стає незбалансованою майже порожньою, майже завжди має бак не на тому місці: звірте спричинений паливом хід CG з діапазоном РОН.

Розділ 59: Достовірність силової установки на практиці

JSBSim відокремлює двигун (виробляє потужність/тягу) від рушія (перетворює її на силу: гвинт, сопло, ротор чи прямо) та баків (тримають паливо). Блок `<propulsion>` з'єднує їх до купи; означення двигуна й рушія містяться в окремих файлах, що завантажуються з теки літака `Engines/` або з глобальної бібліотеки `engine/` (`FGPropulsion.cpp:389-489`).

```
<propulsion>
  <engine file="Continental A-65-8">
    <feed>0</feed>                                <!-- tank index this engine draws -->
    <thruster file="CM7445_MCCauley">
      <location unit="IN"> <x>-5</x><y>0</y><z>0</z> </location>
      <orient unit="DEG"> <roll>0</roll><pitch>0</pitch><yaw>0</yaw> </orient>
    </thruster>
  </engine>
  <tank type="FUEL" number="0">
    <location unit="IN"> <x>45</x><y>0</y><z>30</z> </location>
    <capacity unit="LBS"> 108 </capacity>
    <contents unit="LBS"> 90 </contents>
  </tank>
</propulsion>
```

59.1 Поршневі двигуни

Поршнева модель (`FGPiston`) формує тиск у впускному колекторі з мережі впускного імпедансу й обчислює потужність і витрату палива з робочого об'єму, обертів, об'ємної ефективності та BSFC. Практичний порядок налаштування, згідно з вікі `FlightGear`, такий:

- `ram-air-factor` спочатку, щоб влучити в правильний крейсерський тиск у впускному колекторі.
- `volumetric-efficiency` далі — основний регулятор витрати палива за заданих `MAP/RPM` (наддувні двигуни можуть перевищувати 1.0).
- `bsfc` останнім, щоб узгодити потужність (означення `<bsfc>` перекриває вбудований розрахунок кінських сил).

`<minmp>/<idlerpm>` задають нахил відгуку на холостому ходу / газі; `<maxmp>/<maxrpm>` задають впускний опір; елементи `<numboostspeeds>` та `ratedboost/ratedpower/ratedaltitude` моделюють нагнітачі (`FGPiston.h:66-112`). Реальний, повний приклад (згенерований `Aeromatic`, потім перевірений вручну):

```
<piston_engine name="Continental A-65-8">
  <minmp unit="INHG"> 10.0 </minmp>
  <maxmp unit="INHG"> 28.5 </maxmp>
  <displacement unit="IN3"> 171.0 </displacement>
  <maxhp> 65 </maxhp>
  <cycles> 4.0 </cycles>
  <idlerpm> 700.0 </idlerpm>
  <maxrpm> 2800.0 </maxrpm>
  <volumetric-efficiency> 0.85 </volumetric-efficiency>
  <stroke unit="IN"> 3.625 </stroke>
  <bore unit="IN"> 3.875 </bore>
  <cylinders> 4 </cylinders>
  <compression-ratio> 6.3 </compression-ratio>
</piston_engine>
```


59.2 Турбіни: турбореактивні та турбовентиляторні двигуни

FGTurbine моделює два каскади (N1 — вентилятор, N2 — газогенератор) із запізнено-фільтрованим розкручуванням/гальмуванням, тягою за пошуком у таблиці за числом Маха та висотою й опціональним форсуванням. Ключові регулятори (FGTurbine.h:84-109): `<milthrust>/<maxthrust>` (без форсажу / з форсажем), `<bypassratio>`, `<tsfc>/<atsfc>` (витрата палива), `<idlen1/2>`, `<maxn1/2>`, часи розкручування `<n1spinup>/<n2spinup>` та `<augmented>/<augmethod>` для форсажної камери. Таблиці тяги — це серце достовірності; беріть їх з характеристик двигуна (engine deck), якщо можете. Заголовок J79 (F-4):

```
<turbine_engine name="J79">
  <milthrust> 10000.0 </milthrust>
  <maxthrust> 15800.0 </maxthrust>      <!-- with afterburner -->
  <bypassratio> 0.0 </bypassratio>
  <tsfc> 0.98 </tsfc>
  <atsfc> 1.96 </atsfc>      <!-- afterburning TSFC -->
  <idlen2> 53.0 </idlen2>
  <maxn1> 100.0 </maxn1> <maxn2> 100.0 </maxn2>
  <augmented> 1 </augmented> <augmethod> 1 </augmethod>
  <!-- IdleThrust / MilThrust / MaxThrust tables vs mach, altitude ... -->
</turbine_engine>
```

59.3 Турбогвинтові, електричні та ракетні двигуни

- **Турбогвинтовий** (FGTurboProp): один каскад, бета-діапазон нижче `<betarangeend>`, моделювання ІТТ, обмежувач крутного моменту IELU та потужність (`<maxpower>`, `<psfc>`), що приводить гвинт постійних обертів.
- **Електричний** (FGElectric): єдина `<power>` у ватах, лінійна за газом, нульове вигорання палива — природний вибір для eVTOL та електричних БПЛА.
- **Ракетний** (FGRocket): питомий імпульс `<isp>` та опціональна таблиця тяги від часу; споживає баки і палива, і окисника.

59.4 Паливна система та узгодження характеристик

Паливо споживається за пріоритетом: баки з найменшим числом `<priority>` спорожнюються першими, а двигун живиться лише з баків у своєму списку `<feed>` (FGPropulsion.cpp:169-263). Перевіряйте силову установку за трьома опорними точками з РОН / характеристик двигуна: статична тяга на рівні моря чи злітна потужність, крейсерська витрата палива на відомій висоті та режимі потужності й найкраща швидкопідйомність. Налаштовуйте `volumetric-efficiency/tsfc` за витратою палива, а коефіцієнти тяги/потужності — за швидкопідйомністю та максимальною швидкістю.

Розділ 60: Повітряні гвинти, рушії та регулятор постійних обертів

Для гвинтових літаків рушій — це місце, де потужність двигуна стає тягою, і він є поширеним джерелом нереалістичної поведінки. FGPropeller працює в коефіцієнтній формі, точно як аеродинаміка: безрозмірні коефіцієнти тяги й потужності наводяться в таблицях залежно від відносної ходи та кута установки лопаті.

60.1 Файл повітряного гвинта

Геометрія та обмеження (FGPropeller.h:59-101): <diameter>, <numblades>, полярний момент <ixx> (задає інерцію розкручування та гіроскопічний момент), <gearratio>, діапазон кута установки <minpitch>/<maxpitch> та регульований діапазон обертів <minrpm>/<maxrpm>, плюс множники налаштування <ct_factor>/<cp_factor>.

```
<propeller name="CM7445" version="1.0">
  <ixx>          1.67 </ixx>
  <diameter unit="IN"> 74.0 </diameter>
  <numblades>     2 </numblades>
  <gearratio>     1.0 </gearratio>
  <minpitch>      10 </minpitch>
  <maxpitch>      25 </maxpitch>      <!-- fixed-pitch: omit governor -->
  <table name="C_THRUST" type="internal">
    <independentVar lookup="row">advance-ratio</independentVar>
    <tableData> 0.0  0.0653
                  0.4  0.0524
                  0.8  0.0241
                  1.2 -0.0140 </tableData>
  </table>
  <table name="C_POWER" type="internal">
    <independentVar lookup="row">advance-ratio</independentVar>
    <tableData> 0.0  0.0383
                  0.4  0.0378
                  0.8  0.0291
                  1.2  0.0097 </tableData>
  </table>
</propeller>
```

60.2 Коефіцієнти, відносна хода та число Маха

Відносна хода $J = V/(n \cdot D)$ (n у об/с) — це "кут атаки" гвинта. Тяга й потужність впливають із:

$$T = C_T(J) \cdot \rho \cdot n^2 \cdot D^4 \quad P = C_P(J) \cdot \rho \cdot n^3 \cdot D^5$$

Опціональні таблиці CT_MACH/CP_MACH (індексовані за гвинтовим числом Маха на кінці лопаті) відображають втрати на стисливість, коли кінці лопатей наближаються до швидкості звуку (FGPropeller.h:333-334). Індукована швидкість розв'язується з імпульсною теорії (FGPropeller.cpp:256-261).

60.3 Регулятор постійних обертів

Блок постійних обертів утримує цільові оберти, змінюючи кут установки лопаті між minpitch та maxpitch. Розподіл органів керування пілота такий самий, як на реальному літаку: **газ** задає тиск у впускному колекторі (потужність), **важіль гвинта** задає регульовані оберти, а **якість суміші** задає співвідношення паливо/повітря. Установіть <constspeed>1</constspeed> та розумну смугу обертів; регулятор тоді змінює кут установки, щоб утримувати оберти зі зміною повітряної швидкості та потужності.

60.4 Р-фактор, напрям, авторотація гвинта та реверс

- `<sense>` (± 1) задає напрям обертання; він керує закруткою струменя за гвинтом та реакцією крутного моменту двигуна, проти якої літак має балансуватися.
- `<p_factor>` зміщує точку прикладання тяги залежно від кута атаки, породжуючи характерне рискання на великих α /великій потужності (FGPropeller.cpp:266-278).
- **Авторотація гвинта (windmilling)** (від'ємна тяга, великий опір) виникає природно, коли J робить C_T від'ємним; **флюгування** (maxpitch вирівняно з потоком) знижує цей опір заради достовірності при відмові двигуна.
- **Реверс / бета**-діапазон дає гальмування на землі; поєднайте його з турбогвинтовим `<betarangeend>` чи `<reversepitch>`.

Розділ 61: Шасі та керування рухом по землі

Керування рухом по землі — це місце, де розвалюються багато інакше хороших моделей: літак підстрибує, ковзає, входить у некерований розворот (ground-loop) чи провалюється крізь злітно-посадкову смугу. Шасі JSBSim — це амортизаційна стійка з пружиною й демпфером із моделлю тертя, означена як елементи `<contact>` всередині `<ground_reactions>` (FGLGear.cpp, FGGroundReactions.cpp:135-156).

61.1 Закон сили стійки

Кожна стійка обчислює нормальну силу зі стиснення та швидкості стиснення (FGLGear.cpp:624-653). Сила — це пружний член плюс демпфувальний член, обмежений так, щоб стійка могла лише штовхати:

$$F_{\text{strut}} = -(k \cdot x + c \cdot v), \text{ clamped to } F \leq 0$$

з окремими коефіцієнтами для стиснення (`<damping_coeff>`) та віддачі (`<damping_coeff_rebound>`) і опціональним `type="SQUARE"`, який робить демпфування пропорційним v^2 заради сильнішого контролю над екстремальними стисненнями. Типовий контакт BOGEY:

```
<contact type="BOGEY" name="LEFT_MAIN">
  <location unit="IN"> <x>58</x><y>-50</y><z>-18</z> </location>
  <static_friction> 0.80 </static_friction>
  <dynamic_friction> 0.50 </dynamic_friction>
  <rolling_friction> 0.02 </rolling_friction>
  <spring_coeff unit="LBS/FT"> 5400 </spring_coeff>
  <damping_coeff unit="LBS/FT/SEC"> 160 </damping_coeff>
  <damping_coeff_rebound unit="LBS/FT/SEC"> 320 </damping_coeff_rebound>
  <max_steer unit="DEG"> 0 </max_steer> <!-- 0 = fixed, 360 = caster -->
  <brake_group> LEFT </brake_group>
  <retractable> 1 </retractable>
</contact>
```

61.2 Тертя, керування поворотом і гальма

- **Тертя** використовує статичний, динамічний коефіцієнти та коефіцієнт тертя кочення; гальмування додає частку (static – rolling), масштабовану положенням гальма (FGLGear.cpp:588-594). Бічна сила (у повороті) використовує «магічну формулу» Расаєжа із вбудованими коефіцієнтами (жорсткість 0.06, форма 2.8, пік = статичний коефіцієнт, кривина 1.03), які можна перекрити таблицею CORNERING_COEFF (FGLGear.cpp:596-615).
- **Керування поворотом** задається через `<max_steer>`: 0° = фіксоване, 360° (або `<castered>1`) = вільно орієнтоване, будь-що між = кероване за командою через `fcs/steer-cmd-norm` (FGLGear.cpp:151-165).
- **Групи гальм** LEFT/RIGHT/CENTER (NOSE/TAIL відображаються на CENTER) прив'язують шасі до `fcs/...-brake-cmd-norm` (FGLGear.cpp:204-217); диференціальне гальмування — це те, як літаки з хвостовим колесом і багато реактивних літаків керують поворотом на малій швидкості.
- **Контакти STRUCTURE** (законцівки крила, хвостовий костиль, мотогондולי) — це неколісні точки удару/тертя об землю; `<retractable>` прив'язує стійку до `gear/unit[i]/pos-norm`.

61.3 Поверхня землі

FGSurface надає `ground/solid` (вода чи суша — нетверда поверхня не дає ваги на колесах), `ground/bumpiness` (процедурну нерівність злітно-посадкової смуги) та множники коефіцієнта тертя й максимальної сили, усі з яких можна встановлювати окремо для кожної поверхні, щоб моделювати мокрі, обледенілі чи нерівні аеродроми (`FGSurface.cpp`).

61.4 Налаштування стабільної, реалістичної поведінки на землі

- **Жорсткість пружини** $\approx$ вага на стійку / статичне стиснення. Починайте з реального ходу стійки та статичного розподілу навантаження.
- **Використовуйте найм'якшу пружину, з якою можете змиритися.** Надто жорстке шасі з сильним демпфуванням вносить енергію на кожному часовому кроці й змушує літак підстрибувати — частий збій, про який повідомляють у списку розробників JSBSim.
- **Демпфування віддачі $\geq$ демпфування стиснення**, щоб поглинути посадку, не виштовхуючи літак назад у повітря; розгляньте `type="SQUARE"`, щоб приборкати екстремальні приземлення.
- **Стежте за часовим кроком.** Жорстке шасі потребує малого `dt`; якщо стійка може стиснутися більше, ніж це фізично можливо за один крок, сили вибухають. JSBSim обмежує швидкість стиснення за крок, але м'яке шасі + малий `dt` — це надійне поєднання.
- **Інструментуйте це.** Записуйте `gear/unit[i]/compression-ft, .../WOW` та `.../compression-velocity-fps` під час приземлення, щоб точно бачити, що робить стійка.

Розділ 62: Системи керування польотом на практиці

Система керування польотом відображає команди пілота на відхилення поверхонь. У JSBSim це набір <channel> із компонентів, що виконуються по порядку, кожен з яких читає й записує дерево властивостей, тож канал — це буквально діаграма проходження сигналу в XML (FGFCS.cpp, FGFCSChannel.h). Той самий механізм обслуговує три секції — <flight_control>, <system> та <autopilot>.

62.1 Каталог компонентів

Компонент	Елемент	Ключові параметри / передавальна функція
Чистий коефіцієнт підсилення	pure_gain	gain (константа чи властивість)
Планований коефіцієнт	scheduled_gain	gain × table(schedule)
Масштаб керівної поверхні	aerosurface_scale	відображення domain→range; cmd-norm ↔ surface-rad
Суматор	summer	Σ входів + bias, clipto
Фільтр запізнення	lag_filter	$C1/(s+C1)$
Випередження-з апізнення / washout	lead_lag_filter / washout_filter	$(C1\ s+C2)/(C3\ s+C4)$; $s/(s+C1)$
Фільтр 2-го порядку	second_order_filter	повний біквад C1..C6
Інтегратор	integrator	rect/trap/ab2/ab3, з тригером
ПІД-регулятор	pid	kp, ki, kd; trigger (захист від насичення)
Зона нечутливості	deadband	width, gain
Перемикач	switch	перевірки з умовами AND/OR, default
Кінематичний	kinematic	обмежений за швидкістю перехід між налаштуваннями
Привод	actuator	lag, rate_limit, hysteresis, bias, fail
Функція FCS	fcs_function	довільна <function>

Кожен компонент підтримує <input> (додайте до властивості префікс -, щоб інвертувати), опціональний <output> для копіювання його результату в іншу властивість і <clipto> для його насичення.

62.2 Шлях керування, від початку до кінця

Мінімальний механічний (reversible) канал тангажа: підсумовуємо команди пілота й балансування, обмежуємо до нормованого діапазону, масштабуємо до кута поверхні, потім публікуємо нормоване положення для анімації. З моделі OV-10 (aircraft/OV10/OV10.xml:241-272):

```
<channel name="Pitch">
  <summer name="Pitch Trim Sum">
    <input> fcs/elevator-cmd-norm </input>
    <input> fcs/pitch-trim-cmd-norm </input>
  </summer>
  <clipto> <min>-1</min> <max>1</max> </clipto>
```

```

</summer>
<aerosurface_scale name="Elevator Control">
  <input> fcs/pitch-trim-sum </input>
  <range> <min>-0.35</min> <max>0.35</max> </range>  <!-- radians -->
  <output> fcs/elevator-pos-rad </output>
</aerosurface_scale>
<aerosurface_scale name="Elevator Normalized">
  <input> fcs/elevator-pos-rad </input>
  <domain> <min>-0.35</min> <max>0.35</max> </domain>
  <range> <min>-1</min> <max>1</max> </range>
  <output> fcs/elevator-pos-norm </output>
</aerosurface_scale>
</channel>

```

Аеродинаміка потім читає fcs/elevator-pos-rad (або -norm) у своїх керівних приростах — FCS та аеродинаміка зустрічаються в дереві властивостей.

62.3 Механічне, бустерне керування та fly-by-wire

Ті самі будівельні блоки масштабуються від легкого літака з тросами й блоками (команда прямо на поверхню, з передаточним відношенням і нелінійним kinematic для закрилків) до бустерної системи (додайте динаміку приводів) і до fly-by-wire (вставте фільтри, коефіцієнти підсилення та зворотний зв'язок, щоб сформувати відгук і додати систему покращення стійкості — наступний розділ). Моделюйте передаточне відношення та змішування керування явно: змішування спойлеронів, комбінації елевонів/рудеватормів і взаємозв'язки елерон-руль напряду — це лише суматори й коефіцієнти підсилення.

62.4 Достовірність приводів

Реальні поверхні не миттєві. Компонент <actuator> (FGActuator.h:54-122) застосовує, по порядку, <lag>, <rate_limit> (опціонально різні для збільшення/зменшення), <deadband_width>, <hysteresis_width> (люфт) та <bias>. Обмеження швидкості важливі для пілотажних характеристик: недостатньо швидкий привод спричиняє розкачку, викликану пілотом (pilot-induced oscillation). Компонент також підтримує введення відмов (fail_zero, fail_hardover, fail_stuck) для випробувань систем.

Розділ 63: Покращення стійкості, автопілоти та наведення

Щойно «голий» планер починає літати, автоматичне керування додає достовірності й корисності: демпфери, що гасять небажані рухи, автопілоти, що утримують стани, і наведення, що проводить маршрути. Усе це будується з тих самих компонентів FCS, розміщується в секції <autopilot> і за угодою керується властивостями ap/.

63.1 Покращення стійкості

Демпфер рискання — канонічний приклад: подайте кутову швидкість рискання тіла через washout_filter (щоб він боровся з коливаннями, але не зі сталими, командними розворотами) та коефіцієнт підсилення на руль напряду. Відгук washout-фільтра $s/(s+C1)$ пропускає перехідну кутову швидкість і блокує сталу складову — саме те, що приборкує слабо задемпфований голландський крок. Демпфери тангажа й крену подають q та r аналогічно. Оскільки покращення підсумовується в той самий канал поверхні, що й у пілота, будуйте його як додатковий вхід summer.

63.2 Канали автопілота з ПІД-регуляторами

Режими утримання — це ПІД-контури, замкнені на похибці стану. Вирівнювач крил автопілота C-172 показує цей прийом — реалістичний sensor на куті крену, switch, що вмикає режим, та pid, чий <trigger> забезпечує захист від насичення інтегратора (aircraft/c172x/c172ap.xml):

```
<channel name="Roll wing leveler">
  <sensor name="fcs/attitude/sensor/phi-rad">
    <input> attitude/phi-rad </input>
    <lag> 0.50 </lag>
    <noise variation="PERCENT" distribution="GAUSSIAN"> 0.05 </noise>
    <bias> 0.001 </bias>
  </sensor>
  <switch name="fcs/wing-leveler-ap-on-off">
    <default value="-1"/>
    <test value="0"> ap/attitude_hold == 1 </test>
  </switch>
  <pid name="fcs/roll-ap-error-pid">
    <input> attitude/phi-rad </input>
    <kp> ap/roll-pid-kp </kp>
    <ki> ap/roll-pid-ki </ki>
    <kd> ap/roll-pid-kd </kd>
    <trigger> fcs/wing-leveler-ap-on-off </trigger>    <!-- anti-windup -->
  </pid>
</channel>
```

Виведення коефіцієнтів підсилення як властивостей (ap/roll-pid-kp, ...) дозволяє налаштовувати їх наживо зі сценарію чи консолі без перезбирання. Типові режими утримання: просторове положення, висота, вертикальна швидкість, курс і повітряна швидкість (часто замикається на газ).

63.3 Налаштування контурів

- Замикайте спочатку внутрішні контури (кутова швидкість/просторове положення), потім зовнішні (висота, курс) — внутрішній контур є об'єктом, який бачить зовнішній контур.
- Починайте лише з P, додайте D, щоб задемпфувати перерегулювання, додайте рівно стільки I, скільки треба, щоб усунути сталу похибку; використовуйте <trigger>, щоб

зупинити насичення інтегратора при насиченні.

- Плануйте коефіцієнти підсилення за швидкісним напором (`scheduled_gain` на `aero/qbar-psf` чи повітряній швидкості), щоб контур поведився коректно в усьому діапазоні.

63.4 Наведення та навігація

Компоненти `waypoint_heading` та `waypoint_distance` обчислюють ортодромічний пеленг і відстань до цільових широти/довготи; подайте пеленг в утримання курсу — і ви маєте базовий LNAV. Компонент `angle` повертає найменший заданий кут (зручно для похибки курсу через перехід $\pm 180^\circ$). Для повних місій керуйте заданими значеннями зі сценарію (див. розділ про сценарії).

Розділ 64: Датчики, системи та моделювання відмов

Робота високої достовірності — апаратура в контурі (hardware-in-the-loop), розробка оцінювача стану й автопілота, аналіз відмов — потребує більшого, ніж ідеальні стани. JSBSim моделює недосконалі датчики, довільні підсистеми та відмови, що вводяться, — усе це через фреймворк компонентів FCS і дерево властивостей.

64.1 Недосконалі датчики

Компонент `<sensor>` (`FGSensor.h:56-127`) погіршує чистий сигнал так само, як це робить реальна апаратура:

- `<lag>` — стала часу першого порядку (смуга пропускання датчика).
- `<noise>` — PERCENT або ABSOLUTE, з розподілом UNIFORM чи GAUSSIAN.
- `<quantization>` — біти на діапазоні min/max (роздільність АЦП).
- `<drift_rate>`, `<bias>`, `<gain>` — повільний дрейф, зміщення та похибки масштабу.
- `<delay>` — транспортна затримка в часі чи кадрах.

Спеціалізовані датчики будуються на цьому: `<accelerometer>` та `<gyro>` беруть `<location>/<orientation>` та `<axis>` і повідомляють виміряну питому силу / кутову швидкість у тій точці (тож акселерометр на носі відчуває кутове прискорення тангажа), а `<magnetometer>` зчитує поле. Подавання саме цих сигналів — а не істинних станів — у ваші закони керування і є різницею між демонстрацією та стендом для розробки.

64.2 Довільні підсистеми

Секція `<system>` — це вільний набір каналів FCS, який можна використати для моделювання будь-чого: електричних шин, гідравлічного тиску, логіки керування паливом, виявлення пожежі/перегріву, систем балансування чи логіки озброєння/вантажу. Оскільки кожен компонент читає й записує властивості, дерево властивостей є системною шиною — `switch` може вмикати насос за властивістю напруги, `lag_filter` може моделювати наростання тиску, а `fcs_function` може обчислити будь-яке алгебраїчне співвідношення. JSBSim постачає придатні для повторного використання системи (напр. `systems/catapult.xml`), які можна підключити.

64.3 Зовнішні реакції та плавучість

`<external_reactions>` додають іменовані сили/моменти в системах BODY, LOCAL чи WIND, величина яких керується властивістю чи `<function>` — катапульти, гальмівні гаки, буксирні троси, скидання кінцевих баків, лебідковий старт. Катапульта — це лише `switch`, який записує `external_reactions/catapult/magnitude`, коли вона зведена. `<buoyant_forces>` з `<gas_cell>` (HYDROGEN/HELIUM/AIR) та балонетами моделюють аеростати й дирижаблі (`FGBuoyantForces`, `FGGasCell`).

64.4 Введення відмов

Достовірність включає й те, що йде не так. Приводи надають `fail_zero/fail_hardover/fail_stuck`; датчики можна зміщувати, заморожувати чи зашумлювати через їхні властивості; двигуни можна позбавити палива, спорожнивши бак, чи зупинити через `propulsion/engine[i]/set-running`; керівні поверхні можна заклинити за допомогою `switch`. Керуйте всім цим зі сценарію, щоб побудувати відтворюваний набір тестів сценаріїв відмов.

Розділ 65: Моделювання великих кутів атаки, звалювання та штопора

Лінійні коефіцієнти описують крейсерський діапазон; достовірність на межах — звалювання, зрив у некерований режим, штопор — потребує нелінійної, асиметричної, залежної від передісторії аеродинаміки. Це найважча частина FDM для правильної реалізації й та частина, яка найбільше вирізняє серйозну модель.

65.1 Моделювання звалювання

Звалювання — це відривання течії: піднімальна сила переламується й падає, тоді як опір круто зростає. Відтворіть його, табулюючи C_L та C_D в усьому діапазоні α — далеко за C_{Lmax} , крізь зазвалювальний спад, в ідеалі до $\pm 90^\circ$ для роботи зі штопором та виведенням зі складних положень. Найбільша аеродинамічна ознака звалювання — це зростання опору, тож не нехуйте кінцем таблиці опору на великих α .

65.2 Гістерезис звалювання

Відривання та повторне приєднання течії відбуваються за різних кутів, тож звалювання має пам'ять. JSBSim моделює це за допомогою `<alphalimits>` та смуги `<hysteresis_limits>`, надаючи `aero/stall-hyst-norm` (0→1 уздовж смуги, `FGAerodynamics.cpp`). Використовуйте це як другий вимір таблиці, щоб крива піднімальної сили йшла різним шляхом при звалюванні та при виведенні — модель `c172` робить саме це (`aircraft/c172x/c172x.xml`).

```
<alphalimits unit="DEG"> <min>-12</min> <max>22</max> </alphalimits>
<hysteresis_limits unit="DEG"> <min>12</min> <max>18</max> </hysteresis_limits>
...
<table>
  <independentVar lookup="row">aero/alpha-rad</independentVar>
  <independentVar lookup="column">aero/stall-hyst-norm</independentVar>
  <tableData>
    0.0    1.0    <!-- attached   stalled -->
    0.21   1.25   0.86
    0.28   1.47   0.92
    0.35   1.20   1.05    <!-- post-stall, different recovery path -->
  </tableData>
</table>
```

65.3 Штопор та зрив у некерований режим

Штопор — це нестійкість по осі рискання у поєднанні зі звалюванням. Вирішальний момент моделювання — і задокументоване обмеження стандартного виводу `Aeromatic` — полягає в тому, що момент рискання має залежати від кута атаки: плаский $C_n(\beta)$ ніколи не дасть авторотації. Для правдоподібних штопорів вам потрібні:

- C_n та C_l як функції і α , і β (2-D таблиці), щоб зазвалювальна асиметрія рискання/крену могла спричиняти авторотацію.
- Демпфування крену й рискання (C_{lp} , C_{nr}), яке змінює знак чи величину за звалюванням — саме втрата демпфування крену дозволяє штопору розвинутися.
- Асиметричне звалювання крила та здорове зростання опору на великих α , щоб задати швидкість обертання штопора й зниження.
- Погіршену ефективність керування на великих α (затінені руль висоти/руль напряду), щоб виведення вимагало правильної техніки.

Високостовірні моделі зазвалювального режиму/штопора будуються з даних аеродинамічної труби, отриманих на ротаційному балансі та у вимушених коливаннях

(NASA має обширні набори) або, дедалі частіше, з DES/гібридного CFD; дослідники використовують біфуркаційний аналіз для характеристики відповідних режимів зриву, зазвальнової гірації та штопора.

65.4 Інші крайові ефекти

Вплив екрана землі (таблиця множників піднімальної сили й опору від висоти/розмаху, $aero/h_b-mac-ft$), стисливість (вимір таблиці за числом Маха), ефекти числа Рейнольдса (запускайте CFD за льотного Re) та бафтинг (збурення, подібне до турбулентності, за звалюванням) — усе це додає достовірності на межах. Пам'ятайте, що JSBSim тримає кінці таблиць плоскими — ваші таблиці за α , β та числом Маха мають насправді охоплювати той діапазон, у якому ви маєте намір літати.

Розділ 66: Сценарії для автоматизованих випробувань, налаштування та system ID

Налаштування ручним пілотуванням повільне й невідтворюване. Мова сценаріїв JSBSim прокручує модель без візуалізації за прописаною часовою лінією, вводючи вхідні дані й записуючи виходи — це основа регресійних випробувань, ідентифікації параметрів та досліджень збурень (FGScript.cpp).

66.1 Анатомія сценарію запуску

`<runscript>` називає літак і файл ініціалізації (скидання), задає часове вікно та крок і містить `<event>` та директиви `<output>`:

```
<?xml version="1.0"?>
<runscript name="elevator doublet">
  <use aircraft="c172x" initialize="reset01"/>
  <run start="0.0" end="60.0" dt="0.0083333">
    <event name="Trim">
      <condition> simulation/sim-time-sec ge 0.0 </condition>
      <set name="simulation/do_simple_trim" value="1"/>
    </event>
    <event name="Doublet up">
      <condition> simulation/sim-time-sec ge 5.0 </condition>
      <set name="fcs/elevator-cmd-norm" value="0.3"
        action="FG_STEP"/>
    </event>
    <event name="Doublet down">
      <condition> simulation/sim-time-sec ge 6.0 </condition>
      <set name="fcs/elevator-cmd-norm" value="-0.3" action="FG_STEP"/>
    </event>
    <event name="Center">
      <condition> simulation/sim-time-sec ge 7.0 </condition>
      <set name="fcs/elevator-cmd-norm" value="0.0" action="FG_RAMP" tc="0.2"/>
    </event>
    <output name="doublet.csv" type="CSV" rate="50">
      <rates> ON </rates> <velocities> ON </velocities>
      <position> ON </position> <fcs> ON </fcs>
    </output>
  </run>
</runscript>
```

Запустіть без візуалізації: `JSBSim --script=scripts/doublet.xml`. Умови використовують трійки властивість-оператор-значення (`ge le eq ne` чи символічні форми) й вкладаються через `logic="AND|OR"` (FGCondition.cpp:109-128).

66.2 Дії set: крок, лінійна зміна, експонента

`<set>` змінює властивість як `FG_STEP`, лінійну `FG_RAMP` за сталу часу `tc` або як експоненційне наближення першого порядку `FG_EXP`; `type="FG_DELTA"` робить значення приростом (FGScript.cpp:472-495). Це все, що потрібно для синтезу класичних вхідних впливів system ID:

- **Здвоєний імпульс (doublet)** (показаний вище) — збуджує короткоперіодичний рух / голландський крок для ідентифікації демпфування та частоти.
- **3-2-1-1** — багатокроковий вхідний вплив, багатий у смузі частот, стандарт для ідентифікації параметрів.
- **Розгортка за частотою** — змінюйте частоту синусоїди через `fcs_function`, щоб відобразити частотну характеристику.

- **Кроки газу/керування** — для точок характеристик (набір висоти, розгін) та перевірки балансування.

66.3 Вивід, notify та пакетні робочі процеси

`<output type="CSV">` обирає цілі підсистеми (rates, velocities, forces, moments, aerosurfaces, propulsion, ...) та/або окремі `<property>`, опціонально перетворюючи одиниці функцією `apply` (FGOutputType.cpp:100-128). `<notify>` друкує обрані властивості, коли спрацьовує подія. Обгорніть запуски сценаріїв у модулі Python (`import jsbsim`), щоб розгортати параметри, автоматично балансувати в усьому діапазоні й перевіряти результати твердженнями — перетворюючи ваше налаштування на автоматизований набір регресійних тестів.

66.4 Випробування збуреннями та турбулентністю

Робастність виникає з польоту моделі в погоді. Атмосфера JSBSim забезпечує сталі шари вітру, дискретні пориви та неперервну турбулентність (спектри типу Dryden із обиральною інтенсивністю); задайте їх із файлу IC чи сценарію й переконайтеся, що літак із покращенням стійкості все ще утримує свої власні рухи. Поєднайте це з розгортками похідних за методом Монте-Карло з розділу про дані, щоб обмежити пілотажні характеристики з урахуванням як атмосферної, так і модельної невизначеності.

Розділ 67: Узгодження льотних і пілотажних характеристик

Остаточна перевірка достовірності кількісна: чи відтворює модель опубліковані льотні характеристики літака й чи виявляє правильні пілотажні характеристики? Це фаза приймання — цикл, у якому ви порівнюєте з даними й коригуєте відповідальні числа.

67.1 Балансування та точки характеристик

Починайте з балансування (розділ про алгоритм балансування): переконайтеся, що літак балансується за розумних просторових положень і положень органів керування для різних мас і висот. Потім пропишіть у сценарії опорні точки характеристик і порівняйте з POH:

Цільова характеристика	Як виміряти в JSBSim
Швидкість звалювання (чиста / з закритками)	балансувати за зростання α до C_{Lmax} ; зчитати V
Найкраща швидкопідйомність / V_y	збалансована розгортка набору висоти на повному газі за швидкістю
Крейсерська TAS & витрата палива	балансувати на крейсерських потужності/висоті; зчитати velocities/propulsion
Практична стеля	набирати висоту, доки швидкопідйомність не впаде до 100 ft/min
Дистанція зльоту / посадки	прописаний у сценарії пробіг із шасі + гальмами
Дальність / тривалість польоту	інтегрувати вигорання палива на крейсерському режимі до порожнього

67.2 Цільові пілотажні характеристики

Лінеаризуйте відносно балансування (або підженіть відгуки на здвоєний імпульс) і звірте характеристики власних рухів із критеріями MIL-STD-1797 / старішого MIL-F-8785C та оцінюваної пілотами шкали Cooper-Harper:

- **Короткоперіодичний рух** — частота й демпфування у зоні Level-1 на діаграмі CAP (Control Anticipation Parameter); це рух, який пілоти відчувають найдужче.
- **Фугоїд** — довгоперіодичний, слабо задемпфований, але не розбіжний (демпфування > 0).
- **Голландський крок** — достатні частота й демпфування; демпфер ризику існує саме для того, щоб цього досягти.
- **Крен-мода** — стала часу достатньо мала для чіткого відгуку по крену; **спіральний рух** — у найгіршому разі повільно розбіжний.

67.3 Енергетичні методи та цикл ітерацій

Питома надлишкова потужність $P_s = V(T-D)/W$ пов'язує тягу, опір і вагу з набором висоти та розгоном; узгодження P_s в усьому діапазоні одночасно перевіряє поляру опору та модель тяги. Коли щось не так, розбіжність указує на винуватця: неправильна крейсерська швидкість → поляра опору чи тяга; неправильний набір висоти → надлишкова потужність; неправильна швидкість звалювання → C_{Lmax} ; неправильний короткоперіодичний рух → C_{ma}/C_{mq} або інерція/CG. Виправте це число, перезапустіть сценарій, повторіть. Обмежте залишкову невизначеність розгортками похідних за методом Монте-Карло, щоб знати, що модель робастна, а не лише налаштована під одну точку.

Спершу перевіряйте розімкнений контур: із зафіксованими органами керування вільний відгук хорошої моделі (власні рухи, відхід балансування, планування) має вже відповідати льотним даним ще до ввімкнення будь-якого автопілота.

Розділ 68: Інтеграція з FlightGear та зовнішніми симуляторами

JSBSim — це бібліотека; достовірність, яку сприймає користувач, також залежить від того, як її під'єднано до візуального симулятора чи стека керування. Контрактом завжди є дерево властивостей: хост записує команди й середовище, JSBSim інтегрує фізику, а хост зчитує стани назад для візуалізації та приладів.

68.1 FlightGear

FlightGear вбудовує JSBSim як рідну FDM. Точки інтеграції:

- **Входи:** хост записує `fcs/aileron-cmd-norm`, `elevator-cmd-norm`, `rudder-cmd-norm`, `throttle-cmd-norm`, команди шасі/закрилків/гальм тощо.
- **Виходи:** він зчитує положення для анімацій 3-D моделі — `fcs/...-pos-norm` для поверхонь і шасі, оберти двигуна / N1 та стан тіла для виду.
- **Середовище:** FlightGear надає вітри, температуру та висоту землі/рельєфу, з якими контактує шасі.
- **Зв'язний шар:** тримайте імена властивостей на стандартних шляхах, щоб штатні прив'язки FlightGear і код приладів їх знаходили; документуйте будь-які власні властивості `ap/` чи `systems/`, які ви додаєте.

68.2 Сокети, рідні протоколи та інші рушії

Для зовнішніх стеків `<output type="SOCKET">` та `type="FLIGHTGEAR"` транслюють стан через TCP/UDP, а API Python/C++ вбудовують рушій безпосередньо. JSBSim — це фізичне ядро за FlightGear, проектом Antoinette на Unreal Engine, ArduPilot і PX4 software-in-the-loop та середовищами навчання з підкріпленням на кшталт gym-jsbsim; у кожному випадку хост крокує модель і обмінюється тим самим набором властивостей. Узгодьте частоту кадрів хоста з розумним `dt JSBSim` (часто 120 Hz) та інтерполуйте візуалізацію, а не сповільнюйте фізику.

68.3 Поширені пастки інтеграції

- Подвійні шляхи керування — і хост, і внутрішній автопілот керують тією самою поверхнею; вирішіть, кому належить кожна команда.
- Неузгодженість частоти кадрів / `dt`, що спричиняє тремтіння чи нестійкість, особливо з жорстким шасі.
- Розбіжності опорних значень рельєфу/висоти (AGL чи MSL, геоїд чи еліпсоїд), через які шасі ширяє чи провалюється.
- Неузгодженість одиниць на межі — команди JSBSim нормовані, але виходи стану — у футах, fps, радіанах, якщо не перетворено.

Розділ 69: Зведений контрольний список достовірності та поширені пастки

Цей розділ зводить Частину IV у контрольний список, який ви можете прогнати по будь-якій моделі. Жоден із цих пунктів не екзотичний; кожен — це реальний збій, помічений на форумах і в списку розробників, і більшості тривіально уникнути, щойно ви знаєте, де шукати.

69.1 Геометрія, маса та центрування

- Ліва/права симетрія кожного розташування (шасі, баки, точкові маси, рушії), якщо асиметрія не передбачена навмисно.
- CG тримається біля AERORP та всередині діапазону РОН за всіх завантажень і станів палива; перевірте хід CG від вигорання палива.
- Тензор інерції фізичний: $I_{zz} > I_{yy} > I_{xx}$, $I_{zz} \approx I_{xx} + I_{yy}$, розумний знак ixz .
- Опорні S , b , $\bar{c}$ у `<metrics>` відповідають значенням, використаним для знерозмірнення аеродинамічних даних.

69.2 Аеродинаміка

- Таблиці охоплюють увесь передбачений діапазон за α , β та числом Маха (кінці тримаються плоскими, не екстраполюються).
- $C_{m\alpha} < 0$ (статично стійкий) з розумним запасом поздовжньої стійкості; $C_{n\beta} > 0$, $C_{l\beta} < 0$.
- Похідні демпфування присутні й мають правильний знак (C_{mq} , C_{lp} , $C_{nr} < 0$); одиниці відповідають угоді `bi2vel/ci2vel`.
- Рискання/крен залежать від α , якщо ви хочете звалювання/штопор; присутнє зростання опору на великих α .
- Знаки керування перевірені: додатний `elevator-cmd` робить із тангажем те, що ви очікуєте.

69.3 Силова установка, шасі та керування

- Силова установка узгоджена зі статичною тягою/потужністю, крейсерською витратою палива та набором висоти; напрям/крутий момент гвинта збалансовано.
- Жорсткість пружини шасі $\approx$ навантаження/стиснення, достатньо м'яка, щоб уникнути приросту енергії; демпфування віддачі $\geq$ стиснення; приземлення записане й стабільне.
- Групи гальм і керування поворотом під'єднані; літаки з хвостовим колесом керуються поворотом і не входять у некерований розворот.
- Обмеження швидкості приводів реалістичні (немає розкачки, викликаної пілотом); передаточне відношення/змішування керування правильні.

69.4 Перевірка та інтеграція

- Балансується в усьому діапазоні без розбіжності; вільний відгук у розімкненому контурі відповідає очікуванням ще до автопілота.
- Цільові льотні характеристики й характеристики власних рухів досягнуто відносно РОН та MIL-STD-1797 / Cooper-Harper.

- Поводиться коректно в турбулентності та в розкиді похідних за методом Монте-Карло.
- Імена властивостей на стандартних шляхах; єдиний власник на кожну команду керування; узгоджений dt з хостом.
- Походження кожного коефіцієнта зафіксоване, тож майбутню розбіжність можна простежити до її джерела.

Достовірність — це не один великий секрет; це сотня маленьких правильностей. Дерево властивостей робить кожну з них спостережною — інструментуйте, порівнюйте, коригуйте, повторюйте.

Розділ 70: Додаткова література та авторитетні джерела

Наведений нижче перелік — це впорядкований набір джерел, на які найчастіше посилаються цей посібник і сам вихідний код JSBSim.

70.1 Аеродинаміка та теорія профілю

- Anderson, J. D. Jr. Fundamentals of Aerodynamics, 6th ed., McGraw-Hill, 2017.
- Houghton, E. L., Carpenter, P. W. Aerodynamics for Engineering Students, 7th ed., Butterworth-Heinemann, 2017.
- Drela, M. Flight Vehicle Aerodynamics, MIT Press, 2014.
- McCormick, B. W. Aerodynamics, Aeronautics and Flight Mechanics, 2nd ed., Wiley, 1994.
- Schlichting, H. and Gersten, K. Boundary-Layer Theory, 9th ed., Springer, 2017.
- Abbott, I. H., von Doenhoff, A. E. Theory of Wing Sections, Dover, 1959. — довідник з профілів NACA.

70.2 Динаміка польоту та керування

- Stevens, B. L., Lewis, F. L., Johnson, E. N. Aircraft Control and Simulation, 3rd ed., Wiley, 2015.
- Etkin, B., Reid, L. D. Dynamics of Flight: Stability and Control, 3rd ed., Wiley, 1996.
- Nelson, R. C. Flight Stability and Automatic Control, 2nd ed., McGraw-Hill, 1998.
- Cook, M. V. Flight Dynamics Principles, 3rd ed., Butterworth-Heinemann, 2012.
- Roskam, J. Airplane Flight Dynamics and Automatic Flight Controls, DARcorp, 2003. Восьмитомний комплект Airplane Design Роскама — інженерна біблія.
- USAF Stability and Control DATCOM, AFFDL-TR-79-3032 (1978). Напівемпіричні методи оцінювання для кожної похідної.

70.3 CFD та моделювання турбулентності

- Wilcox, D. C. Turbulence Modeling for CFD, 3rd ed., DCW Industries, 2006.
- Pope, S. B. Turbulent Flows, Cambridge, 2000.
- Hirsch, C. Numerical Computation of Internal and External Flows, 2nd ed., Butterworth-Heinemann, 2007.
- Spalart, P. R., Allmaras, S. R. "A One-Equation Turbulence Model for Aerodynamic Flows." AIAA 92-0439.
- Menter, F. R. "Two-Equation Eddy-Viscosity Turbulence Models for Engineering Applications." AIAA Journal 32, 1994.
- NASA Turbulence Modeling Resource — turbmodels.larc.nasa.gov.

70.4 Атмосфера та геодезія

- NASA-TM-X-74335 / NOAA-S/T 76-1562. U.S. Standard Atmosphere, 1976.
- NIMA / NGA TR 8350.2. Department of Defense World Geodetic System 1984.
- Torge, W., Müller, J. Geodesy, 4th ed., de Gruyter, 2012.
- Hofmann-Wellenhof, B., Lichtenegger, H., Wasle, E. GNSS — Global Navigation Satellite Systems, Springer, 2008.

- Vallado, D. A. Fundamentals of Astrodynamics and Applications, 4th ed., Microcosm Press, 2013.
- NCEI, BGS, NGA. World Magnetic Model 2025 Technical Report, 2025.

70.5 Силіві установки

- Mattingly, J. D. Aircraft Engine Design, 2nd ed., AIAA, 2002.
- Hill, P. G., Peterson, C. R. Mechanics and Thermodynamics of Propulsion, 2nd ed., Addison-Wesley, 1992.
- Sutton, G. P., Biblarz, O. Rocket Propulsion Elements, 9th ed., Wiley, 2017.
- Leishman, J. G. Principles of Helicopter Aerodynamics, 2nd ed., Cambridge, 2006.

70.6 Чисельні методи

- Hairer, E., Nørsett, S. P., Wanner, G. Solving Ordinary Differential Equations I (нежорсткі), 2nd ed., Springer, 1993.
- Diebel, J. "Representing Attitude." Stanford Univ. report, 2006.
- Shoemake, K. "Animating Rotation with Quaternion Curves." SIGGRAPH 1985.
- Buss, S. "Accurate and Efficient Simulation of Rigid Body Rotations." UCSD, 1999.
- Catto, E. "Iterative Dynamics with Temporal Coherence." Crystal Dynamics tech. report, 2005.

70.7 Ресурси щодо JSBSim

- Berndt, J. S. "JSBSim: An Open Source Flight Dynamics Model in C++." AIAA-2004-4923. PDF на jsbsim.sourceforge.net.
- Онлайн-посібник: jsbsim-team.github.io/jsbsim-reference-manual.
- API Doxygen: jsbsim-team.github.io/jsbsim.
- Контрольні приклади NASA NESC з 6 ступенями свободи: nescacademy.nasa.gov/flightsim/2015.
- Реалізації тестових прикладів NASA для JSBSim: github.com/open-aerospace/jsbsim-nasa-test-cases.
- Огляд на DeepWiki: deepwiki.com/JSBSim-Team/jsbsim.
- Вікі FlightGear (багато практичних нотаток щодо JSBSim): wiki.flightgear.org/JSBSim.

70.8 OpenFOAM та робочий процес CFD→FDM

- OpenFOAM User Guide — функціональні об'єкти forces та forceCoeffs: openfoam.com/documentation/guides (ESI) та cpp.openfoam.org (Foundation).
- Greenshields, C. OpenFOAM User Guide, OpenCFD/CFD Direct. Підручники (tutorials) snappyHexMesh, simpleFoam та pimpleFoam (зокрема RAS/wingMotion) — це канонічні відправні точки.
- Menter, F. R. "Two-Equation Eddy-Viscosity Turbulence Models for Engineering Applications." AIAA Journal 32(8), 1994 — модель k- ω SST, що використовується для зовнішньої аеродинаміки.
- Da Ronch, A., et al. "Estimation of Dynamic Stability Derivatives Using Computational Fluid Dynamics." — методи обертальної системи відліку та вимушених коливань.
- Mi, B., et al. "Estimation and Separation of Longitudinal Dynamic Stability Derivatives with the Forced Oscillation Method Using CFD." Aerospace 8(11):354, 2021 — розділення C_{mq} та $C_{m\dot{\alpha}}$ через вертикальний (plunge) і тангажний рухи.

- Tobak, M., Schiff, L. B. "Aerodynamic Mathematical Modeling — Basic Concepts." AGARD LS-114, 1981 — теорія індиціальних відгуків (indicial-response), що лежить в основі динамічних похідних.
- NASA Turbulence Modeling Resource (turbmodels.larc.nasa.gov) та семінари AIAA Drag Prediction / High-Lift Prediction Workshops — валідаційні приклади для CFD літаків.
- Roache, P. J. "Quantification of Uncertainty in Computational Fluid Dynamics." Annu. Rev. Fluid Mech. 29, 1997 — індекс збіжності сітки (Grid Convergence Index) для перевірки незалежності від сітки.
- PyFoam, foamlib і Python-модуль JSBSim — скриптування параметричного перебору та верифікація згенерованої моделі.

70.9 Побудова FDM, джерела даних і валідація

- Berndt, J. S. "JSBSim: An Open Source Flight Dynamics Model in C++." AIAA 2004-4923 — канонічна стаття про це ядро.
- Berndt, J. S., De Marco, A. "Progress on and Usage of the Open Source Flight Dynamics Model Software Library, JSBSim." AIAA 2009-5699.
- JSBSim Reference Manual (jsbsim.sourceforge.net/JSBSimReferenceManual.pdf) та онлайн-посібник на jsbsim-team.github.io/jsbsim-reference-manual.
- Wiki FlightGear: JSBSim, JSBSim Aerodynamics, JSBSim Engines, JSBSim Thrusters, JSBSim GroundReactions — практична база знань спільноти.
- "A Journal for the Creation and Refinement of a JSBSim Aircraft Flight Model" — детальний коментований щоденник побудови моделі.
- Aeromatic / aeromatic++ (jsbsim.sourceforge.net/aeromatic2.html; вихідний код у [utils/aeromatic++/](https://github.com/jsbsim-team/aeromatic++)).
- USAF Stability and Control DATCOM, AFFDL-TR-79-3032; Digital DATCOM (holycows.net/datcom) з експортом у XML для JSBSim.
- Drela, M., Youngren, H. AVL (метод вихрової решітки); XFLR5; OpenVSP / VSPAero — попередня аеродинаміка та похідні.
- Roskam, J. Airplane Design (Part V: ваги компонентів і моменти інерції) — радіуси інерції за класами.
- Cooper, G. E., Harper, R. P. "The Use of Pilot Rating in the Evaluation of Aircraft Handling Qualities." NASA TN D-5153, 1969.
- MIL-STD-1797 / MIL-F-8785C — вимоги до пілотажних характеристик і модальні критерії, яким має відповідати реалістична модель.
- Klein, V., Morelli, E. A. Aircraft System Identification: Theory and Practice, AIAA, 2006 — виділення похідних із даних льотних (та JSBSim) випробувань.

Розділ 71: Глосарій та показчик позначень

Позначення	Значення
α (alpha)	Кут атаки — кут між зв'язаною віссю X та проекцією відносного вітру на зв'язану площину X-Z.
β (beta)	Кут ковзання — кут між відносним вітром та зв'язаною площиною X-Z.
$\dot{\alpha}$ (alphadot)	Швидкість зміни α в часі.
$\bar{q}$ (qbar)	Швидкісний напір, $\frac{1}{2}\rho V^2$.
ρ (rho)	Густина атмосфери.
V_t	Істинна повітряна швидкість (модуль швидкості відносно повітряної маси).
V_c	Приладова виправлена повітряна швидкість (приладова швидкість з поправкою на стисливість).
V_e	Еквівалентна повітряна швидкість (приладова швидкість з поправкою на густину).
M	Число Маха, V_t/a .
a	Швидкість звуку.
p, q, r	Кутові швидкості крену, тангажа, рискання в зв'язаній системі.
u, v, w	Складові поступальної швидкості в зв'язаній системі.
ϕ, θ, ψ	Кути Ейлера крену, тангажа, курсу (послідовність 3-2-1).
γ (gamma)	Кут траєкторії польоту (додатний — набір висоти).
S	Опорна площа крила.
b	Розмах крила.
$\bar{c}$ (cbar)	Середня аеродинамічна хорда.
AR	Видовження, b^2/S .
e	Коефіцієнт ефективності Освальда (зазвичай 0.7-0.95).
AERORP	Аеродинамічна опорна точка — початок відліку для аеродинамічних моментів.
VRP	Візуальна опорна точка — початок відліку, який використовують зовнішні переглядачі.
EYEPOINT	Положення очей пілота.
AGL	Над рівнем землі.
MSL	Середній рівень моря.
WGS-84	Всесвітня геодезична система 1984 — модель земного еліпсоїда, яку використовують GPS і JSBSim.
ECEF	Геоцентрична, пов'язана із Землею (обертається разом із планетою).
ECI	Геоцентрична інерціальна (необертова).
NED	Місцева дотична площина «північ-схід-вниз».
LCP	Задача лінійної доповнюваності — формулювання тертя в контакті.
FDM	Модель динаміки польоту.
FCS	Система керування польотом.
IC	Початкові умови.
BSFC	Питома витрата пального на гальмівній потужності.
MAP	Абсолютний тиск у впускному колекторі (поршневі двигуни).
N1, N2	Частоти обертання роторів низького та високого тиску (турбінні двигуни).
TSFC	Питома витрата пального на одиницю тяги (турбіни).

CFD	Обчислювальна гідрогазодинаміка.
RANS	Осереднені за Рейнольдсом рівняння Нав'є-Стокса.
URANS	Нестационарні RANS.
LES	Моделювання великих вихорів.
AVL	Athena Vortex Lattice (Drela).
DATCOM	Довідник USAF з стійкості та керованості DATCOM.
POH	Посібник з льотної експлуатації.
TAS	Істинна повітряна швидкість ($=V_t$).
IAS	Приладова повітряна швидкість.
CAS	Приладова виправлена повітряна швидкість ($=V_c$).
EAS	Еквівалентна повітряна швидкість ($=V_e$).
KCAS / KTAS / KIAS	Приладова виправлена/істинна/приладова повітряна швидкість у вузлах.

Кінець вичерпного довідника JSBSim. Повідомлення про помилки та вдосконалення вітаються за адресою <https://github.com/JSBSim-Team/jsbsim>.